

Aroplane Fuel Manager - Full Project Code

File: backend/app.py

```
"""
Aircraft Fuel Cost & Trip Management System - Flask Application
Main application entry point with Blueprint registration
"""

import os
from flask import Flask
from flask_cors import CORS
from flask_jwt_extended import JWTManager

from config import Config
from extensions import db, ma
from blueprints.auth import auth_bp
from blueprints.aircraft import aircraft_bp
from blueprints.fuel_purchases import fuel_purchases_bp
from blueprints.flights import flights_bp
from blueprints.fuel_transactions import fuel_transactions_bp
from blueprints.daily_usage import daily_usage_bp
from blueprints.dashboard import dashboard_bp
from blueprints.reports import reports_bp

def create_app(config_class=Config):
    """Application factory function."""
    app = Flask(__name__)
    app.config.from_object(config_class)
    app.url_map.strict_slashes = False

    # Initialize extensions
    db.init_app(app)
    ma.init_app(app)
    JWTManager(app)
    CORS(app, origins=["http://localhost:3000", "http://127.0.0.1:3000", "http://localho

    # Register Blueprints
    app.register_blueprint(auth_bp, url_prefix='/api/auth')
```

```

app.register_blueprint(aircraft_bp, url_prefix='/api/aircraft')
app.register_blueprint(fuel_purchases_bp, url_prefix='/api/fuel-purchases')
app.register_blueprint(flights_bp, url_prefix='/api/flights')
app.register_blueprint(fuel_transactions_bp, url_prefix='/api/fuel-transactions')
app.register_blueprint(daily_usage_bp, url_prefix='/api/daily-usage')
app.register_blueprint/dashboard_bp, url_prefix='/api/dashboard')
app.register_blueprint(reports_bp, url_prefix='/api/reports')

# Health check endpoint
@app.route('/api/health')
def health_check():
    return {'status': 'healthy', 'message': 'Aircraft Fuel Management API is running'}

with app.app_context():
    db.create_all()
    # Seed initial data
    from seed import seed_initial_data
    seed_initial_data()

return app

if __name__ == '__main__':
    app = create_app()
    app.run(host='0.0.0.0', port=5000, debug=True)

```

File: backend/config.py

```

"""
Application Configuration
Loads environment variables and configures Flask/SQLAlchemy settings
"""

import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    """Base configuration class."""
    SECRET_KEY = os.environ.get('JWT_SECRET_KEY', 'dev-secret-key')
    JWT_SECRET_KEY = os.environ.get('JWT_SECRET_KEY', 'dev-secret-key')
    SQLALCHEMY_DATABASE_URI = os.environ.get(

```

```

        'DATABASE_URL',
        'postgresql://postgres:password@localhost:5432/aroplane_fuel'
    )
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    SQLALCHEMY_ENGINE_OPTIONS = {
        'pool_pre_ping': True,
        'pool_recycle': 300,
    }
    JWT_ACCESS_TOKEN_EXPIRES = 86400 # 24 hours in seconds

class DevelopmentConfig(Config):
    DEBUG = True

class ProductionConfig(Config):
    DEBUG = False

```

File: backend/extensions.py

```

"""
Shared Flask extensions - initialized here to avoid circular imports.
"""

from flask_sqlalchemy import SQLAlchemy
from flask_marshmallow import Marshmallow

db = SQLAlchemy()
ma = Marshmallow()

```

File: backend/models.py

```

"""
Database Models for Aircraft Fuel Cost & Trip Management System
All SQLAlchemy ORM models are defined here with proper relationships.
"""

from datetime import datetime, timezone
from werkzeug.security import generate_password_hash, check_password_hash
from extensions import db

```

```

class User(db.Model):
    """User model supporting Admin and Operator roles."""
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(256), nullable=False)
    role = db.Column(db.String(20), nullable=False, default='operator') # admin | opera
    is_active = db.Column(db.Boolean, default=True)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))
    updated_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc),
                           onupdate=lambda: datetime.now(timezone.utc))

    def set_password(self, password):
        self.password_hash = generate_password_hash(password)

    def check_password(self, password):
        return check_password_hash(self.password_hash, password)

    def to_dict(self):
        return {
            'id': self.id,
            'username': self.username,
            'email': self.email,
            'role': self.role,
            'is_active': self.is_active,
            'created_at': self.created_at.isoformat() if self.created_at else None
        }

class FuelSupplier(db.Model):
    """Fuel supplier information."""
    __tablename__ = 'fuel_suppliers'

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    contact_email = db.Column(db.String(120))
    contact_phone = db.Column(db.String(20))
    address = db.Column(db.Text)
    is_active = db.Column(db.Boolean, default=True)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))

    # Relationships
    fuel_purchases = db.relationship('FuelPurchase', back_populates='supplier', lazy='dy

    def to_dict(self):

```

```

    return {
        'id': self.id,
        'name': self.name,
        'contact_email': self.contact_email,
        'contact_phone': self.contact_phone,
        'address': self.address,
        'is_active': self.is_active,
        'created_at': self.created_at.isoformat() if self.created_at else None
    }

```

```
class FuelPurchase(db.Model):
```

```
    """
```

```
    Records each fuel purchase event including supplier, quantity, price, and location.
    Also drives current fuel price and inventory levels.
```

```
    """
```

```
    __tablename__ = 'fuel_purchases'
```

```
    id = db.Column(db.Integer, primary_key=True)
```

```
    supplier_id = db.Column(db.Integer, db.ForeignKey('fuel_suppliers.id'), nullable=True)
```

```
    supplier_name = db.Column(db.String(100), nullable=False) # Denormalized for quick
```

```
    location = db.Column(db.String(150), nullable=False) # Airport or city
```

```
    fuel_type = db.Column(db.String(20), nullable=False) # Jet A1 | Avgas
```

```
    quantity_liters = db.Column(db.Float, nullable=False)
```

```
    price_per_liter = db.Column(db.Float, nullable=False)
```

```
    total_cost = db.Column(db.Float, nullable=False)
```

```
    purchase_date = db.Column(db.Date, nullable=False)
```

```
    invoice_number = db.Column(db.String(50))
```

```
    notes = db.Column(db.Text)
```

```
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))
```

```
    created_by = db.Column(db.Integer, db.ForeignKey('users.id'))
```

```
    # Relationships
```

```
    supplier = db.relationship('FuelSupplier', back_populates='fuel_purchases')
```

```
    def to_dict(self):
```

```
        return {
```

```
            'id': self.id,
```

```
            'supplier_id': self.supplier_id,
```

```
            'supplier_name': self.supplier_name,
```

```
            'location': self.location,
```

```
            'fuel_type': self.fuel_type,
```

```
            'quantity_liters': self.quantity_liters,
```

```
            'price_per_liter': self.price_per_liter,
```

```
            'total_cost': self.total_cost,
```

```
            'purchase_date': self.purchase_date.isoformat() if self.purchase_date else N
```

```
            'invoice_number': self.invoice_number,
```

```

        'notes': self.notes,
        'created_at': self.created_at.isoformat() if self.created_at else None
    }

class FuelInventory(db.Model):
    """
    Tracks current fuel inventory levels per fuel type.
    Updated whenever fuel is purchased or consumed.
    """
    __tablename__ = 'fuel_inventory'

    id = db.Column(db.Integer, primary_key=True)
    fuel_type = db.Column(db.String(20), nullable=False, unique=True) # Jet A1 | Avgas
    total_quantity_liters = db.Column(db.Float, default=0.0)
    updated_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc),
                           onupdate=lambda: datetime.now(timezone.utc))

    def to_dict(self):
        return {
            'id': self.id,
            'fuel_type': self.fuel_type,
            'total_quantity_liters': self.total_quantity_liters,
            'updated_at': self.updated_at.isoformat() if self.updated_at else None
        }

class Aircraft(db.Model):
    """
    Aircraft registry with fuel capacity and consumption characteristics.
    """
    __tablename__ = 'aircraft'

    id = db.Column(db.Integer, primary_key=True)
    aircraft_id = db.Column(db.String(20), unique=True, nullable=False) # e.g. VT-AKC
    model = db.Column(db.String(100), nullable=False)
    manufacturer = db.Column(db.String(100))
    year = db.Column(db.Integer)
    fuel_tank_capacity_liters = db.Column(db.Float, nullable=False)
    fuel_consumption_rate = db.Column(db.Float, nullable=False) # liters per km
    fuel_type = db.Column(db.String(20), nullable=False, default='Jet A1')
    max_range_km = db.Column(db.Float)
    is_active = db.Column(db.Boolean, default=True)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))

    # Relationships
    flights = db.relationship('Flight', back_populates='aircraft', lazy='dynamic')

```

```

def to_dict(self):
    return {
        'id': self.id,
        'aircraft_id': self.aircraft_id,
        'model': self.model,
        'manufacturer': self.manufacturer,
        'year': self.year,
        'fuel_tank_capacity_liters': self.fuel_tank_capacity_liters,
        'fuel_consumption_rate': self.fuel_consumption_rate,
        'fuel_type': self.fuel_type,
        'max_range_km': self.max_range_km,
        'is_active': self.is_active,
        'created_at': self.created_at.isoformat() if self.created_at else None
    }

```

```

class Flight(db.Model):
    """
    Flight trip record. Auto-calculates required fuel based on
    aircraft consumption rate and trip distance.
    """
    __tablename__ = 'flights'

    id = db.Column(db.Integer, primary_key=True)
    flight_number = db.Column(db.String(20), unique=True, nullable=False)
    aircraft_id_fk = db.Column(db.Integer, db.ForeignKey('aircraft.id'), nullable=False)
    source = db.Column(db.String(100), nullable=False)
    destination = db.Column(db.String(100), nullable=False)
    distance_km = db.Column(db.Float, nullable=False)
    required_fuel_liters = db.Column(db.Float, nullable=False) # Auto-calculated
    actual_fuel_used_liters = db.Column(db.Float) # Updated after trip
    fuel_efficiency = db.Column(db.Float) # actual/required ratio
    status = db.Column(db.String(20), default='scheduled') # scheduled | in_flight
    departure_time = db.Column(db.DateTime)
    arrival_time = db.Column(db.DateTime)
    flight_date = db.Column(db.Date)
    fuel_price_at_time = db.Column(db.Float) # Price snapshot at tri
    trip_fuel_cost = db.Column(db.Float) # Calculated cost
    notes = db.Column(db.Text)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))
    created_by = db.Column(db.Integer, db.ForeignKey('users.id'))

    # Relationships
    aircraft = db.relationship('Aircraft', back_populates='flights')
    fuel_transactions = db.relationship('FuelTransaction', back_populates='flight', lazy

```

```

def to_dict(self):
    return {
        'id': self.id,
        'flight_number': self.flight_number,
        'aircraft_id_fk': self.aircraft_id_fk,
        'aircraft': self.aircraft.to_dict() if self.aircraft else None,
        'source': self.source,
        'destination': self.destination,
        'distance_km': self.distance_km,
        'required_fuel_liters': self.required_fuel_liters,
        'actual_fuel_used_liters': self.actual_fuel_used_liters,
        'fuel_efficiency': self.fuel_efficiency,
        'status': self.status,
        'departure_time': self.departure_time.isoformat() if self.departure_time else None,
        'arrival_time': self.arrival_time.isoformat() if self.arrival_time else None,
        'flight_date': self.flight_date.isoformat() if self.flight_date else None,
        'fuel_price_at_time': self.fuel_price_at_time,
        'trip_fuel_cost': self.trip_fuel_cost,
        'notes': self.notes,
        'created_at': self.created_at.isoformat() if self.created_at else None
    }

```

```

class FuelTransaction(db.Model):

```

```

    """

```

```

    Records every fuel allocation or consumption event.

```

```

    Maintains full transaction history for auditing.

```

```

    transaction_type: 'allocation' | 'consumption' | 'refund'

```

```

    """

```

```

    __tablename__ = 'fuel_transactions'

```

```

    id = db.Column(db.Integer, primary_key=True)

```

```

    flight_id = db.Column(db.Integer, db.ForeignKey('flights.id'), nullable=True)

```

```

    fuel_type = db.Column(db.String(20), nullable=False)

```

```

    quantity_liters = db.Column(db.Float, nullable=False)

```

```

    transaction_type = db.Column(db.String(20), nullable=False) # allocation | consumption

```

```

    price_per_liter = db.Column(db.Float)

```

```

    total_cost = db.Column(db.Float)

```

```

    transaction_date = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))

```

```

    notes = db.Column(db.Text)

```

```

    created_by = db.Column(db.Integer, db.ForeignKey('users.id'))

```

```

    # Relationships

```

```

    flight = db.relationship('Flight', back_populates='fuel_transactions')

```

```

    def to_dict(self):

```

```

        return {

```



```

        'id': self.id,
        'flight_id': self.flight_id,
        'flight_number': self.flight.flight_number if self.flight else None,
        'fuel_type': self.fuel_type,
        'quantity_liters': self.quantity_liters,
        'transaction_type': self.transaction_type,
        'price_per_liter': self.price_per_liter,
        'total_cost': self.total_cost,
        'transaction_date': self.transaction_date.isoformat() if self.transaction_da
        'notes': self.notes
    }

class DailyFuelUsage(db.Model):
    """
    Aggregated daily fuel usage summary.
    Generated/updated at end of day or on demand.
    """
    __tablename__ = 'daily_fuel_usage'

    id = db.Column(db.Integer, primary_key=True)
    usage_date = db.Column(db.Date, unique=True, nullable=False)
    total_fuel_used_liters = db.Column(db.Float, default=0.0)
    total_flights = db.Column(db.Integer, default=0)
    total_fuel_cost = db.Column(db.Float, default=0.0)
    jet_a1_used = db.Column(db.Float, default=0.0)
    avgas_used = db.Column(db.Float, default=0.0)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))
    updated_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc),
                           onupdate=lambda: datetime.now(timezone.utc))

    def to_dict(self):
        return {
            'id': self.id,
            'usage_date': self.usage_date.isoformat() if self.usage_date else None,
            'total_fuel_used_liters': self.total_fuel_used_liters,
            'total_flights': self.total_flights,
            'total_fuel_cost': self.total_fuel_cost,
            'jet_a1_used': self.jet_a1_used,
            'avgas_used': self.avgas_used
        }

```

File: backend/blueprints/aircraft.py

```

"""
Aircraft Blueprint
CRUD operations for aircraft registration and management.
"""

from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt
from extensions import db
from models import Aircraft

aircraft_bp = Blueprint('aircraft', __name__)

@aircraft_bp.route('/', methods=['GET'])
@jwt_required()
def list_aircraft():
    """List all aircraft with optional active filter."""
    active_only = request.args.get('active', 'false').lower() == 'true'
    query = Aircraft.query
    if active_only:
        query = query.filter_by(is_active=True)
    aircraft_list = query.order_by(Aircraft.aircraft_id).all()
    return jsonify([a.to_dict() for a in aircraft_list]), 200

@aircraft_bp.route('/<int:aircraft_id>', methods=['GET'])
@jwt_required()
def get_aircraft(aircraft_id):
    """Get a single aircraft by ID."""
    aircraft = Aircraft.query.get_or_404(aircraft_id)
    return jsonify(aircraft.to_dict()), 200

@aircraft_bp.route('/', methods=['POST'])
@jwt_required()
def create_aircraft():
    """Create a new aircraft record (Admin only)."""
    claims = get_jwt()
    if claims.get('role') != 'admin':
        return jsonify({'error': 'Admin access required'}), 403

    data = request.get_json()
    required = ['aircraft_id', 'model', 'fuel_tank_capacity_liters', 'fuel_consumption_r
    for field in required:
        if data.get(field) is None:
            return jsonify({'error': f'{field} is required'}), 400

```

```

if Aircraft.query.filter_by(aircraft_id=data['aircraft_id']).first():
    return jsonify({'error': 'Aircraft ID already exists'}), 409

aircraft = Aircraft(
    aircraft_id=data['aircraft_id'],
    model=data['model'],
    manufacturer=data.get('manufacturer'),
    year=data.get('year'),
    fuel_tank_capacity_liters=float(data['fuel_tank_capacity_liters']),
    fuel_consumption_rate=float(data['fuel_consumption_rate']),
    fuel_type=data.get('fuel_type', 'Jet A1'),
    max_range_km=data.get('max_range_km')
)
db.session.add(aircraft)
db.session.commit()
return jsonify(aircraft.to_dict()), 201

@aircraft_bp.route('/<int:aircraft_id>', methods=['PUT'])
@jwt_required()
def update_aircraft(aircraft_id):
    """Update an aircraft record."""
    claims = get_jwt()
    if claims.get('role') != 'admin':
        return jsonify({'error': 'Admin access required'}), 403

    aircraft = Aircraft.query.get_or_404(aircraft_id)
    data = request.get_json()

    aircraft.model = data.get('model', aircraft.model)
    aircraft.manufacturer = data.get('manufacturer', aircraft.manufacturer)
    aircraft.year = data.get('year', aircraft.year)
    aircraft.fuel_tank_capacity_liters = float(data.get('fuel_tank_capacity_liters', aircraft.fuel_tank_capacity_liters))
    aircraft.fuel_consumption_rate = float(data.get('fuel_consumption_rate', aircraft.fuel_consumption_rate))
    aircraft.fuel_type = data.get('fuel_type', aircraft.fuel_type)
    aircraft.max_range_km = data.get('max_range_km', aircraft.max_range_km)
    aircraft.is_active = data.get('is_active', aircraft.is_active)

    db.session.commit()
    return jsonify(aircraft.to_dict()), 200

@aircraft_bp.route('/<int:aircraft_id>', methods=['DELETE'])
@jwt_required()
def delete_aircraft(aircraft_id):
    """Soft-delete an aircraft (marks as inactive)."""
    claims = get_jwt()

```

```

    if claims.get('role') != 'admin':
        return jsonify({'error': 'Admin access required'}), 403

    aircraft = Aircraft.query.get_or_404(aircraft_id)
    aircraft.is_active = False
    db.session.commit()
    return jsonify({'message': f'Aircraft {aircraft.aircraft_id} deactivated'}), 200

@aircraft_bp.route('/<int:aircraft_id>/stats', methods=['GET'])
@jwt_required()
def aircraft_stats(aircraft_id):
    """Get consumption statistics for a specific aircraft."""
    from models import Flight
    aircraft = Aircraft.query.get_or_404(aircraft_id)

    completed_flights = Flight.query.filter_by(
        aircraft_id_fk=aircraft_id, status='completed'
    ).all()

    total_fuel_used = sum(f.actual_fuel_used_liters or 0 for f in completed_flights)
    total_distance = sum(f.distance_km for f in completed_flights)
    total_cost = sum(f.trip_fuel_cost or 0 for f in completed_flights)

    return jsonify({
        'aircraft': aircraft.to_dict(),
        'total_flights': len(completed_flights),
        'total_fuel_used_liters': round(total_fuel_used, 2),
        'total_distance_km': round(total_distance, 2),
        'total_fuel_cost': round(total_cost, 2),
        'avg_consumption_rate': round(total_fuel_used / total_distance, 4) if total_dist
    }), 200

```

File: backend/blueprints/auth.py

```

"""
Authentication Blueprint
Handles user login, registration, and profile management with JWT tokens.
"""
from flask import Blueprint, request, jsonify
from flask_jwt_extended import (
    create_access_token, jwt_required, get_jwt_identity, get_jwt
)

```

```
from extensions import db
from models import User

auth_bp = Blueprint('auth', __name__)

@auth_bp.route('/login', methods=['POST'])
def login():
    """
    Login endpoint.
    Request: { username, password }
    Response: { access_token, user }
    """
    data = request.get_json()
    if not data or not data.get('username') or not data.get('password'):
        return jsonify({'error': 'Username and password are required'}), 400

    user = User.query.filter_by(username=data['username']).first()
    if not user or not user.check_password(data['password']):
        return jsonify({'error': 'Invalid credentials'}), 401

    if not user.is_active:
        return jsonify({'error': 'Account is deactivated'}), 403

    additional_claims = {'role': user.role, 'email': user.email}
    access_token = create_access_token(
        identity=str(user.id),
        additional_claims=additional_claims
    )

    return jsonify({
        'access_token': access_token,
        'user': user.to_dict()
    }), 200

@auth_bp.route('/me', methods=['GET'])
@jwt_required()
def get_profile():
    """Returns the profile of the currently authenticated user."""
    user_id = get_jwt_identity()
    user = User.query.get(int(user_id))
    if not user:
        return jsonify({'error': 'User not found'}), 404
    return jsonify(user.to_dict()), 200
```

```
@auth_bp.route('/users', methods=['GET'])
@jwt_required()
def list_users():
    """List all users (Admin only)."""
    claims = get_jwt()
    if claims.get('role') != 'admin':
        return jsonify({'error': 'Admin access required'}), 403
    users = User.query.all()
    return jsonify([u.to_dict() for u in users]), 200


@auth_bp.route('/users', methods=['POST'])
@jwt_required()
def create_user():
    """Create a new user (Admin only)."""
    claims = get_jwt()
    if claims.get('role') != 'admin':
        return jsonify({'error': 'Admin access required'}), 403

    data = request.get_json()
    required = ['username', 'password', 'role']
    for field in required:
        if not data.get(field):
            return jsonify({'error': f'{field} is required'}), 400

    # Auto-generate email if not provided
    email = data.get('email') or f"{data['username']}@aerofuel.internal"

    if User.query.filter_by(username=data['username']).first():
        return jsonify({'error': 'Username already exists'}), 409

    if User.query.filter_by(email=email).first():
        # Try a unique variant
        import time
        email = f"{data['username']}.{int(time.time())}@aerofuel.internal"

    if data['role'] not in ['admin', 'operator']:
        return jsonify({'error': 'Role must be admin or operator'}), 400

    user = User(username=data['username'], email=email, role=data['role'])
    user.set_password(data['password'])
    db.session.add(user)
    db.session.commit()

    return jsonify(user.to_dict()), 201
```

```

@auth_bp.route('/change-password', methods=['POST'])
@jwt_required()
def change_password():
    """Change password for authenticated user."""
    user_id = get_jwt_identity()
    data = request.get_json()

    if not data.get('current_password') or not data.get('new_password'):
        return jsonify({'error': 'Current and new password are required'}), 400

    user = User.query.get(int(user_id))
    if not user.check_password(data['current_password']):
        return jsonify({'error': 'Current password is incorrect'}), 401

    user.set_password(data['new_password'])
    db.session.commit()
    return jsonify({'message': 'Password changed successfully'}), 200

```

File: backend/blueprints/daily_usage.py

```

"""
Daily Usage Blueprint
Aggregate daily fuel consumption logs and trends.
"""

from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required
from models import DailyFuelUsage

daily_usage_bp = Blueprint('daily_usage', __name__)

@daily_usage_bp.route('/', methods=['GET'])
@jwt_required()
def list_daily_usage():
    """
    Returns daily fuel usage records.
    Query params: start_date, end_date, days (default: 30)
    """
    days = int(request.args.get('days', 30))
    start_date = request.args.get('start_date')
    end_date = request.args.get('end_date')

    query = DailyFuelUsage.query

```

```

    if start_date:
        query = query.filter(DailyFuelUsage.usage_date >= start_date)
    if end_date:
        query = query.filter(DailyFuelUsage.usage_date <= end_date)

    records = query.order_by(DailyFuelUsage.usage_date.desc()).limit(days).all()
    records.reverse()

    return jsonify([r.to_dict() for r in records]), 200

@daily_usage_bp.route('/summary', methods=['GET'])
@jwt_required()
def usage_summary():
    """Overall summary of daily usage statistics."""
    from sqlalchemy import func
    from extensions import db

    result = db.session.query(
        func.sum(DailyFuelUsage.total_fuel_used_liters).label('total_fuel'),
        func.sum(DailyFuelUsage.total_flights).label('total_flights'),
        func.sum(DailyFuelUsage.total_fuel_cost).label('total_cost'),
        func.avg(DailyFuelUsage.total_fuel_used_liters).label('avg_daily_fuel'),
        func.count(DailyFuelUsage.id).label('total_days')
    ).first()

    return jsonify({
        'total_fuel_used_liters': round(result.total_fuel or 0, 2),
        'total_flights': result.total_flights or 0,
        'total_fuel_cost': round(result.total_cost or 0, 2),
        'avg_daily_fuel_liters': round(result.avg_daily_fuel or 0, 2),
        'total_days_tracked': result.total_days or 0
    }), 200

```

File: backend/blueprints/dashboard.py

```

"""
Dashboard Blueprint
Aggregated KPIs and summary data for the main dashboard.
"""

from flask import Blueprint, jsonify
from flask_jwt_extended import jwt_required
from sqlalchemy import func, desc

```



```
from datetime import date, timedelta
from extensions import db
from models import (FuelInventory, FuelPurchase, Flight, FuelTransaction,
                    DailyFuelUsage, Aircraft)

dashboard_bp = Blueprint('dashboard', __name__)

@dashboard_bp.route('/kpis', methods=['GET'])
@jwt_required()
def get_kpis():
    """
    Returns core KPIs for the dashboard:
    - Current fuel stock (per type)
    - Current fuel price
    - Today's fuel usage
    - Total flights this month
    - Monthly fuel cost
    """
    today = date.today()
    month_start = today.replace(day=1)

    # Current inventory
    inventories = FuelInventory.query.all()
    stock = {inv.fuel_type: inv.total_quantity_liters for inv in inventories}

    # Current prices
    jet_price = FuelPurchase.query.filter_by(fuel_type='Jet A1').order_by(
        desc(FuelPurchase.purchase_date), desc(FuelPurchase.id)
    ).first()
    avgas_price = FuelPurchase.query.filter_by(fuel_type='Avgas').order_by(
        desc(FuelPurchase.purchase_date), desc(FuelPurchase.id)
    ).first()

    # Today's usage
    today_usage = DailyFuelUsage.query.filter_by(usage_date=today).first()

    # Monthly stats
    monthly = db.session.query(
        func.sum(DailyFuelUsage.total_fuel_used_liters).label('total_fuel'),
        func.sum(DailyFuelUsage.total_flights).label('total_flights'),
        func.sum(DailyFuelUsage.total_fuel_cost).label('total_cost')
    ).filter(DailyFuelUsage.usage_date >= month_start).first()

    # Aircraft count
    total_aircraft = Aircraft.query.filter_by(is_active=True).count()
```

```

# Active flights
active_flights = Flight.query.filter(
    Flight.status.in_(['scheduled', 'in_flight'])
).count()

# Low fuel alert (below 20% capacity threshold = 50,000L)
LOW_THRESHOLD = 50000
alerts = []
for inv in inventories:
    if inv.total_quantity_liters < LOW_THRESHOLD:
        alerts.append({
            'type': 'low_fuel',
            'fuel_type': inv.fuel_type,
            'current_level': inv.total_quantity_liters,
            'threshold': LOW_THRESHOLD,
            'message': f'{inv.fuel_type} stock is critically low ({inv.total_quantit
        })

return jsonify({
    'fuel_stock': stock,
    'current_prices': {
        'Jet A1': jet_price.price_per_liter if jet_price else None,
        'Avgas': avgas_price.price_per_liter if avgas_price else None
    },
    'today': {
        'fuel_used_liters': today_usage.total_fuel_used_liters if today_usage else 0
        'flights': today_usage.total_flights if today_usage else 0,
        'fuel_cost': today_usage.total_fuel_cost if today_usage else 0
    },
    'this_month': {
        'total_fuel_used_liters': round(monthly.total_fuel or 0, 2),
        'total_flights': monthly.total_flights or 0,
        'total_fuel_cost': round(monthly.total_cost or 0, 2)
    },
    'total_active_aircraft': total_aircraft,
    'active_flights': active_flights,
    'alerts': alerts
}), 200

@dashboard_bp.route('/fuel-consumption-chart', methods=['GET'])
@jwt_required()
def fuel_consumption_chart():
    """Last 30 days daily fuel usage for bar chart."""
    end_date = date.today()
    start_date = end_date - timedelta(days=29)

```

```

records = DailyFuelUsage.query.filter(
    DailyFuelUsage.usage_date >= start_date,
    DailyFuelUsage.usage_date <= end_date
).order_by(DailyFuelUsage.usage_date).all()

return jsonify([r.to_dict() for r in records]), 200

@dashboard_bp.route('/price-chart', methods=['GET'])
@jwt_required()
def price_chart():
    """Last 30 days price trend for line chart with gap filling."""
    # Get all purchases to have historical context for carrying forward prices
    purchases = FuelPurchase.query.order_by(FuelPurchase.purchase_date).all()

    # Build maps of price history
    jet_history = {}
    avgas_history = {}
    for p in purchases:
        dt_str = p.purchase_date.isoformat()
        if p.fuel_type == 'Jet A1':
            jet_history[dt_str] = p.price_per_liter
        elif p.fuel_type == 'Avgas':
            avgas_history[dt_str] = p.price_per_liter

    # Generate the last 30 dates
    today = date.today()
    date_list = [(today - timedelta(days=i)) for i in range(29, -1, -1)]

    result = []

    # For each date, find the most recent price
    for d in date_list:
        d_str = d.isoformat()

        # Get Jet A1 price (exact match or most recent historical)
        jet_p = jet_history.get(d_str)
        if jet_p is None:
            past_dates = sorted([k for k in jet_history.keys() if k < d_str], reverse=True)
            if past_dates:
                jet_p = jet_history[past_dates[0]]

        # Get Avgas price
        avgas_p = avgas_history.get(d_str)
        if avgas_p is None:
            past_dates = sorted([k for k in avgas_history.keys() if k < d_str], reverse=True)
            if past_dates:

```

```

        avgas_p = avgas_history[past_dates[0]]

    result.append({
        'date': d_str,
        'jet_a1_price': jet_p,
        'avgas_price': avgas_p
    })

    return jsonify(result), 200

@dashboard_bp.route('/aircraft-consumption', methods=['GET'])
@jwt_required()
def aircraft_consumption():
    """Per-aircraft fuel consumption for bar chart."""
    aircraft_list = Aircraft.query.filter_by(is_active=True).all()
    result = []

    for aircraft in aircraft_list:
        flights = Flight.query.filter_by(
            aircraft_id_fk=aircraft.id, status='completed'
        ).all()
        total_fuel = sum(f.actual_fuel_used_liters or 0 for f in flights)
        total_cost = sum(f.trip_fuel_cost or 0 for f in flights)

        result.append({
            'aircraft_id': aircraft.aircraft_id,
            'model': aircraft.model,
            'total_fuel_used': round(total_fuel, 2),
            'total_cost': round(total_cost, 2),
            'flight_count': len(flights)
        })

    return jsonify(result), 200

```

File: backend/blueprints/flights.py

```

"""
Flights Blueprint
Trip management with auto-calculated fuel requirements and fuel allocation.
Business Logic:
    - required_fuel = distance_km × aircraft.fuel_consumption_rate
    - trip_fuel_cost = actual_fuel_used × current_price_per_liter

```

```

- Completing a trip deducts fuel from inventory
"""

from datetime import datetime, timezone
from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt, get_jwt_identity
from sqlalchemy import desc
from extensions import db
from models import Flight, Aircraft, FuelPurchase, FuelInventory, FuelTransaction, Daily

flights_bp = Blueprint('flights', __name__)

def _get_current_price(fuel_type):
    """Helper: get the current price per liter for a fuel type."""
    latest = FuelPurchase.query.filter_by(fuel_type=fuel_type).order_by(
        desc(FuelPurchase.purchase_date), desc(FuelPurchase.id)
    ).first()
    return latest.price_per_liter if latest else 0.0

@flights_bp.route('/', methods=['GET'])
@jwt_required()
def list_flights():
    """List all flights with optional filters."""
    status = request.args.get('status')
    aircraft_id = request.args.get('aircraft_id')
    start_date = request.args.get('start_date')
    end_date = request.args.get('end_date')
    per_page = int(request.args.get('per_page', 20))
    page = int(request.args.get('page', 1))

    query = Flight.query
    if status:
        query = query.filter_by(status=status)
    if aircraft_id:
        query = query.filter_by(aircraft_id_fk=int(aircraft_id))
    if start_date:
        query = query.filter(Flight.flight_date >= start_date)
    if end_date:
        query = query.filter(Flight.flight_date <= end_date)

    pagination = query.order_by(desc(Flight.created_at)).paginate(
        page=page, per_page=per_page, error_out=False
    )

    return jsonify({
        'flights': [f.to_dict() for f in pagination.items],

```

```

        'total': pagination.total,
        'pages': pagination.pages,
        'current_page': page
    }), 200

@flights_bp.route('/<int:flight_id>', methods=['GET'])
@jwt_required()
def get_flight(flight_id):
    """Get a single flight by ID."""
    flight = Flight.query.get_or_404(flight_id)
    return jsonify(flight.to_dict()), 200

@flights_bp.route('/', methods=['POST'])
@jwt_required()
def create_flight():
    """
    Create a new flight/trip.
    Auto-calculates required_fuel = distance × aircraft.consumption_rate.
    Automatically allocates fuel from inventory.
    """
    user_id = get_jwt_identity()
    data = request.get_json()

    required = ['flight_number', 'aircraft_id_fk', 'source', 'destination', 'distance_km']
    for field in required:
        if not data.get(field) and data.get(field) != 0:
            return jsonify({'error': f'{field} is required'}), 400

    if Flight.query.filter_by(flight_number=data['flight_number']).first():
        return jsonify({'error': 'Flight number already exists'}), 409

    aircraft = Aircraft.query.get(data['aircraft_id_fk'])
    if not aircraft:
        return jsonify({'error': 'Aircraft not found'}), 404

    distance = float(data['distance_km'])
    required_fuel = round(distance * aircraft.fuel_consumption_rate, 2)
    current_price = _get_current_price(aircraft.fuel_type)

    # Check inventory availability
    inventory = FuelInventory.query.filter_by(fuel_type=aircraft.fuel_type).first()
    if not inventory or inventory.total_quantity_liters < required_fuel:
        return jsonify({
            'error': f'Insufficient {aircraft.fuel_type} inventory. '
                    f'Required: {required_fuel}L, Available: {inventory.total_quantity_

```

```

    }), 400

flight_date = datetime.strptime(data['flight_date'], '%Y-%m-%d').date()

flight = Flight(
    flight_number=data['flight_number'],
    aircraft_id_fk=aircraft.id,
    source=data['source'],
    destination=data['destination'],
    distance_km=distance,
    required_fuel_liters=required_fuel,
    status='scheduled',
    flight_date=flight_date,
    fuel_price_at_time=current_price,
    notes=data.get('notes'),
    created_by=int(user_id)
)

if data.get('departure_time'):
    flight.departure_time = datetime.fromisoformat(data['departure_time'])

db.session.add(flight)
db.session.flush() # Get flight ID

# Allocate fuel from inventory
inventory.total_quantity_liters = round(inventory.total_quantity_liters - required_f

# Record fuel transaction
tx = FuelTransaction(
    flight_id=flight.id,
    fuel_type=aircraft.fuel_type,
    quantity_liters=required_fuel,
    transaction_type='allocation',
    price_per_liter=current_price,
    total_cost=round(required_fuel * current_price, 2),
    notes=f'Fuel allocated for flight {flight.flight_number}',
    created_by=int(user_id)
)
db.session.add(tx)
db.session.commit()

return jsonify(flight.to_dict()), 201

@flights_bp.route('/<int:flight_id>/complete', methods=['POST'])
@jwt_required()
def complete_flight(flight_id):

```

```

"""
Mark a flight as completed with actual fuel usage.
Calculates efficiency and trip cost.
Records refund if less fuel was used than allocated.
"""

user_id = get_jwt_identity()
data = request.get_json()

if not data.get('actual_fuel_used_liters'):
    return jsonify({'error': 'actual_fuel_used_liters is required'}), 400

flight = Flight.query.get_or_404(flight_id)
if flight.status == 'completed':
    return jsonify({'error': 'Flight already completed'}), 400

actual_fuel = float(data['actual_fuel_used_liters'])
required_fuel = flight.required_fuel_liters
current_price = flight.fuel_price_at_time or _get_current_price(flight.aircraft.fuel

efficiency = round(actual_fuel / required_fuel, 4) if required_fuel > 0 else 1.0
trip_cost = round(actual_fuel * current_price, 2)

flight.actual_fuel_used_liters = actual_fuel
flight.fuel_efficiency = efficiency
flight.trip_fuel_cost = trip_cost
flight.status = 'completed'

if data.get('arrival_time'):
    flight.arrival_time = datetime.fromisoformat(data['arrival_time'])
else:
    flight.arrival_time = datetime.now(timezone.utc)

# If actual < allocated, refund the difference to inventory
difference = round(required_fuel - actual_fuel, 2)
if difference > 0:
    aircraft = flight.aircraft
    inventory = FuelInventory.query.filter_by(fuel_type=aircraft.fuel_type).first()
    if inventory:
        inventory.total_quantity_liters = round(inventory.total_quantity_liters + di

    refund_tx = FuelTransaction(
        flight_id=flight.id,
        fuel_type=aircraft.fuel_type,
        quantity_liters=difference,
        transaction_type='refund',
        price_per_liter=current_price,
        total_cost=round(difference * current_price, 2),

```



```

        notes=f'Fuel refund for flight {flight.flight_number} (unused)',
        created_by=int(user_id)
    )
    db.session.add(refund_tx)

# Record consumption transaction
consumption_tx = FuelTransaction(
    flight_id=flight.id,
    fuel_type=flight.aircraft.fuel_type,
    quantity_liters=actual_fuel,
    transaction_type='consumption',
    price_per_liter=current_price,
    total_cost=trip_cost,
    notes=f'Actual fuel consumed for flight {flight.flight_number}',
    created_by=int(user_id)
)
db.session.add(consumption_tx)

# Update daily usage
_update_daily_usage(flight.flight_date, actual_fuel, flight.aircraft.fuel_type, trip

db.session.commit()

return jsonify({
    'flight': flight.to_dict(),
    'efficiency': efficiency,
    'efficiency_label': 'excellent' if efficiency < 0.95 else ('good' if efficiency
    'fuel_saved_liters': max(0, difference),
    'trip_fuel_cost': trip_cost
}), 200

@flights_bp.route('/<int:flight_id>/cancel', methods=['POST'])
@jwt_required()
def cancel_flight(flight_id):
    """Cancel a flight and return allocated fuel to inventory."""
    user_id = get_jwt_identity()
    flight = Flight.query.get_or_404(flight_id)

    if flight.status in ['completed', 'cancelled']:
        return jsonify({'error': f'Cannot cancel a {flight.status} flight'}), 400

    aircraft = flight.aircraft
    inventory = FuelInventory.query.filter_by(fuel_type=aircraft.fuel_type).first()
    if inventory:
        inventory.total_quantity_liters = round(
            inventory.total_quantity_liters + flight.required_fuel_liters, 2

```

```

    )

    # Refund allocation
    refund_tx = FuelTransaction(
        flight_id=flight.id,
        fuel_type=aircraft.fuel_type,
        quantity_liters=flight.required_fuel_liters,
        transaction_type='refund',
        price_per_liter=flight.fuel_price_at_time or 0,
        total_cost=round(flight.required_fuel_liters * (flight.fuel_price_at_time or 0),
        notes=f'Flight {flight.flight_number} cancelled - fuel returned to inventory',
        created_by=int(user_id)
    )
    db.session.add(refund_tx)

    flight.status = 'cancelled'
    db.session.commit()
    return jsonify({'message': f'Flight {flight.flight_number} cancelled', 'fuel_returne

def _update_daily_usage(flight_date, fuel_liters, fuel_type, fuel_cost):
    """Update or create DailyFuelUsage record for a given date."""
    daily = DailyFuelUsage.query.filter_by(usage_date=flight_date).first()
    if not daily:
        daily = DailyFuelUsage(usage_date=flight_date, total_fuel_used_liters=0.0,
                               total_flights=0, total_fuel_cost=0.0, jet_a1_used=0.0, av
        db.session.add(daily)

    daily.total_fuel_used_liters = round(daily.total_fuel_used_liters + fuel_liters, 2)
    daily.total_flights += 1
    daily.total_fuel_cost = round(daily.total_fuel_cost + fuel_cost, 2)

    if fuel_type == 'Jet A1':
        daily.jet_a1_used = round(daily.jet_a1_used + fuel_liters, 2)
    else:
        daily.avgas_used = round(daily.avgas_used + fuel_liters, 2)

```

File: backend/blueprints/fuel_purchases.py

```

"""
Fuel Purchases Blueprint
Handles recording, querying fuel purchases and price tracking.
Business Logic:

```

```

- Current fuel price = most recent purchase price per fuel type
- Inventory is incremented on each purchase
"""

from datetime import datetime, date
from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required, get_jwt, get_jwt_identity
from sqlalchemy import desc
from extensions import db
from models import FuelPurchase, FuelInventory, FuelSupplier

fuel_purchases_bp = Blueprint('fuel_purchases', __name__)

@fuel_purchases_bp.route('/', methods=['GET'])
@jwt_required()
def list_purchases():
    """List fuel purchases with optional filters: fuel_type, start_date, end_date."""
    fuel_type = request.args.get('fuel_type')
    start_date = request.args.get('start_date')
    end_date = request.args.get('end_date')
    per_page = int(request.args.get('per_page', 50))
    page = int(request.args.get('page', 1))

    query = FuelPurchase.query

    if fuel_type:
        query = query.filter_by(fuel_type=fuel_type)
    if start_date:
        query = query.filter(FuelPurchase.purchase_date >= start_date)
    if end_date:
        query = query.filter(FuelPurchase.purchase_date <= end_date)

    pagination = query.order_by(desc(FuelPurchase.purchase_date)).paginate(
        page=page, per_page=per_page, error_out=False
    )

    return jsonify({
        'purchases': [p.to_dict() for p in pagination.items],
        'total': pagination.total,
        'pages': pagination.pages,
        'current_page': page
    }), 200

@fuel_purchases_bp.route('/<int:purchase_id>', methods=['GET'])
@jwt_required()
def get_purchase(purchase_id):

```

```
"""Get a single fuel purchase by ID."""
purchase = FuelPurchase.query.get_or_404(purchase_id)
return jsonify(purchase.to_dict()), 200


@fuel_purchases_bp.route('/', methods=['POST'])
@jwt_required()
def create_purchase():
    """
    Record a new fuel purchase.
    Automatically updates the FuelInventory for the relevant fuel type.
    """
    user_id = get_jwt_identity()
    data = request.get_json()

    required = ['supplier_name', 'location', 'fuel_type', 'quantity_liters', 'price_per_
for field in required:
    if not data.get(field) and data.get(field) != 0:
        return jsonify({'error': f'{field} is required'}), 400

    if data['fuel_type'] not in ['Jet A1', 'Avgas']:
        return jsonify({'error': 'fuel_type must be Jet A1 or Avgas'}), 400

    qty = float(data['quantity_liters'])
    price = float(data['price_per_liter'])

    purchase = FuelPurchase(
        supplier_name=data['supplier_name'],
        location=data['location'],
        fuel_type=data['fuel_type'],
        quantity_liters=qty,
        price_per_liter=price,
        total_cost=round(qty * price, 2),
        purchase_date=datetime.strptime(data['purchase_date'], '%Y-%m-%d').date(),
        invoice_number=data.get('invoice_number'),
        notes=data.get('notes'),
        created_by=int(user_id)
    )

    # Link supplier if ID provided
    if data.get('supplier_id'):
        supplier = FuelSupplier.query.get(data['supplier_id'])
        if supplier:
            purchase.supplier_id = supplier.id

    db.session.add(purchase)
```

```

# Update inventory
inventory = FuelInventory.query.filter_by(fuel_type=data['fuel_type']).first()
if not inventory:
    inventory = FuelInventory(fuel_type=data['fuel_type'], total_quantity_liters=0.0)
    db.session.add(inventory)
inventory.total_quantity_liters = round(inventory.total_quantity_liters + qty, 2)

db.session.commit()
return jsonify(purchase.to_dict()), 201

@fuel_purchases_bp.route('/current-price', methods=['GET'])
@jwt_required()
def current_price():
    """
    Returns the current (latest) fuel price for each fuel type.
    Current price = most recent purchase price.
    """
    jet = FuelPurchase.query.filter_by(fuel_type='Jet A1').order_by(
        desc(FuelPurchase.purchase_date), desc(FuelPurchase.id)
    ).first()

    avgas = FuelPurchase.query.filter_by(fuel_type='Avgas').order_by(
        desc(FuelPurchase.purchase_date), desc(FuelPurchase.id)
    ).first()

    return jsonify({
        'Jet A1': {
            'price_per_liter': jet.price_per_liter if jet else None,
            'last_purchase_date': jet.purchase_date.isoformat() if jet else None,
            'supplier': jet.supplier_name if jet else None
        },
        'Avgas': {
            'price_per_liter': avgas.price_per_liter if avgas else None,
            'last_purchase_date': avgas.purchase_date.isoformat() if avgas else None,
            'supplier': avgas.supplier_name if avgas else None
        }
    }), 200

@fuel_purchases_bp.route('/price-trend', methods=['GET'])
@jwt_required()
def price_trend():
    """
    Returns price trend data grouped by date for charting.
    Query param: fuel_type (default: Jet A1), days (default: 30)
    """

```

```

fuel_type = request.args.get('fuel_type', 'Jet A1')
days = int(request.args.get('days', 30))

purchases = FuelPurchase.query.filter_by(fuel_type=fuel_type).order_by(
    FuelPurchase.purchase_date
).all()

# Group by date, take the last price of each day
price_map = {}
for p in purchases:
    key = p.purchase_date.isoformat()
    price_map[key] = p.price_per_liter

trend = [{'date': k, 'price_per_liter': v} for k, v in sorted(price_map.items())]

# Return only last N days
return jsonify(trend[-days:]), 200

@fuel_purchases_bp.route('/average-price', methods=['GET'])
@jwt_required()
def average_price():
    """Calculate weighted average fuel price per fuel type."""
    from sqlalchemy import func

    result = db.session.query(
        FuelPurchase.fuel_type,
        func.avg(FuelPurchase.price_per_liter).label('avg_price'),
        func.sum(FuelPurchase.quantity_liters).label('total_qty'),
        func.count(FuelPurchase.id).label('purchase_count')
    ).group_by(FuelPurchase.fuel_type).all()

    return jsonify([
        {
            'fuel_type': r.fuel_type,
            'avg_price_per_liter': round(r.avg_price, 4) if r.avg_price else 0,
            'total_quantity_liters': round(r.total_qty, 2) if r.total_qty else 0,
            'purchase_count': r.purchase_count
        } for r in result]), 200

@fuel_purchases_bp.route('/predict-price', methods=['GET'])
@jwt_required()
def predict_price():
    """
    Basic linear regression to predict future fuel prices.
    Query param: fuel_type, days_ahead (default: 7)
    """

```

```

import numpy as np
from sklearn.linear_model import LinearRegression

fuel_type = request.args.get('fuel_type', 'Jet A1')
days_ahead = int(request.args.get('days_ahead', 7))

purchases = FuelPurchase.query.filter_by(fuel_type=fuel_type).order_by(
    FuelPurchase.purchase_date
).all()

if len(purchases) < 5:
    return jsonify({'error': 'Insufficient data for prediction (need at least 5 reco

from datetime import datetime as dt
base_date = purchases[0].purchase_date
X = np.array([(p.purchase_date - base_date).days for p in purchases]).reshape(-1, 1)
y = np.array([p.price_per_liter for p in purchases])

model = LinearRegression()
model.fit(X, y)

last_day = (purchases[-1].purchase_date - base_date).days
predictions = []
for i in range(1, days_ahead + 1):
    future_day = last_day + i
    pred_price = round(float(model.predict([[future_day]])[0]), 4)
    from datetime import timedelta
    future_date = purchases[-1].purchase_date + timedelta(days=i)
    predictions.append({
        'date': future_date.isoformat(),
        'predicted_price': pred_price
    })

return jsonify({
    'fuel_type': fuel_type,
    'current_price': purchases[-1].price_per_liter,
    'predictions': predictions,
    'trend': 'increasing' if model.coef_[0] > 0 else 'decreasing',
    'daily_change_rate': round(float(model.coef_[0]), 4)
}), 200

@fuel_purchases_bp.route('/suppliers', methods=['GET'])
@jwt_required()
def list_suppliers():
    """List all fuel suppliers."""
    suppliers = FuelSupplier.query.filter_by(is_active=True).all()

```

```

        return jsonify([s.to_dict() for s in suppliers]), 200

@fuel_purchases_bp.route('/suppliers', methods=['POST'])
@jwt_required()
def create_supplier():
    """Create a new fuel supplier."""
    data = request.get_json()
    if not data.get('name'):
        return jsonify({'error': 'Supplier name is required'}), 400

    supplier = FuelSupplier(
        name=data['name'],
        contact_email=data.get('contact_email'),
        contact_phone=data.get('contact_phone'),
        address=data.get('address')
    )
    db.session.add(supplier)
    db.session.commit()
    return jsonify(supplier.to_dict()), 201

```

File: backend/blueprints/fuel_transactions.py

```

"""
Fuel Transactions Blueprint
Full transaction history with filtering and audit trails.
"""

from flask import Blueprint, request, jsonify
from flask_jwt_extended import jwt_required
from sqlalchemy import desc
from models import FuelTransaction, FuelInventory

fuel_transactions_bp = Blueprint('fuel_transactions', __name__)

@fuel_transactions_bp.route('', methods=['GET'])
@jwt_required()
def list_transactions():
    """List all fuel transactions with filters."""
    tx_type = request.args.get('transaction_type')
    fuel_type = request.args.get('fuel_type')
    flight_id = request.args.get('flight_id')
    start_date = request.args.get('start_date')

```



```

end_date = request.args.get('end_date')
per_page = int(request.args.get('per_page', 50))
page = int(request.args.get('page', 1))

query = FuelTransaction.query
if tx_type:
    query = query.filter_by(transaction_type=tx_type)
if fuel_type:
    query = query.filter_by(fuel_type=fuel_type)
if flight_id:
    query = query.filter_by(flight_id=int(flight_id))
if start_date:
    query = query.filter(FuelTransaction.transaction_date >= start_date)
if end_date:
    query = query.filter(FuelTransaction.transaction_date <= end_date)

pagination = query.order_by(desc(FuelTransaction.transaction_date)).paginate(
    page=page, per_page=per_page, error_out=False
)

return jsonify({
    'transactions': [t.to_dict() for t in pagination.items],
    'total': pagination.total,
    'pages': pagination.pages,
    'current_page': page
}), 200

@fuel_transactions_bp.route('/inventory', methods=['GET'])
@jwt_required()
def get_inventory():
    """Get current fuel inventory levels."""
    inventories = FuelInventory.query.all()
    return jsonify([i.to_dict() for i in inventories]), 200

@fuel_transactions_bp.route('/summary', methods=['GET'])
@jwt_required()
def transaction_summary():
    """Summary of transactions grouped by type."""
    from sqlalchemy import func
    from extensions import db

    result = db.session.query(
        FuelTransaction.transaction_type,
        FuelTransaction.fuel_type,
        func.sum(FuelTransaction.quantity_liters).label('total_liters'),

```

```

        func.sum(FuelTransaction.total_cost).label('total_cost'),
        func.count(FuelTransaction.id).label('count')
    ).group_by(FuelTransaction.transaction_type, FuelTransaction.fuel_type).all()

    return jsonify([
        {
            'transaction_type': r.transaction_type,
            'fuel_type': r.fuel_type,
            'total_liters': round(r.total_liters or 0, 2),
            'total_cost': round(r.total_cost or 0, 2),
            'count': r.count
        } for r in result]), 200

```

File: backend/blueprints/reports.py

```

"""
Reports Blueprint
PDF generation for fuel cost reports, monthly summaries, and aircraft consumption.
"""

from io import BytesIO
from datetime import date, datetime
from flask import Blueprint, request, jsonify, send_file
from flask_jwt_extended import jwt_required
from sqlalchemy import func, desc
from extensions import db
from models import Flight, FuelPurchase, Aircraft, DailyFuelUsage, FuelTransaction

reports_bp = Blueprint('reports', __name__)

def _get_date_filters(request):
    start = request.args.get('start_date')
    end = request.args.get('end_date')
    if not start:
        start = date.today().replace(day=1).isoformat()
    if not end:
        end = date.today().isoformat()
    return start, end

@reports_bp.route('/trip-costs', methods=['GET'])
@jwt_required()
def trip_costs_report():
    """Per-trip fuel cost report with filters."""

```

```

start, end = _get_date_filters(request)
aircraft_id = request.args.get('aircraft_id')

query = Flight.query.filter(
    Flight.status == 'completed',
    Flight.flight_date >= start,
    Flight.flight_date <= end
)
if aircraft_id:
    query = query.filter_by(aircraft_id_fk=int(aircraft_id))

flights = query.order_by(Flight.flight_date).all()

data = []
for f in flights:
    data.append({
        'flight_number': f.flight_number,
        'flight_date': f.flight_date.isoformat(),
        'route': f'{f.source} → {f.destination}',
        'aircraft': f.aircraft.aircraft_id if f.aircraft else 'N/A',
        'distance_km': f.distance_km,
        'required_fuel': f.required_fuel_liters,
        'actual_fuel': f.actual_fuel_used_liters,
        'efficiency': f.fuel_efficiency,
        'price_per_liter': f.fuel_price_at_time,
        'trip_fuel_cost': f.trip_fuel_cost,
    })

total_cost = sum(d['trip_fuel_cost'] or 0 for d in data)
total_fuel = sum(d['actual_fuel'] or 0 for d in data)

return jsonify({
    'report_type': 'trip_costs',
    'period': {'start': start, 'end': end},
    'trips': data,
    'summary': {
        'total_trips': len(data),
        'total_fuel_used': round(total_fuel, 2),
        'total_fuel_cost': round(total_cost, 2),
        'avg_cost_per_trip': round(total_cost / len(data), 2) if data else 0
    }
}), 200

@reports_bp.route('/monthly-usage', methods=['GET'])
@jwt_required()
def monthly_usage_report():

```

```

"""Monthly fuel usage aggregated by month."""
from sqlalchemy import extract

result = db.session.query(
    extract('year', DailyFuelUsage.usage_date).label('year'),
    extract('month', DailyFuelUsage.usage_date).label('month'),
    func.sum(DailyFuelUsage.total_fuel_used_liters).label('total_fuel'),
    func.sum(DailyFuelUsage.total_flights).label('total_flights'),
    func.sum(DailyFuelUsage.total_fuel_cost).label('total_cost'),
    func.sum(DailyFuelUsage.jet_a1_used).label('jet_a1_used'),
    func.sum(DailyFuelUsage.avgas_used).label('avgas_used')
).group_by('year', 'month').order_by('year', 'month').all()

months = []
month_names = ['', 'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
                'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
for r in result:
    months.append({
        'year': int(r.year),
        'month': int(r.month),
        'month_name': f"{month_names[int(r.month)]} {int(r.year)}",
        'total_fuel_used': round(r.total_fuel or 0, 2),
        'total_flights': r.total_flights or 0,
        'total_cost': round(r.total_cost or 0, 2),
        'jet_a1_used': round(r.jet_a1_used or 0, 2),
        'avgas_used': round(r.avgas_used or 0, 2)
    })

return jsonify({
    'report_type': 'monthly_usage',
    'data': months
}), 200

```

```

@reports_bp.route('/aircraft-consumption', methods=['GET'])
@jwt_required()
def aircraft_consumption_report():
    """Aircraft-wise fuel consumption comparison report."""
    start, end = _get_date_filters(request)

    result = db.session.query(
        Aircraft.aircraft_id,
        Aircraft.model,
        func.count(Flight.id).label('total_flights'),
        func.sum(Flight.actual_fuel_used_liters).label('total_fuel_used'),
        func.sum(Flight.required_fuel_liters).label('total_required_fuel'),
        func.sum(Flight.trip_fuel_cost).label('total_cost'),

```

```

        func.sum(Flight.distance_km).label('total_distance'),
        func.avg(Flight.fuel_efficiency).label('avg_efficiency')
    ).join(Flight, Flight.aircraft_id_fk == Aircraft.id
    ).filter(
        Flight.status == 'completed',
        Flight.flight_date >= start,
        Flight.flight_date <= end
    ).group_by(Aircraft.aircraft_id, Aircraft.model).all()

data = []
for r in result:
    data.append({
        'aircraft_id': r.aircraft_id,
        'model': r.model,
        'total_flights': r.total_flights or 0,
        'total_fuel_used_liters': round(r.total_fuel_used or 0, 2),
        'total_required_fuel_liters': round(r.total_required_fuel or 0, 2),
        'total_fuel_cost': round(r.total_cost or 0, 2),
        'total_distance_km': round(r.total_distance or 0, 2),
        'avg_efficiency': round(r.avg_efficiency or 1.0, 4)
    })

return jsonify({
    'report_type': 'aircraft_consumption',
    'period': {'start': start, 'end': end},
    'data': data
}), 200

@reports_bp.route('/export-pdf', methods=['GET'])
@jwt_required()
def export_pdf():
    """
    Generate and return a PDF report.
    Query param: report_type (trip_costs | monthly_usage | aircraft_consumption)
    """
    from reportlab.lib.pagesizes import A4, landscape
    from reportlab.lib import colors
    from reportlab.lib.units import cm
    from reportlab.platypus import (SimpleDocTemplate, Table, TableStyle,
                                     Paragraph, Spacer)
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle

    report_type = request.args.get('report_type', 'trip_costs')
    start, end = _get_date_filters(request)

    buffer = BytesIO()

```

```

doc = SimpleDocTemplate(buffer, pagesize=landscape(A4),
                        rightMargin=1.5*cm, leftMargin=1.5*cm,
                        topMargin=2*cm, bottomMargin=2*cm)

styles = getSampleStyleSheet()
title_style = ParagraphStyle('Title', parent=styles['Heading1'],
                             fontSize=18, spaceAfter=6,
                             textColor=colors.HexColor('#1a1a2e'))
subtitle_style = ParagraphStyle('Subtitle', parent=styles['Normal'],
                                fontSize=10, spaceAfter=12,
                                textColor=colors.grey)

elements = []
title_map = {
    'trip_costs': 'Fuel Cost Per Trip Report',
    'monthly_usage': 'Monthly Fuel Usage Report',
    'aircraft_consumption': 'Aircraft-Wise Fuel Consumption Report',
    'fuel_purchases': 'Fuel Procurement & Purchase Report'
}

elements.append(Paragraph('✈ Aircraft Fuel Management System', title_style))
elements.append(Paragraph(f'{title_map.get(report_type, "Report")} | Period: {start}'
                           f'Generated: {datetime.now().strftime("%Y-%m-%d %H:%M")}',
                           elements.append(Spacer(1, 0.5*cm))

header_style = TableStyle([
    ('BACKGROUND', (0, 0), (-1, 0), colors.HexColor('#1a1a2e')),
    ('TEXTCOLOR', (0, 0), (-1, 0), colors.white),
    ('FONTNAME', (0, 0), (-1, 0), 'Helvetica-Bold'),
    ('FONTSIZE', (0, 0), (-1, 0), 10),
    ('ROWBACKGROUNDS', (0, 1), (-1, -1), [colors.white, colors.HexColor('#f0f4ff')]),
    ('GRID', (0, 0), (-1, -1), 0.5, colors.HexColor('#cccccc')),
    ('FONTNAME', (0, 1), (-1, -1), 'Helvetica'),
    ('FONTSIZE', (0, 1), (-1, -1), 8),
    ('TOPPADDING', (0, 0), (-1, -1), 6),
    ('BOTTOMPADDING', (0, 0), (-1, -1), 6),
    ('ALIGN', (0, 0), (-1, -1), 'CENTER'),
])

if report_type == 'trip_costs':
    flights = Flight.query.filter(
        Flight.status == 'completed',
        Flight.flight_date >= start,
        Flight.flight_date <= end
    ).order_by(Flight.flight_date).all()

    table_data = [['Flight #', 'Date', 'Route', 'Aircraft', 'Distance (km)',

```

```

        'Fuel Used (L)', 'Price/L (₹)', 'Trip Cost (₹)']]
    for f in flights:
        table_data.append([
            f.flight_number,
            f.flight_date.isoformat(),
            f'{f.source} → {f.destination}',
            f.aircraft.aircraft_id if f.aircraft else 'N/A',
            f'{f.distance_km:,.0f}',
            f'{f.actual_fuel_used_liters:,.1f}' if f.actual_fuel_used_liters else '-',
            f'₹{f.fuel_price_at_time:.2f}' if f.fuel_price_at_time else '-',
            f'₹{f.trip_fuel_cost:,.2f}' if f.trip_fuel_cost else '-'
        ])

    if len(table_data) > 1:
        tbl = Table(table_data, colWidths=[2.5*cm, 2.5*cm, 6*cm, 2.5*cm, 3*cm, 3*cm,
        tbl.setStyle(header_style)
        elements.append(tbl)
    else:
        elements.append(Paragraph('No completed flights in selected period.', styles

elif report_type == 'fuel_purchases':
    purchases = FuelPurchase.query.filter(
        FuelPurchase.purchase_date >= start,
        FuelPurchase.purchase_date <= end
    ).order_by(FuelPurchase.purchase_date).all()

    table_data = [['Date', 'Supplier', 'Location', 'Fuel Type', 'Invoice #', 'Quanti
    total_qty = 0
    total_cost = 0

    for p in purchases:
        row_total = p.quantity_liters * p.price_per_liter
        total_qty += p.quantity_liters
        total_cost += row_total
        table_data.append([
            p.purchase_date.strftime('%Y-%m-%d'),
            p.supplier_name,
            p.location,
            p.fuel_type,
            p.invoice_number or '-',
            f'{p.quantity_liters:,.1f}',
            f'₹{p.price_per_liter:,.2f}',
            f'₹{row_total:,.2f}'
        ])

    if len(table_data) > 1:
        # Summary Row

```

```

        table_data.append(['TOTAL', '', '', '', '', f'{total_qty:,.1f}', '', f'₹{tot

tbl = Table(table_data, colWidths=[2.5*cm, 4.5*cm, 4*cm, 2.5*cm, 3*cm, 3*cm,
tbl.setStyle(header_style)
# Add some extra styling for the summary row
tbl.setStyle(TableStyle([
    ('BACKGROUND', (0, -1), (-1, -1), colors.HexColor('#e2e8f0')),
    ('FONTNAME', (0, -1), (-1, -1), 'Helvetica-Bold'),
]))
elements.append(tbl)
else:
    elements.append(Paragraph('No purchase records found in selected period.', s

elif report_type == 'monthly_usage':
    records = DailyFuelUsage.query.order_by(DailyFuelUsage.usage_date).all()
    month_map = {}
    for r in records:
        key = r.usage_date.strftime('%b %Y')
        if key not in month_map:
            month_map[key] = {'fuel': 0, 'flights': 0, 'cost': 0}
        month_map[key]['fuel'] += r.total_fuel_used_liters
        month_map[key]['flights'] += r.total_flights
        month_map[key]['cost'] += r.total_fuel_cost

    table_data = [['Month', 'Total Fuel Used (L)', 'Total Flights', 'Total Cost (₹)']
    for month, vals in month_map.items():
        table_data.append([
            month,
            f"{vals['fuel']:,.1f}",
            str(vals['flights']),
            f"₹{vals['cost']:,.2f}"
        ])

    tbl = Table(table_data, colWidths=[4*cm, 6*cm, 5*cm, 6*cm])
    tbl.setStyle(header_style)
    elements.append(tbl)

elif report_type == 'aircraft_consumption':
    aircraft_list = Aircraft.query.all()
    table_data = [['Aircraft ID', 'Model', 'Total Flights', 'Fuel Used (L)', 'Distan

    for aircraft in aircraft_list:
        flights = Flight.query.filter_by(aircraft_id_fk=aircraft.id, status='complet
        total_fuel = sum(f.actual_fuel_used_liters or 0 for f in flights)
        total_distance = sum(f.distance_km for f in flights)
        total_cost = sum(f.trip_fuel_cost or 0 for f in flights)
        avg_eff = sum(f.fuel_efficiency or 1.0 for f in flights) / len(flights) if f

```



```

        table_data.append([
            aircraft.aircraft_id,
            aircraft.model,
            str(len(flights)),
            f'{total_fuel:,.1f}',
            f'{total_distance:,.0f}',
            f'{total_cost:,.2f}',
            f'{avg_eff:.4f}'
        ])

tbl = Table(table_data, colWidths=[2.5*cm, 5*cm, 3*cm, 3.5*cm, 3.5*cm, 4*cm, 3.5*cm])
tbl.setStyle(header_style)
elements.append(tbl)

doc.build(elements)
buffer.seek(0)

return send_file(
    buffer,
    as_attachment=True,
    download_name=f'{report_type}_{start}_{end}.pdf',
    mimetype='application/pdf'
)

```

File: backend/requirements.txt

```

Flask==3.0.3
Flask-Cors==4.0.1
Flask-JWT-Extended==4.6.0
Flask-SQLAlchemy==3.1.1
SQLAlchemy==2.0.36
psycopg2-binary==2.9.10
python-dotenv==1.0.1
Werkzeug==3.0.6
marshmallow==3.23.1
Flask-Marshmallow==1.2.1
marshmallow-sqlalchemy==1.1.0
reportlab==4.2.5
numpy==1.26.4
scikit-learn==1.5.2
pandas==2.2.3

```

```
unicorn==23.0.0  
python-dateutil==2.9.0
```

File: frontend/package.json

```
{  
  "name": "frontend",  
  "version": "0.1.0",  
  "private": true,  
  "dependencies": {  
    "@splinetool/react-spline": "^4.1.0",  
    "@splinetool/runtime": "^1.12.81",  
    "@testing-library/dom": "^10.4.1",  
    "@testing-library/jest-dom": "^6.9.1",  
    "@testing-library/react": "^16.3.2",  
    "@testing-library/user-event": "^13.5.0",  
    "axios": "^1.15.0",  
    "framer-motion": "^12.38.0",  
    "gsap": "^3.14.2",  
    "lucide-react": "^1.8.0",  
    "react": "^19.2.4",  
    "react-dom": "^19.2.4",  
    "react-router-dom": "^7.14.0",  
    "react-scripts": "5.0.1",  
    "recharts": "^3.8.1",  
    "web-vitals": "^2.1.4"  
  },  
  "scripts": {  
    "start": "set NODE_OPTIONS=--max-old-space-size=4096 && react-scripts start",  
    "build": "react-scripts build",  
    "test": "react-scripts test",  
    "eject": "react-scripts eject"  
  },  
  "eslintConfig": {  
    "extends": [  
      "react-app",  
      "react-app/jest"  
    ]  
  },  
  "browserslist": {  
    "production": [  
      ">0.2%",  
      "not dead",  
    ]  
  }  
}
```

```
    "not op_mini all"
  ],
  "development": [
    "last 1 chrome version",
    "last 1 firefox version",
    "last 1 safari version"
  ]
}
```

File: frontend/src/App.js

```
import React from 'react';
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './contexts/AuthContext';
import Sidebar from './components/Sidebar';
import LoginPage from './pages/LoginPage';
import DashboardPage from './pages/DashboardPage';
import FuelPurchasesPage from './pages/FuelPurchasesPage';
import FuelInventoryPage from './pages/FuelInventoryPage';
import AircraftPage from './pages/AircraftPage';
import FlightsPage from './pages/FlightsPage';
import PriceTrendsPage from './pages/PriceTrendsPage';
import TransactionsPage from './pages/TransactionsPage';
import ReportsPage from './pages/ReportsPage';
import DailyUsagePage from './pages/DailyUsagePage';
import UsersPage from './pages/UsersPage';

const ProtectedRoute = ({ children, adminOnly = false }) => {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  if (adminOnly && user.role !== 'admin') return <Navigate to="/" replace />;
  return (
    <div className="main-layout">
      <Sidebar />
      <main className="content-area animate-fade-in">
        {children}
      </main>
    </div>
  );
};

function AppRoutes() {
```

```

const { user, loading } = useAuth();

if (loading) return (
  <div style={{ height: '100vh', display: 'flex', alignItems: 'center', justifyContent: 'center' }}>
    <div className="spinner" />
    <span style={{ color: 'var(--text-muted)', fontSize: '0.9rem' }}>Loading AeroFuel...</span>
  </div>
);

return (
  <Routes>
    <Route path="/login" element={user ? <Navigate to="/" replace /> : <LoginPage />} />
    <Route path="/" element={<ProtectedRoute><DashboardPage /></ProtectedRoute>} />
    <Route path="/flights" element={<ProtectedRoute><FlightsPage /></ProtectedRoute>} />
    <Route path="/aircraft" element={<ProtectedRoute><AircraftPage /></ProtectedRoute>} />
    <Route path="/inventory" element={<ProtectedRoute><FuelInventoryPage /></ProtectedRoute>} />
    <Route path="/purchases" element={<ProtectedRoute><FuelPurchasesPage /></ProtectedRoute>} />
    <Route path="/transactions" element={<ProtectedRoute><TransactionsPage /></ProtectedRoute>} />
    <Route path="/trends" element={<ProtectedRoute><PriceTrendsPage /></ProtectedRoute>} />
    <Route path="/reports" element={<ProtectedRoute><ReportsPage /></ProtectedRoute>} />
    <Route path="/daily-usage" element={<ProtectedRoute><DailyUsagePage /></ProtectedRoute>} />
    <Route path="/users" element={<ProtectedRoute adminOnly><UsersPage /></ProtectedRoute>} />
    <Route path="*" element={<Navigate to="/" replace />} />
  </Routes>
);
}

function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <AppRoutes />
      </BrowserRouter>
    </AuthProvider>
  );
}

export default App;

```

File: frontend/src/index.js

```

import React from 'react';
import ReactDOM from 'react-dom/client';

```

```
import './index.css';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

File: frontend/src/index.css

```
@import url('https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700;800;900&display=block');

:root {
  /* Core colors - Clean Light Mode */
  --bg-color: #f0f4f8;
  --card-bg: #ffffff;
  --card-bg-subtle: #f8fafc;
  --border-color: #e2e8f0;
  --border-strong: #cbd5e1;

  /* Brand */
  --primary-accent: #1d4ed8;
  --primary-dark: #1e3a8a;
  --primary-light: #eff6ff;
  --primary-mid: #dbeafe;
  --secondary-accent: #059669;
  --secondary-light: #d1fae5;
  --tertiary-accent: #7c3aed;
  --tertiary-light: #ede9fe;

  /* Text */
  --text-main: #0f172a;
  --text-sub: #1e293b;
  --text-muted: #64748b;
  --text-faint: #94a3b8;

  /* Status */
  --danger: #dc2626;
  --danger-light: #fee2e2;
  --warning: #d97706;
  --warning-light: #fef3c7;
```

```

--success: #059669;
--success-light: #d1fae5;
--info: #0284c7;
--info-light: #e0f2fe;

/* Layout */
--sidebar-width: 268px;
--radius-sm: 6px;
--radius-md: 10px;
--radius-lg: 16px;
--radius-xl: 20px;

/* Shadows */
--shadow-xs: 0 1px 2px 0 rgb(0 0 0 / 0.04);
--shadow-sm: 0 1px 3px 0 rgb(0 0 0 / 0.06), 0 1px 2px -1px rgb(0 0 0 / 0.06);
--shadow-md: 0 4px 6px -1px rgb(0 0 0 / 0.05), 0 2px 4px -2px rgb(0 0 0 / 0.05);
--shadow-lg: 0 10px 15px -3px rgb(0 0 0 / 0.06), 0 4px 6px -4px rgb(0 0 0 / 0.05);
--shadow-xl: 0 20px 25px -5px rgb(0 0 0 / 0.07), 0 8px 10px -6px rgb(0 0 0 / 0.05);
--shadow-blue: 0 4px 14px rgba(29, 78, 216, 0.18);
}

*,
*::before,
*::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Plus Jakarta Sans', -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  background-color: var(--bg-color);
  color: var(--text-main);
  overflow-x: hidden;
  background-image:
    radial-gradient(ellipse at 0% 0%, rgba(29, 78, 216, 0.05) 0px, transparent 60%),
    radial-gradient(ellipse at 100% 100%, rgba(5, 150, 105, 0.04) 0px, transparent 60%);
  min-height: 100vh;
}

h1,
h2,
h3,
h4,
h5,

```

```
h6 {
  font-family: 'Outfit', sans-serif;
  color: var(--text-main);
  letter-spacing: -0.01em;
}

a {
  text-decoration: none;
  color: inherit;
}

/* — CARDS — */
.glass-card {
  background: var(--card-bg);
  border: 1px solid var(--border-color);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-md);
  transition: box-shadow 0.25s ease, transform 0.25s ease;
}

.glass-card:hover {
  box-shadow: var(--shadow-lg);
}

.card {
  transition: all 0.4s cubic-bezier(0.165, 0.84, 0.44, 1);
  border-radius: 12px !important;
}

/* premium hover effect */
.card:hover {
  transform: translateY(-10px) scale(1.03) !important;
  box-shadow: 0 20px 50px rgba(0,0,0,0.2) !important;
  z-index: 10;
}

.card-elevated {
  background: var(--card-bg);
  border: 1px solid var(--border-color);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-lg);
}

/* — GRADIENT TEXT — */
.gradient-text {
  background: linear-gradient(135deg, #1d4ed8 0%, #059669 100%);
  -webkit-background-clip: text;
```

```
-webkit-text-fill-color: transparent;
background-clip: text;
}

/* ——— BUTTONS ——— */
.btn-primary {
  background: linear-gradient(135deg, #1d4ed8 0%, #1e40af 100%);
  color: white;
  border: none;
  padding: 10px 20px;
  border-radius: var(--radius-md);
  font-family: 'Plus Jakarta Sans', sans-serif;
  font-weight: 600;
  font-size: 0.9rem;
  cursor: pointer;
  transition: all 0.25s cubic-bezier(0.4, 0, 0.2, 1);
  box-shadow: var(--shadow-blue);
  display: inline-flex;
  align-items: center;
  gap: 8px;
}

.btn-primary:hover {
  transform: translateY(-2px);
  box-shadow: 0 6px 20px rgba(29, 78, 216, 0.3);
}

.btn-primary:active {
  transform: translateY(0);
}

.btn-secondary {
  background: white;
  color: var(--text-main);
  border: 1px solid var(--border-color);
  padding: 10px 20px;
  border-radius: var(--radius-md);
  font-family: 'Plus Jakarta Sans', sans-serif;
  font-weight: 600;
  font-size: 0.9rem;
  cursor: pointer;
  transition: all 0.2s ease;
  display: inline-flex;
  align-items: center;
  gap: 8px;
}
```



```
.btn-secondary:hover {
  background: var(--card-bg-subtle);
  border-color: var(--border-strong);
  box-shadow: var(--shadow-sm);
}

.btn-danger {
  background: linear-gradient(135deg, #dc2626 0%, #b91c1c 100%);
  color: white;
  border: none;
  padding: 8px 16px;
  border-radius: var(--radius-md);
  font-family: 'Plus Jakarta Sans', sans-serif;
  font-weight: 600;
  font-size: 0.875rem;
  cursor: pointer;
  transition: all 0.2s ease;
  display: inline-flex;
  align-items: center;
  gap: 6px;
}

.btn-danger:hover {
  transform: translateY(-1px);
  box-shadow: 0 4px 12px rgba(220, 38, 38, 0.25);
}

.btn-success {
  background: linear-gradient(135deg, #059669 0%, #047857 100%);
  color: white;
  border: none;
  padding: 8px 16px;
  border-radius: var(--radius-md);
  font-family: 'Plus Jakarta Sans', sans-serif;
  font-weight: 600;
  font-size: 0.875rem;
  cursor: pointer;
  transition: all 0.2s ease;
  display: inline-flex;
  align-items: center;
  gap: 6px;
}

.btn-success:hover {
  transform: translateY(-1px);
  box-shadow: 0 4px 12px rgba(5, 150, 105, 0.25);
}
```

```
.btn-icon {
  background: var(--card-bg-subtle);
  color: var(--text-muted);
  border: 1px solid var(--border-color);
  width: 36px;
  height: 36px;
  border-radius: var(--radius-md);
  cursor: pointer;
  transition: all 0.2s ease;
  display: inline-flex;
  align-items: center;
  justify-content: center;
}

.btn-icon:hover {
  background: var(--primary-light);
  color: var(--primary-accent);
  border-color: var(--primary-mid);
}

/* ——— TYPOGRAPHY ——— */

.text-h1 {
  font-size: 2rem;
  font-weight: 700;
  letter-spacing: -0.03em;
  line-height: 1.2;
}

.text-h2 {
  font-size: 1.5rem;
  font-weight: 600;
  line-height: 1.3;
}

.text-h3 {
  font-size: 1.125rem;
  font-weight: 600;
  line-height: 1.4;
}

.text-body {
  font-size: 0.9375rem;
  color: var(--text-main);
  line-height: 1.6;
}
```

```
.text-muted {
  font-size: 0.875rem;
  color: var(--text-muted);
}

.text-small {
  font-size: 0.8125rem;
  color: var(--text-muted);
}

.text-label {
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  color: var(--text-muted);
}

/* — LAYOUT — */
.main-layout {
  display: flex;
  min-height: 100vh;
}

/* — SIDEBAR — */
.sidebar {
  width: var(--sidebar-width);
  height: 100vh;
  position: fixed;
  left: 0;
  top: 0;
  background: white;
  border-right: 1px solid var(--border-color);
  padding: 0;
  display: flex;
  flex-direction: column;
  z-index: 100;
  box-shadow: var(--shadow-sm);
}

.sidebar-header {
  padding: 24px 20px 20px;
  border-bottom: 1px solid var(--border-color);
  display: flex;
  align-items: center;
  gap: 12px;
}
```

```
.sidebar-logo {
  width: 40px;
  height: 40px;
  border-radius: 10px;
  background: linear-gradient(135deg, #1d4ed8 0%, #059669 100%);
  display: flex;
  align-items: center;
  justify-content: center;
  color: white;
  font-weight: 800;
  font-size: 15px;
  font-family: 'Outfit', sans-serif;
  flex-shrink: 0;
  box-shadow: 0 4px 12px rgba(29, 78, 216, 0.3);
}

.sidebar-nav {
  flex: 1;
  overflow-y: auto;
  padding: 16px 12px;
  display: flex;
  flex-direction: column;
  gap: 2px;
}

.sidebar-section-label {
  font-size: 0.65rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.1em;
  color: var(--text-faint);
  padding: 10px 8px 4px;
  margin-top: 6px;
}

.sidebar-footer {
  padding: 16px 12px;
  border-top: 1px solid var(--border-color);
}

.content-area {
  flex: 1;
  margin-left: var(--sidebar-width);
  padding: 36px 40px;
  max-width: calc(100vw - var(--sidebar-width));
}
```

```
/* — NAV ITEMS ————— */
.nav-item {
  display: flex;
  align-items: center;
  gap: 10px;
  padding: 10px 12px;
  border-radius: var(--radius-md);
  color: var(--text-muted);
  text-decoration: none;
  font-weight: 500;
  font-size: 0.9rem;
  transition: all 0.18s ease;
  cursor: pointer;
  border: none;
  background: none;
  width: 100%;
  font-family: 'Plus Jakarta Sans', sans-serif;
}

.nav-item:hover {
  background: var(--card-bg-subtle);
  color: var(--text-main);
}

.nav-item.active {
  background: var(--primary-light);
  color: var(--primary-accent);
  font-weight: 600;
}

/* — PAGE HEADER ————— */
.page-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  margin-bottom: 28px;
  gap: 16px;
  flex-wrap: wrap;
}

.page-header-left {
  display: flex;
  flex-direction: column;
  gap: 4px;
}
```

```
.page-actions {
  display: flex;
  gap: 10px;
  align-items: center;
  flex-wrap: wrap;
}

/* — STAT CARDS — */
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
  margin-bottom: 28px;
}

.stat-card {
  padding: 24px;
  display: flex;
  flex-direction: column;
  position: relative;
  overflow: hidden;
}

.stat-card-icon {
  width: 44px;
  height: 44px;
  border-radius: var(--radius-md);
  display: flex;
  align-items: center;
  justify-content: center;
  margin-bottom: 14px;
}

.stat-label {
  color: var(--text-muted);
  font-size: 0.75rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.06em;
  margin-bottom: 6px;
}

.stat-value {
  font-family: 'Outfit', sans-serif;
  font-size: 2rem;
  font-weight: 700;
  color: var(--text-main);
}
```

```
    line-height: 1.1;
    letter-spacing: -0.02em;
}

.stat-sub {
    font-size: 0.8rem;
    color: var(--text-muted);
    margin-top: 6px;
    display: flex;
    align-items: center;
    gap: 4px;
}

/* — TABLES — */
.table-container {
    overflow-x: auto;
    border-radius: var(--radius-lg);
}

.data-table {
    width: 100%;
    border-collapse: collapse;
}

.data-table th {
    text-align: left;
    padding: 14px 16px;
    color: var(--text-muted);
    font-size: 0.72rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.06em;
    border-bottom: 1px solid var(--border-color);
    background: var(--card-bg-subtle);
    white-space: nowrap;
}

.data-table th:first-child {
    border-radius: var(--radius-sm) 0 0 0;
}

.data-table th:last-child {
    border-radius: 0 var(--radius-sm) 0 0;
}

.data-table td {
    padding: 14px 16px;
```

```
border-bottom: 1px solid var(--border-color);
font-size: 0.9rem;
color: var(--text-sub);
white-space: nowrap;
}

.data-table tbody tr:last-child td {
border-bottom: none;
}

.data-table tbody tr:hover {
background-color: var(--card-bg-subtle);
}

.data-table tbody tr {
transition: background 0.15s ease;
}

/* — BADGES — */
.badge {
padding: 3px 10px;
border-radius: 9999px;
font-size: 0.72rem;
font-weight: 700;
display: inline-flex;
align-items: center;
gap: 4px;
white-space: nowrap;
letter-spacing: 0.02em;
}

.badge-blue {
background: #dbeafe;
color: #1e40af;
}

.badge-green {
background: #d1fae5;
color: #065f46;
}

.badge-amber {
background: #fef3c7;
color: #92400e;
}

.badge-red {
```



```
background: #fee2e2;
color: #991b1b;
}

.badge-purple {
background: #ede9fe;
color: #5b21b6;
}

.badge-gray {
background: #f1f5f9;
color: #475569;
}

.badge-sky {
background: #e0f2fe;
color: #0369a1;
}

/* — FORMS ————— */
.form-group {
margin-bottom: 18px;
}

.form-label {
display: block;
font-size: 0.85rem;
font-weight: 600;
color: var(--text-sub);
margin-bottom: 6px;
}

.form-input,
.form-select {
width: 100%;
padding: 10px 14px;
border: 1.5px solid var(--border-color);
border-radius: var(--radius-md);
font-family: 'Plus Jakarta Sans', sans-serif;
font-size: 0.9rem;
color: var(--text-main);
background: white;
transition: all 0.2s ease;
-webkit-appearance: none;
}

.form-input:focus,
```

```
.form-select:focus {
  outline: none;
  border-color: var(--primary-accent);
  box-shadow: 0 0 0 3px rgba(29, 78, 216, 0.1);
}

.form-input::placeholder {
  color: var(--text-faint);
}

.form-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 16px;
}

/* — MODAL ————— */
.modal-overlay {
  position: fixed;
  inset: 0;
  background: rgba(15, 23, 42, 0.45);
  backdrop-filter: blur(4px);
  display: flex;
  align-items: center;
  justify-content: center;
  z-index: 1000;
  padding: 20px;
  animation: overlayFadeIn 0.2s ease;
}

.modal-box {
  background: white;
  border-radius: var(--radius-xl);
  box-shadow: var(--shadow-xl);
  width: 100%;
  max-width: 520px;
  max-height: 90vh;
  overflow-y: auto;
  animation: modalSlideUp 0.25s cubic-bezier(0.16, 1, 0.3, 1);
}

.modal-header {
  padding: 24px 28px 0;
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
}
```

```
.modal-body {
  padding: 20px 28px 28px;
}

.modal-footer {
  display: flex;
  justify-content: flex-end;
  gap: 10px;
  margin-top: 24px;
  padding-top: 20px;
  border-top: 1px solid var(--border-color);
}

/* — ALERTS — */
.alert {
  padding: 14px 16px;
  border-radius: var(--radius-md);
  display: flex;
  align-items: flex-start;
  gap: 12px;
}

.alert-warning {
  background: var(--warning-light);
  border: 1px solid #fde68a;
  color: #78350f;
}

.alert-error {
  background: var(--danger-light);
  border: 1px solid #fca5a5;
  color: #7f1d1d;
}

.alert-success {
  background: var(--success-light);
  border: 1px solid #6ee7b7;
  color: #064e3b;
}

.alert-info {
  background: var(--info-light);
  border: 1px solid #7dd3fc;
  color: #0c4a6e;
}
```

```
/* — PROGRESS BAR ————— */
.progress-bar {
  width: 100%;
  height: 8px;
  background: var(--border-color);
  border-radius: 999px;
  overflow: hidden;
}

.progress-fill {
  height: 100%;
  border-radius: 999px;
  transition: width 0.8s cubic-bezier(0.4, 0, 0.2, 1);
}

/* — LOADING ————— */
.spinner {
  width: 36px;
  height: 36px;
  border: 3px solid var(--border-color);
  border-top-color: var(--primary-accent);
  border-radius: 50%;
  animation: spin 0.7s linear infinite;
}

.loading-screen {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  gap: 16px;
  padding: 60px 20px;
  color: var(--text-muted);
}

.skeleton {
  background: linear-gradient(90deg, #f0f4f8 25%, #e8eef4 50%, #f0f4f8 75%);
  background-size: 200% 100%;
  animation: skeleton-pulse 1.5s ease infinite;
  border-radius: var(--radius-sm);
}

/* — DIVIDER ————— */
.divider {
  height: 1px;
  background: var(--border-color);
  margin: 20px 0;
```

```
}

/* ——— TOOLTIP ——— */
.recharts-tooltip-wrapper .recharts-default-tooltip {
  border-radius: var(--radius-md) !important;
  border: 1px solid var(--border-color) !important;
  box-shadow: var(--shadow-lg) !important;
  font-family: 'Plus Jakarta Sans', sans-serif !important;
}

/* ——— EMPTY STATE ——— */
.empty-state {
  text-align: center;
  padding: 60px 20px;
  color: var(--text-muted);
}

.empty-state-icon {
  font-size: 3rem;
  margin-bottom: 16px;
  opacity: 0.4;
}

/* ——— FILTERS ROW ——— */
.filters-row {
  display: flex;
  gap: 12px;
  flex-wrap: wrap;
  align-items: flex-end;
  margin-bottom: 20px;
}

.filter-group {
  display: flex;
  flex-direction: column;
  gap: 6px;
  min-width: 160px;
}

/* ——— ICON ACCENT BOXES ——— */
.icon-box-blue {
  background: var(--primary-light);
  color: var(--primary-accent);
}

.icon-box-green {
  background: var(--secondary-light);
```

```
    color: var(--secondary-accent);
  }

  .icon-box-amber {
    background: var(--warning-light);
    color: var(--warning);
  }

  .icon-box-red {
    background: var(--danger-light);
    color: var(--danger);
  }

  .icon-box-purple {
    background: var(--tertiary-light);
    color: var(--tertiary-accent);
  }

  /* — ANIMATIONS — */
  .animate-fade-in {
    animation: fadeIn 0.4s cubic-bezier(0.16, 1, 0.3, 1);
  }

  @keyframes fadeIn {
    from {
      opacity: 0;
      transform: translateY(10px);
    }

    to {
      opacity: 1;
      transform: translateY(0);
    }
  }

  @keyframes overlayFadeIn {
    from {
      opacity: 0;
    }

    to {
      opacity: 1;
    }
  }

  @keyframes modalSlideUp {
    from {
```

```
    opacity: 0;
    transform: translateY(20px) scale(0.97);
  }

  to {
    opacity: 1;
    transform: translateY(0) scale(1);
  }
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

@keyframes float {
  0% {
    transform: translateY(0px) rotate(0deg);
  }

  50% {
    transform: translateY(-15px) rotate(1deg);
  }

  100% {
    transform: translateY(0px) rotate(0deg);
  }
}

.animate-float {
  animation: float 5s ease-in-out infinite;
}

/* — MARQUEE ————— */
.marquee-container {
  overflow: hidden;
  user-select: none;
  display: flex;
  gap: 20px;
  mask-image: linear-gradient(
    to right,
    rgba(0, 0, 0, 0) 0%,
    rgba(0, 0, 0, 1) 10%,
    rgba(0, 0, 0, 1) 90%,
    rgba(0, 0, 0, 0) 100%
  );
}
```

```
padding: 20px 0;
}

.marquee-content {
  display: flex;
  flex-shrink: 0;
  gap: 20px;
  animation: scroll-left-to-right 15s linear infinite;
}

@keyframes scroll-left-to-right {
  0% {
    transform: translateX(-50%);
  }
  100% {
    transform: translateX(0);
  }
}

/* — SCROLLBAR — */
::-webkit-scrollbar {
  width: 6px;
  height: 6px;
}

::-webkit-scrollbar-track {
  background: transparent;
}

::-webkit-scrollbar-thumb {
  background: #cbd5e1;
  border-radius: 99px;
}

::-webkit-scrollbar-thumb:hover {
  background: #94a3b8;
}

/* — WATERMARK — */
.watermark-plane {
  position: fixed;
  top: 50%;
  left: 60%;
  width: 600px;
  height: 600px;
  background: url('plane.png') no-repeat center;
  background-size: contain;
}
```



```
transform: translate(-50%, -50%);
opacity: 0.04; /* very subtle */
z-index: 0;
pointer-events: none;
filter: grayscale(100%);
}
```

File: frontend/src/api/client.js

```
import axios from 'axios';

const API_BASE = 'http://localhost:5000/api';

const client = axios.create({
  baseURL: API_BASE,
  headers: {
    'Content-Type': 'application/json',
  },
});

client.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem('token');
    if (token) {
      config.headers['Authorization'] = `Bearer ${token}`;
    }
    return config;
  },
  (error) => Promise.reject(error)
);

export default client;
```

File: frontend/src/api/services.js

```
import client from './client';

export const auth = {
  login: (username, password) => client.post('/auth/login', { username, password })
}
```

```
profile:    () => client.get('/auth/me'),
listUsers:  () => client.get('/auth/users'),
createUser: (data) => client.post('/auth/users', data),
};

export const dashboard = {
  getKPIs:      () => client.get('/dashboard/kpis'),
  getConsumptionChart: () => client.get('/dashboard/fuel-consumption-chart'),
  getPriceChart:  () => client.get('/dashboard/price-chart'),
  getAircraftConsumption: () => client.get('/dashboard/aircraft-consumption'),
};

export const aircraft = {
  list:    ()      => client.get('/aircraft/'),
  create:  (data)   => client.post('/aircraft/', data),
  stats:   (id)     => client.get(`/aircraft/${id}/stats`),
  update:  (id, data) => client.put(`/aircraft/${id}`, data),
};

export const purchases = {
  list:      (params) => client.get('/fuel-purchases/', { params }),
  create:    (data)   => client.post('/fuel-purchases/', data),
  currentPrice: ()      => client.get('/fuel-purchases/current-price'),
  priceTrend: (params) => client.get('/fuel-purchases/price-trend', { params }),
  predictPrice: (params) => client.get('/fuel-purchases/predict-price', { params }),
  averagePrice: ()      => client.get('/fuel-purchases/average-price'),
};

export const flights = {
  list:    (params)   => client.get('/flights/', { params }),
  create:  (data)     => client.post('/flights/', data),
  complete: (id, data) => client.post(`/flights/${id}/complete`, data),
  cancel:   (id)      => client.post(`/flights/${id}/cancel`),
};

export const inventory = {
  getLevels:    ()      => client.get('/fuel-transactions/inventory'),
  getTransactions: (params) => client.get('/fuel-transactions/', { params }),
};

export const usage = {
  list: (params) => client.get('/daily-usage/', { params }),
};

export const reports = {
  getTripCosts:      (params) => client.get('/reports/trip-costs', { params }),
  getMonthlyUsage:    ()      => client.get('/reports/monthly-usage'),
};
```

```

getAircraftConsumption: (params) => client.get('/reports/aircraft-consumption', { para
exportPdf: (params) =>
  client.get('/reports/export-pdf', {
    params,
    responseType: 'blob',
  }),
};

```

File: frontend/src/components/Sidebar.js

```

import React from 'react';
import { NavLink, useNavigate } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';
import {
  LayoutDashboard, Plane, PlaneTakeoff, Fuel, History, MapPin,
  TrendingUp, FileText, LogOut, Users, BarChart3,
  ArrowLeftRight, CalendarDays, ShieldCheck
} from 'lucide-react';

const Sidebar = () => {
  const { user, logout } = useAuth();
  const navigate = useNavigate();

  const navSections = [
    {
      label: 'OVERVIEW',
      items: [
        {
          to: '/',
          icon: (
            <svg width="18" height="18" viewBox="0 0 24 24" fill="none" xmlns="http://ww
              <path d="M3 3v18h18" stroke="#cbd5e1" strokeWidth="2" strokeLinecap="round
              <rect x="6" y="14" width="4" height="7" fill="#ef4444" rx="1"/>
              <rect x="11" y="9" width="4" height="12" fill="#3b82f6" rx="1"/>
              <rect x="16" y="4" width="4" height="17" fill="#10b981" rx="1"/>
            </svg>
          ),
          label: 'Dashboard',
          end: true
        },
      ],
    },
  ],
};

```

```

    label: 'FUEL OPERATIONS',
    items: [
      { to: '/purchases', icon: <Fuel size={18} fill="#ef4444" color="#ef4444" />,
      { to: '/inventory', icon: <FileText size={18} fill="#8b5cf6" color="#8b5cf6"
      { to: '/transactions', icon: <ArrowLeftRight size={18} color="#0284c7" strokeWi
    ]
  },
  {
    label: 'FLIGHT MANAGEMENT',
    items: [
      { to: '/aircraft', icon: <Plane size={18} fill="#3b82f6" color="#3b82f6" />,
      { to: '/flights', icon: <PlaneTakeoff size={18} color="#3b82f6" strokeWidth={2.
      { to: '/daily-usage', icon: <CalendarDays size={18} color="#6366f1" fill="#e0e
    ]
  },
  {
    label: 'ANALYTICS',
    items: [
      { to: '/reports', icon: <FileText size={18} fill="#ffedd5" color="#f97316" />,
      { to: '/trends', icon: <TrendingUp size={18} color="#8b5cf6" strokeWidth={2.5}
    ]
  },
  ...(user?.role === 'admin' ? [{
    label: 'ADMIN',
    items: [
      { to: '/users', icon: <Users size={18} color="#64748b" />, label: 'User Manageme
    ]
  }] : [])
];

const handleLogout = () => {
  logout();
  navigate('/login');
};

return (
  <aside className="sidebar">
    </* Logo */>
    <div className="sidebar-header" style={{ display: 'flex', alignItems: 'center', ga
      <div style={{ width: 44, height: 44, borderRadius: 12, backgroundColor: '#3b82f6
        <Plane size={24} color="white" fill="white" />
      </div>
      <div>
        <h2 style={{ fontSize: '0.95rem', fontWeight: 800, margin: 0, lineHeight: 1.2,
          <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', fontWeight: 500
        </div>
      </div>
    </div>
  )

```

```

{ /* Nav */}
<nav className="sidebar-nav">
  {navSections.map((section) => (
    <div key={section.label}>
      <div className="sidebar-section-label" style={{ marginTop: 24, marginBottom:
        {section.items.map((item) => (
          <NavLink
            key={item.to}
            to={item.to}
            end={item.end}
            className={({ isActive }) => `nav-item ${isActive ? 'active' : ''}`}
          >
            {item.icon}
            <span>{item.label}</span>
          </NavLink>
        ))}
      </div>
    )})}
</nav>

{ /* Footer */}
<div className="sidebar-footer">
  <div style={{
    display: 'flex', alignItems: 'center', gap: '10px',
    padding: '10px 12px', marginBottom: '4px',
    background: 'var(--card-bg-subtle)',
    borderRadius: 'var(--radius-md)',
    border: '1px solid var(--border-color)'
  }}>
    <div style={{
      width: 32, height: 32, borderRadius: '50%',
      background: 'linear-gradient(135deg, #1d4ed8, #059669)',
      display: 'flex', alignItems: 'center', justifyContent: 'center',
      color: 'white', fontWeight: 700, fontSize: '13px', flexShrink: 0
    }}>
      {user?.username?.charAt(0).toUpperCase()}
    </div>
    <div style={{ overflow: 'hidden', flex: 1 }}>
      <div style={{ fontWeight: 600, fontSize: '0.85rem', color: 'var(--text-main)'
        {user?.username}
      </div>
      <div style={{ display: 'flex', alignItems: 'center', gap: 4 }}>
        <ShieldCheck size={10} style={{ color: user?.role === 'admin' ? 'var(--pri
        <span style={{ fontSize: '0.7rem', color: 'var(--text-muted)', textTransfo
      </div>
    </div>
  </div>

```

```

    </div>

    <button onClick={handleLogout} className="nav-item" style={{ color: 'var(--dange
      <LogOut size={18} />
      <span>Sign Out</span>
    </button>
  </div>
</aside>
);
};

export default Sidebar;

```

File: frontend/src/contexts/AuthContext.js

```

import React, { createContext, useContext, useState, useEffect } from 'react';
import { auth as authApi } from '../api/services';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const initAuth = async () => {
      const token = localStorage.getItem('token');
      if (token) {
        try {
          const res = await authApi.profile();
          setUser(res.data);
        } catch {
          localStorage.removeItem('token');
        }
      }
      setLoading(false);
    };
    initAuth();
  }, []);

  /**
   * Returns { success: true } or { success: false, error: string }
   */

```

```

const login = async (username, password) => {
  try {
    const res = await authApi.login(username, password);
    localStorage.setItem('token', res.data.access_token);
    setUser(res.data.user);
    return { success: true };
  } catch (err) {
    console.error('Login Error:', err);
    const backendError = err.response?.data?.error;
    const status = err.response?.status;
    const msg = backendError
      ? `${backendError} (Status: ${status})`
      : `Network/CORS Error: ${err.message}. Backend: ${err.config?.url}`;
    return { success: false, error: msg };
  }
};

const logout = () => {
  localStorage.removeItem('token');
  setUser(null);
};

return (
  <AuthContext.Provider value={{ user, loading, login, logout }}>
    {children}
  </AuthContext.Provider>
);
}

export const useAuth = () => useContext(AuthContext);

```

File: frontend/src/pages/AircraftPage.js

```

import React, { useState, useEffect, useCallback } from 'react';
import { aircraft as aircraftApi } from '../api/services';
import { PlaneTakeoff, Fuel, Plus, X, AlertCircle, CheckCircle, Gauge, Route } from 'lucide-react';
import { useAuth } from '../contexts/AuthContext';

const FUEL_TYPES = ['Jet A1', 'Avgas'];

const AircraftModal = ({ aircraft, onClose, onSuccess }) => {
  const isEdit = !!aircraft;
  const [form, setForm] = useState({

```

```

    aircraft_id: aircraft?.aircraft_id || '',
    manufacturer: aircraft?.manufacturer || '',
    model: aircraft?.model || '',
    year: aircraft?.year || '',
    fuel_type: aircraft?.fuel_type || 'Jet A1',
    fuel_tank_capacity_liters: aircraft?.fuel_tank_capacity_liters || '',
    fuel_consumption_rate: aircraft?.fuel_consumption_rate || '',
    max_range_km: aircraft?.max_range_km || '',
    is_active: aircraft?.is_active ?? true
  });
const [error, setError] = useState('');
const [saving, setSaving] = useState(false);

const set = (k, v) => setForm(f => ({ ...f, [k]: v }));

const handleSubmit = async (e) => {
  e.preventDefault();
  setError('');
  setSaving(true);
  try {
    const payload = {
      ...form,
      year: parseInt(form.year),
      fuel_tank_capacity_liters: parseFloat(form.fuel_tank_capacity_liters),
      fuel_consumption_rate: parseFloat(form.fuel_consumption_rate),
      max_range_km: parseFloat(form.max_range_km),
    };

    if (isEdit) {
      await aircraftApi.update(aircraft.id, payload);
    } else {
      await aircraftApi.create(payload);
    }
    onSuccess();
  } catch (err) {
    setError(err.response?.data?.error || `Failed to ${isEdit ? 'update' : 'register'}`);
  } finally {
    setSaving(false);
  }
};

return (
  <div className="modal-overlay" onClick={e => e.target === e.currentTarget} && onClose
    <div className="modal-box">
      <div className="modal-header">
        <div>
          <h2 className="text-h3">{isEdit ? 'Edit Aircraft' : 'Register Aircraft'}</h2>

```



```

    <p className="text-muted" style={{ marginTop: 4, fontSize: '0.85rem' }}>
      {isEdit ? `Updating ${aircraft.aircraft_id}` : 'Add a new aircraft to the
    </p>
  </div>
  <button className="btn-icon" onClick={onClose} id="close-aircraft-modal"><X si
</div>
<div className="modal-body">
  {error && (
    <div className="alert alert-error" style={{ marginBottom: 16 }}>
      <AlertCircle size={15} /><span style={{ fontSize: '0.85rem' }}>{error}</sp
    </div>
  )}
  <form onSubmit={handleSubmit}>
    <div className="form-grid">
      <div className="form-group">
        <label className="form-label">Aircraft ID *</label>
        <input id="ac-aircraft-id" className="form-input" placeholder="e.g. VT-A
      </div>
      <div className="form-group">
        <label className="form-label">Year *</label>
        <input id="ac-year" type="number" className="form-input" placeholder="20
      </div>
      <div className="form-group">
        <label className="form-label">Manufacturer *</label>
        <input id="ac-manufacturer" className="form-input" placeholder="Boeing"
      </div>
      <div className="form-group">
        <label className="form-label">Model *</label>
        <input id="ac-model" className="form-input" placeholder="737-800" value=
      </div>
      <div className="form-group">
        <label className="form-label">Fuel Type *</label>
        <select id="ac-fuel-type" className="form-select" value={form.fuel_type}
          {FUEL_TYPES.map(t => <option key={t}>{t}</option>)}
        </select>
      </div>
      <div className="form-group">
        <label className="form-label">Tank Capacity (L) *</label>
        <input id="ac-tank" type="number" step="0.1" className="form-input" plac
      </div>
      <div className="form-group">
        <label className="form-label">Consumption Rate (L/km) *</label>
        <input id="ac-rate" type="number" step="0.01" className="form-input" pla
      </div>
      <div className="form-group">
        <label className="form-label">Max Range (km) *</label>
        <input id="ac-range" type="number" className="form-input" placeholder="5

```

```

    </div>
    {isEdit && (
      <div className="form-group" style={{ gridColumn: 'span 2' }}>
        <label className="form-label" style={{ display: 'flex', alignItems: 'c
          <input type="checkbox" checked={form.is_active} onChange={e => set('
            Already Active / In Service
          </label>
        </div>
      )}
    </div>
    <div className="modal-footer">
      <button type="button" className="btn-secondary" onClick={onClose}>Cancel</
      <button id="save-aircraft" type="submit" className="btn-primary" disabled=
        {saving ? 'Saving...' : <><CheckCircle size={16} /> {isEdit ? 'Update Chan
      </button>
    </div>
  </form>
</div>
</div>
</div>
);
};

const AircraftPage = () => {
  const [fleet, setFleet] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showModal, setShowModal] = useState(false);
  const [editingAircraft, setEditingAircraft] = useState(null);
  const { user } = useAuth();

  const fetchFleet = useCallback(async () => {
    try {
      const res = await aircraftApi.list();
      setFleet(res.data);
    } catch (err) {
      console.error('Failed to fetch aircraft', err);
    } finally {
      setLoading(false);
    }
  }, []);

  useEffect(() => { fetchFleet(); }, [fetchFleet]);

  const handleSuccess = () => {
    setShowModal(false);
    setEditingAircraft(null);
    fetchFleet();
  }

```

```

};

const handleEdit = (ac) => {
  setEditingAircraft(ac);
  setShowModal(true);
};

const handleAdd = () => {
  setEditingAircraft(null);
  setShowModal(true);
};

if (loading) return <div className="loading-screen"><div className="spinner" /><span>L

return (
  <div>
    <div className="page-header">
      <div className="page-header-left">
        <h1 className="text-h1" style={{ display: 'flex', alignItems: 'center', gap: 1
          <div style={{ width: 32, height: 32, background: 'var(--primary-accent)', bo
            <PlaneTakeoff size={18} style={{ color: 'white' }} />
          </div>
          Aircraft
        </h1>
        <p className="text-muted">Manage your fleet registry and fuel specifications</
      </div>
      {user?.role === 'admin' && (
        <button id="add-aircraft-btn" className="btn-primary" onClick={handleAdd}>
          <Plus size={16} /> Add Aircraft
        </button>
      )}
    </div>

    {fleet.length === 0 ? (
      <div className="glass-card">
        <div className="empty-state">
          <div className="empty-state-icon">✈️</div>
          <h3 className="text-h3">No aircraft registered</h3>
          <p className="text-muted" style={{ marginTop: 8 }}>Add your first aircraft t
        </div>
      </div>
    ) : (
      <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(30
        {fleet.map(ac => {
          const pct = Math.min(100, (ac.fuel_tank_capacity_liters / 30000) * 100);
          return (
            <div key={ac.id} className="glass-card" style={{ padding: 24, display: 'fl

```

```

<div style={{ display: 'flex', justifyContent: 'space-between', alignIte
  <div>
    <h3 style={{ fontSize: '1.4rem', fontWeight: 800, color: 'var(--prim
      {ac.aircraft_id}
    </h3>
    <div style={{ fontWeight: 600, marginTop: 4, color: 'var(--text-main
      <div className="text-muted" style={{ fontSize: '0.85rem', marginTop:
    </div>
    <div style={{ display: 'flex', flexDirection: 'column', gap: 6, alignI
      <span className={`badge ${ac.is_active ? 'badge-green' : 'badge-gray
        {ac.is_active ? 'ACTIVE' : 'INACTIVE'}}
      </span>
      <span className={`badge ${ac.fuel_type === 'Jet A1' ? 'badge-blue' :
    </div>
  </div>

  <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 12,
    <div style={{ background: 'var(--card-bg-subtle)', padding: '12px 16px
      <div className="text-muted" style={{ display: 'flex', alignItems: 'c
        <Fuel size={12} style={{ color: 'var(--danger)' }} /> Tank Capacit
      </div>
      <div style={{ fontWeight: 800, fontSize: '1.1rem', marginTop: 4 }}>{
    </div>
    <div style={{ background: 'var(--card-bg-subtle)', padding: '12px 16px
      <div className="text-muted" style={{ display: 'flex', alignItems: 'c
        <Gauge size={12} style={{ color: 'var(--text-faint)' }} /> Consump
      </div>
      <div style={{ fontWeight: 800, fontSize: '1.1rem', marginTop: 4 }}>{
    </div>
    <div style={{ background: 'var(--card-bg-subtle)', padding: '12px 16px
      <div className="text-muted" style={{ display: 'flex', alignItems: 'c
        <Route size={12} style={{ color: 'var(--info)' }} /> Max Range
      </div>
      <div style={{ fontWeight: 800, fontSize: '1.1rem', marginTop: 4 }}>{
    </div>
    <div style={{ background: 'var(--card-bg-subtle)', padding: '12px 16px
      <div className="text-muted" style={{ display: 'flex', alignItems: 'c
        <PlaneTakeoff size={12} style={{ color: 'var(--text-faint)' }} />
      </div>
      <div style={{ fontWeight: 800, fontSize: '1.1rem', marginTop: 4 }}>{
    </div>
  </div>

  <div style={{ marginBottom: 20 }}>
    <div style={{ display: 'flex', justifyContent: 'space-between', fontSi
      <span>Tank Capacity</span>
      <span>{ac.fuel_tank_capacity_liters?.toLocaleString()} L</span>

```

```

        </div>
        <div className="progress-bar" style={{ height: 6, background: 'var(--b
          <div className="progress-fill" style={{
            width: `${pct}%`,
            background: ac.fuel_type === 'Jet A1' ? 'var(--primary-accent)' :
          }} />
        </div>
      </div>
    </div>

    <div style={{ display: 'flex', gap: 10, marginTop: 'auto' }}>
      <button className="btn-secondary" style={{ flex: 1, padding: '8px 0',
      <button className="btn-secondary" style={{ flex: 1, padding: '8px 0',
    </div>
  </div>
  );
  })}
</div>
)}
{showModal && (
  <AircraftModal
    aircraft={editingAircraft}
    onClose={() => { setShowModal(false); setEditingAircraft(null); }}
    onSuccess={handleSuccess}
  />
)}
</div>
);
};

export default AircraftPage;

```

File: frontend/src/pages/DailyUsagePage.js

```

import React, { useState, useEffect, useCallback } from 'react';
import { usage } from '../api/services';
import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip,
  ResponsiveContainer, Legend
} from 'recharts';
import { CalendarDays, RefreshCw } from 'lucide-react';

const DailyUsagePage = () => {

```

```

const [data, setData] = useState([]);
const [loading, setLoading] = useState(true);
const [days, setDays] = useState(14);

const fetchData = useCallback(async () => {
  setLoading(true);
  try {
    const res = await usage.list({ days });
    const raw = res.data.records || res.data;
    const formatted = raw.map(d => ({
      ...d,
      label: new Date(d.usage_date).toLocaleDateString('en-US', { month: 'short', day:
      jet_a1_used: d.jet_a1_used || 0,
      avgas_used: d.avgas_used || 0,
      total_fuel_used_liters: d.total_fuel_used_liters || 0,
      total_fuel_cost: d.total_fuel_cost || 0,
      total_flights: d.total_flights || 0,
    })));
    setData(formatted);
  } catch (err) {
    console.error('Failed to load daily usage', err);
  } finally {
    setLoading(false);
  }
}, [days]);

useEffect(() => { fetchData(); }, [fetchData]);

const totalFuel = data.reduce((s, d) => s + d.total_fuel_used_liters, 0);
const totalCost = data.reduce((s, d) => s + d.total_fuel_cost, 0);
const totalFlights = data.reduce((s, d) => s + d.total_flights, 0);
const avgCostPerL = totalFuel > 0 ? totalCost / totalFuel : 0;

return (
  <div>
    <div className="page-header">
      <div className="page-header-left">
        <h1 className="text-h1">Daily Fuel Usage</h1>
        <p className="text-muted">Jet A1 & Avgas consumption – stacked daily break
      </div>
      <div className="page-actions">
        <div style={{ display: 'flex', gap: 4, background: 'white', padding: 4, border
          [[7, 14, 30, 60].map(d => (
            <button
              key={d}
              id={`usage-range-${d}`}
              onClick={() => setDays(d)}

```

```

        style={{
          padding: '6px 14px', borderRadius: 7, border: 'none', cursor: 'pointer',
          fontWeight: 600, fontSize: '0.82rem', fontFamily: 'Plus Jakarta Sans',
          background: days === d ? 'var(--primary-accent)' : 'transparent',
          color: days === d ? 'white' : 'var(--text-muted)',
          transition: 'all 0.2s ease'
        }}
      >{d}D</button>
    )))
  </div>
  <button className="btn-secondary" onClick={fetchData} id="refresh-usage">
    <RefreshCw size={16} />
  </button>
</div>
</div>

{/* Summary KPIs */}
<div style={{ display: 'grid', gridTemplateColumns: 'repeat(4,1fr)', gap: 16, margin: 0 auto, width: 100% }}>
  {
    [
      { label: `Total Fuel (${days}D)`, value: `${totalFuel.toLocaleString(undefined, { notation: 'compact' })}`,
        { label: `Total Cost (${days}D)`, value: `₹${totalCost.toLocaleString(undefined, { notation: 'compact' })}`,
        { label: `Total Flights (${days}D)`, value: totalFlights,
        { label: 'Avg Cost / Litre', value: `₹${avgCostPerL.toFixed(2)}`,
    ].map(s => (
      <div key={s.label} className="glass-card card" style={{ padding: '18px 20px', width: 100% }}>
        <div className="text-label">{s.label}</div>
        <div style={{ fontFamily: 'Outfit,sans-serif', fontWeight: 800, fontSize: '1.2em' }}>{s.value}</div>
      </div>
    )))
  </div>

{/* Stacked bar chart */}
<div className="glass-card" style={{ padding: 24, marginBottom: 24 }}>
  <h3 className="text-h3" style={{ marginBottom: 4 }}>Stacked Daily Consumption</h3>
  <p className="text-muted" style={{ fontSize: '0.82rem', marginBottom: 20 }}>Jet
  {loading ? (
    <div className="loading-screen"><div className="spinner" /><span>Loading chart</span>
  ) : (
    <div style={{ height: 320 }}>
      <ResponsiveContainer width="100%" height="100%">
        <BarChart data={data} margin={{ top: 5, right: 10, left: 0, bottom: 0 }} b
          <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#e2e8f0" /
          <XAxis dataKey="label" stroke="#94a3b8" fontSize={11} tickLine={false} a
          <YAxis stroke="#94a3b8" fontSize={11} tickLine={false} axisLine={false}
          <Tooltip
            contentStyle={{ borderRadius: 10, border: '1px solid #e2e8f0', boxShad
            formatter={(v, name) => [`₹${v?.toLocaleString()} L`, name === 'jet_a1_

```

```

    />
    <Legend iconType="circle" iconSize={8} formatter={v => v === 'jet_a1_use
    <Bar dataKey="jet_a1_used" stackId="a" fill="#1d4ed8" name="jet_a1_used"
    <Bar dataKey="avgas_used" stackId="a" fill="#d97706" name="avgas_used"
    </BarChart>
  </ResponsiveContainer>
</div>
  )}
</div>

{/* Daily log table */}
<div className="glass-card" style={{ padding: 0, overflow: 'hidden' }}>
  <div style={{ padding: '18px 24px', borderBottom: '1px solid var(--border-color)'
    <CalendarDays size={18} style={{ color: 'var(--primary-accent)' }} />
    <h3 className="text-h3">Daily Log</h3>
  </div>
  <div className="table-container">
    <table className="data-table">
      <thead>
        <tr>
          <th>Date</th>
          <th style={{ textAlign: 'right' }}>Flights</th>
          <th style={{ textAlign: 'right' }}>Jet A1 (L)</th>
          <th style={{ textAlign: 'right' }}>Avgas (L)</th>
          <th style={{ textAlign: 'right' }}>Total Fuel (L)</th>
          <th style={{ textAlign: 'right' }}>Total Cost (₹)</th>
          <th style={{ textAlign: 'right' }}>Avg ₹ / L</th>
        </tr>
      </thead>
      <tbody>
        {data.slice().reverse().map(d => (
          <tr key={d.usage_date}>
            <td style={{ fontWeight: 600 }}>{d.label}</td>
            <td style={{ textAlign: 'right' }}>
              <span className="badge badge-blue">{d.total_flights}</span>
            </td>
            <td style={{ textAlign: 'right', color: 'var(--primary-accent)', fontW
              {d.jet_a1_used?.toLocaleString()}
            </td>
            <td style={{ textAlign: 'right', color: 'var(--warning)', fontWeight:
              {d.avgas_used?.toLocaleString()}
            </td>
            <td style={{ textAlign: 'right', fontWeight: 700 }}>
              {d.total_fuel_used_liters?.toLocaleString()}
            </td>
            <td style={{ textAlign: 'right', fontWeight: 700 }}>
              ₹{d.total_fuel_cost?.toLocaleString(undefined, { minimumFractionDigi

```



```

        </td>
        <td style={{ textAlign: 'right', color: 'var(--text-muted)' }}>
            ₹{d.total_fuel_used_liters > 0
                ? (d.total_fuel_cost / d.total_fuel_used_liters).toFixed(2)
                : '-'}
        </td>
    </tr>
    )))
    {data.length === 0 && !loading && (
        <tr><td colSpan="7" style={{ textAlign: 'center', padding: '48px', color
        )}
    </tbody>
    </table>
    </div>
    </div>
    </div>
    );
};

export default DailyUsagePage;

```

File: frontend/src/pages/DashboardPage.js

```

import React, { useState, useEffect } from 'react';
import { dashboard } from '../api/services';
import {
    BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip,
    ResponsiveContainer, AreaChart, Area, LineChart, Line, Legend,
    PieChart, Pie, Cell
} from 'recharts';
import {
    Fuel, DollarSign, Plane, Activity, AlertTriangle,
    TrendingUp, TrendingDown, Minus, RefreshCw, BarChart2, PieChart as PieIcon,
    Shield, CheckCircle
} from 'lucide-react';
import { useAuth } from '../contexts/AuthContext';

const KpiCard = ({ label, value, sub, icon, accent = 'blue', trend }) => {
    const colorMap = {
        blue: { bg: 'var(--primary-light)', color: 'var(--primary-accent)' },
        green: { bg: 'var(--secondary-light)', color: 'var(--secondary-accent)' },
        amber: { bg: 'var(--warning-light)', color: 'var(--warning)' },
        purple: { bg: 'var(--tertiary-light)', color: 'var(--tertiary-accent)' },
    }

```

```

    };
    const c = colorMap[accent];
    return (
      <div className="stat-card glass-card card" style={{ padding: '20px 16px' }}>
        <div className="stat-card-icon" style={{ background: c.bg, color: c.color }}>
          {icon}
        </div>
        <div>
          <div className="stat-label" style={{ fontSize: '0.65rem' }}>{label}</div>
          <div className="stat-value" style={{ fontSize: '1.4rem' }}>{value}</div>
          {sub && (
            <div className="stat-sub" style={{ fontSize: '0.7rem' }}>
              {trend === 'up' && <TrendingUp size={12} style={{ color: 'var(--danger)' }}
              {trend === 'down' && <TrendingDown size={12} style={{ color: 'var(--secondar'
              {!trend && <Minus size={12} />}
              {sub}
            </div>
          )}
        </div>
      </div>
    );
  };
};

const FuelSelector = ({ activeFuels, onToggle }) => (
  <div style={{ display: 'flex', gap: 8, marginBottom: 16 }}>
    [
      { label: 'Jet A1', color: '#1d4ed8' },
      { label: 'Avgas', color: '#059669' }
    ].map(f => (
      <button
        key={f.label}
        onClick={() => onToggle(f.label)}
        style={{
          padding: '6px 14px',
          borderRadius: '20px',
          fontSize: '0.75rem',
          fontWeight: 700,
          border: '1px solid',
          borderColor: activeFuels.includes(f.label) ? f.color : 'var(--border-color)',
          background: activeFuels.includes(f.label) ? `${f.color}15` : 'rgba(255,255,255',
          color: activeFuels.includes(f.label) ? f.color : 'var(--text-faint)',
          cursor: 'pointer',
          transition: 'all 0.3s cubic-bezier(0.4, 0, 0.2, 1)',
          display: 'flex',
          alignItems: 'center',
          gap: 8,
          boxShadow: activeFuels.includes(f.label) ? `0 2px 8px ${f.color}33` : 'none'
        }}
      >

```

```

    }}
  >
  <div style={{
    width: 8,
    height: 8,
    borderRadius: '50%',
    background: activeFuels.includes(f.label) ? f.color : '#94a3b8',
    boxShadow: activeFuels.includes(f.label) ? `0 0 8px ${f.color}` : 'none'
  }} />
  {f.label}
</button>
  )))
</div>
);

const DashboardPage = () => {
  const { user } = useAuth();
  const [kpis, setKpis] = useState(null);
  const [usageData, setUsageData] = useState([]);
  const [priceData, setPriceData] = useState([]);
  const [acData, setAcData] = useState([]);
  const [activeFuels, setActiveFuels] = useState(['Jet A1', 'Avgas']);
  const [loading, setLoading] = useState(true);
  const [lastRefresh, setLastRefresh] = useState(new Date());

  const toggleFuel = (fuel) => {
    setActiveFuels(prev => {
      if (prev.includes(fuel)) {
        if (prev.length === 1) return prev; // Keep at least one selected
        return prev.filter(f => f !== fuel);
      } else {
        return [...prev, fuel];
      }
    });
  };

  const fetchData = async () => {
    try {
      const [kpiRes, usageRes, priceRes, acRes] = await Promise.all([
        dashboard.getKPIs(),
        dashboard.getConsumptionChart(),
        dashboard.getPriceChart(),
        dashboard.getAircraftConsumption(),
      ]);
      setKpis(kpiRes.data);

      setUsageData(usageRes.data.map(d => ({

```

```

        ...d,
        label: new Date(d.usage_date).toLocaleDateString('en-US', { month: 'short', day:
    }));

    setPriceData(priceRes.data.map(d => ({
        ...d,
        label: new Date(d.date).toLocaleDateString('en-US', { month: 'short', day: 'nume
    }));

    setAcData(acRes.data);
    setLastRefresh(new Date());
  } catch (err) {
    console.error('Dashboard fetch error', err);
  } finally {
    setLoading(false);
  }
};

useEffect(() => { fetchData(); }, []);

if (loading) return (
  <div className="loading-screen" style={{
    backgroundImage: 'url("/landing_plane.png")',
    backgroundSize: 'cover',
    backgroundPosition: 'center',
    position: 'fixed',
    inset: 0,
    zIndex: 2000
  }}>
    <div style={{ position: 'absolute', inset: 0, background: 'rgba(255, 255, 255, 0.8'
    <div style={{ position: 'relative', zIndex: 1, display: 'flex', flexDirection: 'co
      <div className="spinner" style={{ width: 48, height: 48, borderWidth: 4 }} />
      <span style={{ fontWeight: 600, color: 'var(--primary-accent)', letterSpacing: '
    </div>
  </div>
);

const inventoryData = [
  { name: 'Jet A1', value: kpis?.fuel_stock?.['Jet A1'] || 0, color: '#1d4ed8' },
  { name: 'Avgas', value: kpis?.fuel_stock?.['Avgas'] || 0, color: '#059669' }
];

return (
  <div>
    { /* Premium Welcome Banner */ }
    <div className="glass-card" style={{
      marginBottom: 32,

```

```

padding: '32px 40px',
background: 'linear-gradient(135deg, rgba(29, 78, 216, 0.03) 0%, rgba(5, 150, 10
backgroundSize: 'cover',
backgroundPosition: 'center 45%',
borderRadius: 'var(--radius-xl)',
position: 'relative',
overflow: 'hidden',
border: '1px solid var(--border-color)',
display: 'flex',
justifyContent: 'space-between',
alignItems: 'center',
boxShadow: 'var(--shadow-lg)'
}}>
{/* Banner Overlay for readability */}
<div style={{
  position: 'absolute',
  inset: 0,
  background: 'linear-gradient(90deg, rgba(255, 255, 255, 0.95) 0%, rgba(255, 25
  zIndex: 1
}} />

<div style={{ position: 'relative', zIndex: 2 }}>
  <div style={{ display: 'flex', alignItems: 'center', gap: 10, color: 'var(--pr
    <div style={{ background: 'var(--primary-light)', padding: 6, borderRadius:
      <Shield size={20} />
    </div>
    <span style={{ fontSize: '0.75rem', fontWeight: 800, textTransform: 'upperca
      {user?.role} Portal Access
    </span>
  </div>
  <h1 style={{ fontSize: '2.2rem', fontWeight: 800, marginBottom: 8, color: 'var
    Welcome back, <span className="gradient-text">{user?.username}</span>
  </h1>
  <p style={{ color: 'var(--text-muted)', fontSize: '1rem', fontWeight: 500, max
    Here's the latest data for your aviation fuel operations as of {new Date().t
  </p>
</div>

<div style={{ position: 'relative', zIndex: 2, display: 'flex', gap: 12 }}>
  <div style={{ textAlign: 'right', display: 'flex', flexDirection: 'column', ga
    <div style={{ fontSize: '0.75rem', fontWeight: 700, color: 'var(--text-faint
      <div style={{ display: 'flex', alignItems: 'center', gap: 6, color: 'var(--s
        <CheckCircle size={14} /> Operational
      </div>
    </div>
  </div>
  <button className="btn-primary" onClick={fetchData} style={{ padding: '12px 24
    <RefreshCw size={18} /> Update Data

```

```

    </button>
  </div>
</div>

{/* Stats/KPIs Section Header */}
<div style={{ marginBottom: 20, display: 'flex', alignItems: 'center', gap: 10 }}>
  <div style={{ height: 1, flex: 1, background: 'var(--border-color)' }} />
  <span style={{ color: 'var(--text-faint)', fontSize: '0.7rem', fontWeight: 800,
    <div style={{ height: 1, flex: 1, background: 'var(--border-color)' }} />
  </div>

{/* Alerts */}
{kpis?.alerts?.length > 0 && (
  <div style={{ display: 'flex', flexDirection: 'column', gap: 10, marginBottom: 2
    {kpis.alerts.map((alert, idx) => (
      <div key={idx} className="alert alert-warning" style={{ alignItems: 'center'
        <AlertTriangle size={18} style={{ flexShrink: 0 }} />
        <div style={{ fontWeight: 600 }}>{alert.message}</div>
      </div>
    )))
  </div>
)}

{/* KPI Cards */}
<div style={{ display: 'grid', gridTemplateColumns: 'repeat(5, 1fr)', gap: 16, mar
  <KpiCard
    label="TOTAL FUEL STOCK"
    value={`$${(kpis?.fuel_stock?.['Jet A1'] + kpis?.fuel_stock?.['Avgas'] || 0).to
    icon=<Fuel size={20} />}
    accent="blue"
    sub={`Jet A1: ${((kpis?.fuel_stock?.['Jet A1'] || 0).toLocaleString())} L | Avgas
  />
  <KpiCard
    label="JET A1 PRICE"
    value={`₹${(kpis?.current_prices?.['Jet A1'] || '95.81')}`}
    icon=<DollarSign size={20} />}
    accent="amber"
    sub="per Liter (latest)"
  />
  <KpiCard
    label="TODAY'S USAGE"
    value={`$${(kpis?.today?.fuel_used_liters || 0).toLocaleString()} L`}
    icon=<Activity size={20} />}
    accent="green"
    sub={`$${(kpis?.today?.flights || 0)} flights · ₹0`}
  />
  <KpiCard

```

```

    label="MONTHLY FLIGHTS"
    value={kpis?.this_month?.total_flights || 0}
    icon={<Plane size={20} />}
    accent="purple"
    sub={`₹$${((kpis?.this_month?.total_fuel_cost || 0)/1000).toFixed(1)}K cost thi
  />
<KpiCard
  label="ACTIVE AIRCRAFT"
  value={kpis?.total_active_aircraft || 0}
  icon={<Plane size={20} />}
  accent="blue"
  sub={`$${kpis?.active_flights || 0} flights scheduled`}
/>
<KpiCard
  label="MONTHLY FUEL USED"
  value={`$${((kpis?.this_month?.total_fuel_used_liters || 0)/1000).toFixed(1)}K
  icon={<Fuel size={20} />}
  accent="purple"
  sub="liters this month"
/>
</div>

{/* Charts Row 1 */}
<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 20, marginBott
  {/* Consumption Area Chart */}
  <div className="glass-card card" style={{ padding: 20 }}>
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'f
      <h3 className="text-h3" style={{ display: 'flex', alignItems: 'center', gap:
        <Activity size={16} style={{ color: 'var(--primary-accent)' }} /> Daily F
      </h3>
    </div>
    <p style={{ fontSize: '0.75rem', color: 'var(--text-muted)', marginBottom: 12
    <FuelSelector activeFuels={activeFuels} onToggle={toggleFuel} />

    <div style={{ height: 210 }}>
      <ResponsiveContainer width="100%" height="100%">
        <AreaChart data={usageData} margin={{ top: 5, right: 10, left: 0, bottom:
          <defs>
            <linearGradient id="gradJet" x1="0" y1="0" x2="0" y2="1">
              <stop offset="5%" stopColor="#1d4ed8" stopOpacity={0.12} />
              <stop offset="95%" stopColor="#1d4ed8" stopOpacity={0} />
            </linearGradient>
            <linearGradient id="gradAvgas" x1="0" y1="0" x2="0" y2="1">
              <stop offset="5%" stopColor="#059669" stopOpacity={0.12} />
              <stop offset="95%" stopColor="#059669" stopOpacity={0} />
            </linearGradient>
          </defs>

```

```

<CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#e2e8f0" /
<XAxis dataKey="label" stroke="#94a3b8" fontSize={11} tickLine={false} a
<YAxis stroke="#94a3b8" fontSize={11} tickLine={false} axisLine={false}
<Tooltip
  contentStyle={{ borderRadius: 10, border: '1px solid #e2e8f0', boxShad
  formatter={(v, name) => [`$${v?.toLocaleString()} L`, name] === 'jet_a1_
/>
<Legend iconType="circle" iconSize={8} />
{activeFuels.includes('Jet A1')} && (
  <Area type="monotone" dataKey="jet_a1_used" name="Jet A1" stroke="#1d4
)}
{activeFuels.includes('Avgas')} && (
  <Area type="monotone" dataKey="avgas_used" name="Avgas" stroke="#059
)}
</AreaChart>
</ResponsiveContainer>
</div>
</div>

{/* Price Trend Chart */}
<div className="glass-card card" style={{ padding: 20 }}>
  <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'f
    <h3 className="text-h3" style={{ display: 'flex', alignItems: 'center', gap:
      <TrendingUp size={16} style={{ color: 'var(--tertiary-accent)' }} /> Fuel
    </h3>
  </div>
  <p style={{ fontSize: '0.75rem', color: 'var(--text-muted)', marginBottom: 12
  <FuelSelector activeFuels={activeFuels} onToggle={toggleFuel} />

  <div style={{ height: 210 }}>
    <ResponsiveContainer width="100%" height="100%">
      <LineChart data={priceData} margin={{ top: 5, right: 10, left: 0, bottom:
        <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#e2e8f0" /
        <XAxis dataKey="label" stroke="#94a3b8" fontSize={11} tickLine={false} a
        <YAxis stroke="#94a3b8" fontSize={11} tickLine={false} axisLine={false}
        <Tooltip
          contentStyle={{ borderRadius: 10, border: '1px solid #e2e8f0', boxShad
          formatter={(v, name) => [`$${v?.toFixed(2)}`, name]}
        />
        <Legend iconType="circle" iconSize={8} />
        {activeFuels.includes('Jet A1')} && (
          <Line type="monotone" dataKey="jet_a1_price" name="Jet A1" stroke="#1d
        )}
        {activeFuels.includes('Avgas')} && (
          <Line type="monotone" dataKey="avgas_price" name="Avgas" stroke="#05
        )}
      </LineChart>

```



```

    </ResponsiveContainer>
  </div>
</div>
</div>

{/* Charts Row 2 */}
<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 20, marginBott
  {/* Aircraft Consumption Bar Chart */}
  <div className="glass-card card" style={{ padding: 20 }}>
    <h3 className="text-h3" style={{ marginBottom: 4, display:'flex', alignItems:
      <BarChart2 size={16} style={{ color: 'var(--secondary-accent)' }} /> Aircra
    </h3>
    <div style={{ height: 210, marginTop: 16 }}>
      <ResponsiveContainer width="100%" height="100%">
        <BarChart data={acData} margin={{ top: 5, right: 10, left: 0, bottom: 0 }}
          <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#e2e8f0" /
          <XAxis dataKey="model" stroke="#94a3b8" fontSize={11} tickLine={false} a
          <YAxis stroke="#94a3b8" fontSize={11} tickLine={false} axisLine={false}
          <Tooltip
            contentStyle={{ borderRadius: 10, border: '1px solid #e2e8f0', boxShad
              formatter={(v) => [`${v?.toLocaleString()} L`, 'Total Fuel Used']}
            />
          <Bar dataKey="total_fuel_used" name="Total Fuel Used" fill="var(--primar
        </BarChart>
      </ResponsiveContainer>
    </div>
  </div>

{/* Current Inventory Pie Chart */}
<div className="glass-card card" style={{ padding: 20 }}>
  <h3 className="text-h3" style={{ marginBottom: 4, display:'flex', alignItems:
    <PieIcon size={16} style={{ color: 'var(--warning)' }} /> Stock Distributio
  </h3>
  <div style={{ height: 210, marginTop: 16 }}>
    <ResponsiveContainer width="100%" height="100%">
      <PieChart>
        <Pie data={inventoryData} cx="50%" cy="50%" innerRadius={70} outerRadius
          {inventoryData.map((entry, index) => (
            <Cell key={`cell-${index}`} fill={entry.color} />
          ))}
        </Pie>
        <Tooltip
          contentStyle={{ borderRadius: 10, border: '1px solid #e2e8f0', boxShad
            formatter={(v, name) => [`${v?.toLocaleString()} L`, name]}
          />
        <Legend iconType="circle" iconSize={8} verticalAlign="bottom" height={36
      </PieChart>

```

```

        </ResponsiveContainer>
      </div>
    </div>
  </div>
</div>
);
};

export default DashboardPage;

```

File: frontend/src/pages/FlightsPage.js

```

import React, { useState, useEffect, useCallback } from 'react';
import { flights, aircraft as aircraftApi } from '../api/services';
import { Plus, X, AlertCircle, CheckCircle, Navigation2, MapPin, Filter } from 'lucide-r

const STATUS_BADGE = {
  scheduled: { cls: 'badge-amber', label: 'Scheduled' },
  completed: { cls: 'badge-green', label: 'Completed' },
  cancelled: { cls: 'badge-red', label: 'Cancelled' },
  in_flight: { cls: 'badge-blue', label: 'In Flight' },
};

/* — Schedule Modal ————— */
const ScheduleModal = ({ fleet, onClose, onSuccess }) => {
  const [form, setForm] = useState({
    flight_number: '', aircraft_id_fk: '', source: '', destination: '',
    distance_km: '', flight_date: new Date().toISOString().slice(0,10), notes: ''
  });
  const [error, setError] = useState('');
  const [saving, setSaving] = useState(false);

  const set = (k,v) => setForm(f => ({ ...f, [k]: v }));

  const selectedAc = fleet.find(a => a.id === parseInt(form.aircraft_id_fk));
  const reqFuel = selectedAc && form.distance_km
    ? (parseFloat(form.distance_km) * selectedAc.fuel_consumption_rate).toFixed(1)
    : null;

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setSaving(true);

```

```

try {
  await flights.create({
    ...form,
    aircraft_id_fk: parseInt(form.aircraft_id_fk),
    distance_km: parseFloat(form.distance_km),
  });
  onSuccess();
} catch (err) {
  setError(err.response?.data?.error || 'Failed to schedule flight');
} finally {
  setSaving(false);
}
};

return (
  <div className="modal-overlay" onClick={e => e.target===e.currentTarget && onClose()}
    <div className="modal-box" style={{ maxWidth: 560 }}>
      <div className="modal-header">
        <div>
          <h2 className="text-h3">Schedule Flight</h2>
          <p className="text-muted" style={{ marginTop:4, fontSize:'0.85rem' }}>Create
        </div>
          <button className="btn-icon" onClick={onClose} id="close-schedule-modal"><X si
        </div>
      <div className="modal-body">
        {error && <div className="alert alert-error" style={{ marginBottom:16 }}><Aler
        <form onSubmit={handleSubmit}>
          <div className="form-grid">
            <div className="form-group">
              <label className="form-label">Flight Number *</label>
              <input id="fl-number" className="form-input" placeholder="AF-2024-001" v
            </div>
            <div className="form-group">
              <label className="form-label">Flight Date *</label>
              <input id="fl-date" type="date" className="form-input" value={form.fligh
            </div>
            <div className="form-group" style={{ gridColumn:'1/-1' }}>
              <label className="form-label">Aircraft *</label>
              <select id="fl-aircraft" className="form-select" value={form.aircraft_id.
                <option value="">- Select aircraft -</option>
                {fleet.map(a => <option key={a.id} value={a.id}>{a.aircraft_id} · {a.m
              </select>
            </div>
            <div className="form-group">
              <label className="form-label">Source *</label>
              <input id="fl-source" className="form-input" placeholder="Mumbai (BOM)"
            </div>
          </div>
        </form>
      </div>
    </div>
  </div>
);

```

```

    <div className="form-group">
      <label className="form-label">Destination *</label>
      <input id="fl-dest" className="form-input" placeholder="Delhi (DEL)" val
    </div>
    <div className="form-group">
      <label className="form-label">Distance (km) *</label>
      <input id="fl-dist" type="number" step="0.1" className="form-input" plac
    </div>
    <div className="form-group">
      <label className="form-label">Required Fuel (auto)</label>
      <input className="form-input" value={reqFuel ? `${reqFuel} L` : 'Select
    </div>
  </div>
  <div className="form-group">
    <label className="form-label">Notes</label>
    <input id="fl-notes" className="form-input" placeholder="Optional notes" v
  </div>
  <div className="modal-footer">
    <button type="button" className="btn-secondary" onClick={onClose}>Cancel</
    <button id="save-flight" type="submit" className="btn-primary" disabled={s
      {saving ? 'Scheduling...' : <><Navigation2 size={16}/> Schedule Flight</>
    </button>
  </div>
</form>
</div>
</div>
</div>
);
};

/* — Complete Modal ————— */
const CompleteModal = ({ flight, onClose, onSuccess }) => {
  const [actualFuel, setActualFuel] = useState('');
  const [error, setError] = useState('');
  const [saving, setSaving] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setSaving(true);
    try {
      await flights.complete(flight.id, { actual_fuel_used_liters: parseFloat(actualFuel)
      onSuccess();
    } catch (err) {
      setError(err.response?.data?.error || 'Failed to complete flight');
      setSaving(false);
    }
  };
};

```

```

return (
  <div className="modal-overlay" onClick={e=>e.target===e.currentTarget&&onClose()}>
    <div className="modal-box" style={{ maxWidth: 440 }}>
      <div className="modal-header">
        <div>
          <h2 className="text-h3">Complete Flight</h2>
          <p className="text-muted" style={{ marginTop:4, fontSize:'0.85rem' }}>{flight}
        </div>
          <button className="btn-icon" onClick={onClose}><X size={16}/></button>
        </div>
      <div className="modal-body">
        <div style={{ padding:'14px 16px', background:'var(--primary-light)', borderRa
          <span className="text-label">ALLOCATED FUEL</span>
          <div style={{ fontWeight:700, fontSize:'1.4rem', color:'var(--primary-accent
        </div>
        {error && <div className="alert alert-error" style={{marginBottom:16}}><AlertC
        <form onSubmit={handleSubmit}>
          <div className="form-group">
            <label className="form-label">Actual Fuel Used (L) *</label>
            <input id="complete-fuel" type="number" step="0.1" className="form-input"
              {actualFuel && (
                <div style={{ marginTop:6, fontSize:'0.82rem', color: parseFloat(actualF
                  {parseFloat(actualFuel) < flight.required_fuel_liters
                    ? `✓ Saved ${((flight.required_fuel_liters - parseFloat(actualFuel)).
                      : `⚠ ${((parseFloat(actualFuel) - flight.required_fuel_liters).toFixe
                </div>
              )}
            </div>
          <div className="modal-footer">
            <button type="button" className="btn-secondary" onClick={onClose}>Cancel</
            <button id="confirm-complete" type="submit" className="btn-success" disabl
              {saving ? 'Completing...' : <><CheckCircle size={16}/> Mark Completed</>
            </button>
          </div>
        </form>
      </div>
    </div>
  </div>
);
};

/* — Main Page ————— */
const FlightsPage = () => {
  const [list, setList]      = useState([]);
  const [total, setTotal]    = useState(0);
  const [fleet, setFleet]    = useState([]);

```

```

const [loading, setLoading] = useState(true);
const [showSchedule, setShowSchedule] = useState(false);
const [completeTarget, setCompleteTarget] = useState(null);
const [statusFilter, setStatusFilter] = useState('');
const [page, setPage] = useState(1);
const perPage = 15;

const fetchFlights = useCallback(async () => {
  setLoading(true);
  try {
    const params = { page, per_page: perPage };
    if (statusFilter) params.status = statusFilter;
    const [flRes, acRes] = await Promise.all([flights.list(params), aircraftApi.list()]);
    const data = flRes.data;
    setList(data.flights || data);
    setTotal(data.total || (data.flights || data).length);
    setFleet(acRes.data);
  } catch (err) {
    console.error(err);
  } finally {
    setLoading(false);
  }
}, [page, statusFilter]);

useEffect(() => { fetchFlights(); }, [fetchFlights]);

const handleCancel = async (id) => {
  if (!window.confirm('Cancel this flight and return fuel to inventory?')) return;
  try { await flights.cancel(id); fetchFlights(); } catch (err) { alert(err.response?.); }
};

const totalPages = Math.ceil(total / perPage);

return (
  <div>
    <div className="page-header">
      <div className="page-header-left">
        <h1 className="text-h1">Flights & Trips</h1>
        <p className="text-muted">Trip management with automatic fuel allocation – {to
      </div>
      <div className="page-actions">
        <div style={{ position: 'relative' }}>
          <Filter size={14} style={{ position: 'absolute', left: 10, top: '50%', transform:
          <select id="flight-filter-status" className="form-select" style={{ padding: 10px
            <option value="">All Status</option>
            <option value="scheduled">Scheduled</option>
            <option value="completed">Completed</option>

```

```

      <option value="cancelled">Cancelled</option>
    </select>
  </div>
  <button id="schedule-flight-btn" className="btn-primary" onClick={() => setSho
    <Plus size={16}/> Schedule Trip
  </button>
</div>
</div>

<div className="glass-card" style={{ padding:0, overflow:'hidden' }}>
  {loading ? (
    <div className="loading-screen"><div className="spinner"/><span>Loading flight
  ) : (
    <>
      <div className="table-container">
        <table className="data-table">
          <thead>
            <tr>
              <th>Flight #</th>
              <th>Date</th>
              <th>Aircraft</th>
              <th>Route</th>
              <th style={{textAlign:'right'}}>Dist (km)</th>
              <th style={{textAlign:'right'}}>Req. Fuel</th>
              <th style={{textAlign:'right'}}>Actual</th>
              <th style={{textAlign:'right'}}>Trip Cost</th>
              <th>Status</th>
              <th>Actions</th>
            </tr>
          </thead>
          <tbody>
            {list.map(f => {
              const badge = STATUS_BADGE[f.status] || { cls:'badge-gray', label: f
              return (
                <tr key={f.id}>
                  <td style={{ fontWeight:700, fontFamily:'Outfit,sans-serif' }}>{
                    <td style={{ color:'var(--text-muted)', fontSize:'0.85rem' }}>{f
                    <td>
                      <span style={{ fontWeight:600, color:'var(--primary-accent)' }
                    </td>
                    <td>
                      <div style={{ display:'flex', alignItems:'center', gap:5 }}>
                        <MapPin size={12} style={{ color:'var(--text-faint)', flexSh
                        <span style={{ fontSize:'0.88rem' }}>{f.source} → {f.destina
                      </div>
                    </td>
                    <td style={{ textAlign:'right' }}>{f.distance_km?.toLocaleString

```

```

<td style={{ textAlign:'right', color:'var(--primary-accent)', f
<td style={{ textAlign:'right', color: f.actual_fuel_used_liters
    {f.actual_fuel_used_liters ? `${f.actual_fuel_used_liters?.tol
</td>
<td style={{ textAlign:'right', fontWeight:700 }}>
    {f.trip_fuel_cost ? `₹${parseFloat(f.trip_fuel_cost).toLocaleS
</td>
<td><span className={`badge ${badge.cls}`}>{badge.label}</span><
<td>
    <div style={{ display:'flex', gap:6 }}>
        {f.status === 'scheduled' && (
            <>
                <button className="btn-success" style={{ padding:'5px 10
                    ✓ Done
                </button>
                <button className="btn-danger" style={{ padding:'5px 10p
                    ✕ Cancel
                </button>
            </>
        )}
    </div>
</td>
</tr>
);
}}
{list.length === 0 && (
    <tr><td colspan="10" style={{ textAlign:'center', padding:'48px', co
    }}
</tbody>
</table>
</div>
{totalPages > 1 && (
    <div style={{ display:'flex', justifyContent:'space-between', alignItems:'
        <span className="text-muted" style={{ fontSize:'0.85rem' }}>Page {page}
        <div style={{ display:'flex', gap:8 }}>
            <button className="btn-secondary" style={{ padding:'6px 14px', fontSiz
            <button className="btn-secondary" style={{ padding:'6px 14px', fontSiz
        </div>
    </div>
    )}
</>
)}
</div>

{showSchedule && <ScheduleModal fleet={fleet} onClose={()=>setShowSchedule(false)}
{completeTarget && <CompleteModal flight={completeTarget} onClose={()=>setComplete
</div>

```



```
);  
};  
  
export default FlightsPage;
```

File: frontend/src/pages/FuelInventoryPage.js

```
import React, { useState, useEffect, useCallback } from 'react';  
import { inventory } from '../api/services';  
import { Database, ArrowDownRight, ArrowUpRight, TrendingDown, RefreshCw, AlertTriangle }  
  
const TX_BADGE = {  
  allocation: { label: 'Allocation', cls: 'badge-amber' },  
  consumption: { label: 'Consumed', cls: 'badge-red' },  
  refund: { label: 'Refund', cls: 'badge-green' },  
  purchase: { label: 'Purchase', cls: 'badge-blue' },  
};  
  
const CAPACITY = 250000; // assumed max for gauge  
  
const FuelInventoryPage = () => {  
  const [levels, setLevels] = useState([]);  
  const [txs, setTxs] = useState([]);  
  const [txTotal, setTxTotal] = useState(0);  
  const [loading, setLoading] = useState(true);  
  const [txPage, setTxPage] = useState(1);  
  const perPage = 15;  
  
  const fetchData = useCallback(async () => {  
    setLoading(true);  
    try {  
      const [invRes, txRes] = await Promise.all([  
        inventory.getLevels(),  
        inventory.getTransactions({ page: txPage, per_page: perPage })  
      ]);  
      setLevels(invRes.data);  
      const txData = txRes.data;  
      setTxs(txData.transactions || txData);  
      setTxTotal(txData.total || (txData.transactions || txData).length);  
    } catch (err) {  
      console.error('Failed to fetch inventory', err);  
    } finally {  
      setLoading(false);  
    }  
  });  
};
```

```

    }
  }, [txPage]);

useEffect(() => { fetchData(); }, [fetchData]);

const txPages = Math.ceil(txTotal / perPage);

return (
  <div>
    <div className="page-header">
      <div className="page-header-left">
        <h1 className="text-h1">Fuel Inventory</h1>
        <p className="text-muted">Live stock levels and transaction movements</p>
      </div>
      <button className="btn-secondary" onClick={fetchData} id="refresh-inventory">
        <RefreshCw size={16} /> Refresh
      </button>
    </div>

    { /* Stock Gauges */ }
    <div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 20, marginBott
      {levels.map(inv => {
        const pct = Math.min(100, (inv.total_quantity_liters / CAPACITY) * 100);
        const low = inv.total_quantity_liters < 50000;
        const isJet = inv.fuel_type === 'Jet A1';
        const fillColor = isJet
          ? 'linear-gradient(90deg,#1d4ed8,#3b82f6)'
          : 'linear-gradient(90deg,#d97706,#f59e0b)';
        const accent = isJet ? 'var(--primary-accent)' : 'var(--warning)';

        return (
          <div key={inv.fuel_type} className="glass-card card" style={{ padding: '24px
            <div style={{ width: 48, height: 48, borderRadius: 12, background: 'var(--
              <Database size={24} style={{ color: 'white' }} />
            </div>
            <div style={{ flex: 1 }}>
              <div style={{ display: 'flex', justifyContent: 'space-between', alignIte
                <div className="text-label">{inv.fuel_type} STOCK</div>
              </div>
              <div style={{ fontFamily: 'Outfit,sans-serif', fontSize: '2.2rem', fontW
                {inv.total_quantity_liters >= 1000 ? (inv.total_quantity_liters / 1000
              </div>
              {low && (
                <div style={{ display: 'flex', alignItems: 'center', gap: 6, color: 'v
                  <AlertTriangle size={14} /> Low Stock Alert
                </div>
              )}
            </div>
          )}
        )}
      )}
    </div>
  )}

```

```

        <div className="progress-bar" style={{ height: 6, background: 'var(--car
          <div className="progress-fill" style={{ width: `${pct}%`, background:
        </div>
        <div style={{ fontSize: '0.75rem', color: 'var(--text-muted)', marginTop
          Updated: {new Date().toISOString().slice(0,10)}
        </div>
      </div>
    </div>
  );
}}
</div>

{/* Transaction History */}
<h3 className="text-h3" style={{ marginBottom: 16 }}>Transaction History</h3>
<div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'cente
  <div style={{ display: 'flex', gap: 10 }}>
    <select className="form-select" style={{ width: 140, height: 38, background: '
      <option>All Types</option>
    </select>
    <select className="form-select" style={{ width: 140, height: 38, background: '
      <option>All Fuel Types</option>
    </select>
    <button className="btn-secondary" style={{ padding: '6px 14px', height: 38 }}>
  </div>
  <span className="text-muted" style={{ fontSize: '0.85rem' }}>{txTotal} records</
</div>

<div className="glass-card" style={{ padding: 0, overflow: 'hidden' }}>
  {loading ? (
    <div className="loading-screen"><div className="spinner" /><span>Loading trans
  ) : txs.length === 0 ? (
    <div className="empty-state">
      <div style={{ margin: '0 auto', width: 64, height: 64, background: 'var(--pr
        <RefreshCw size={28} style={{ color: 'white' }} />
      </div>
      <div style={{ fontSize: '1.1rem', fontWeight: 500, color: 'var(--text-sub)'
    </div>
  ) : (
    <>
      <div className="table-container">
        <table className="data-table">
          <thead>
            <tr>
              <th>Timestamp</th>
              <th>Type</th>
              <th>Fuel</th>
              <th>Flight Ref</th>

```

```

        <th style={{ textAlign: 'right' }}>Quantity (L)</th>
        <th style={{ textAlign: 'right' }}>₹ / L</th>
        <th style={{ textAlign: 'right' }}>Value (₹)</th>
      </tr>
    </thead>
    <tbody>
      {txs.map(tx => {
        const badge = TX_BADGE[tx.transaction_type] || { label: tx.transacti
        const isIn = tx.transaction_type === 'refund' || tx.transaction_type
        return (
          <tr key={tx.id}>
            <td style={{ color: 'var(--text-muted)', fontSize: '0.82rem' }}>
              {new Date(tx.transaction_date).toLocaleString('en-IN', { day:
            </td>
            <td><span className={`badge ${badge.cls}`}>{badge.label}</span><
            <td><span style={{ fontWeight: 600 }}>{tx.fuel_type}</span></td>
            <td style={{ fontFamily: 'monospace', fontSize: '0.82rem', color:
            <td style={{ textAlign: 'right', fontWeight: 600, color: isIn ?
              {isIn ? '+' : '-'}>{tx.quantity_liters?.toLocaleString()}
            </td>
            <td style={{ textAlign: 'right', color: 'var(--text-muted)' }}>₹
            <td style={{ textAlign: 'right', fontWeight: 600 }}>₹{parseFloat
          </tr>
        );
      })}
    </tbody>
  </table>
</div>
{txPages > 1 && (
  <div style={{ display: 'flex', justifyContent: 'space-between', alignItems
    <span className="text-muted" style={{ fontSize: '0.85rem' }}>Page {txPag
    <div style={{ display: 'flex', gap: 8 }}>
      <button className="btn-secondary" style={{ padding: '6px 14px', fontSi
      <button className="btn-secondary" style={{ padding: '6px 14px', fontSi
    </div>
  </div>
)}
</>
)}
</div>
</div>
);
};

export default FuelInventoryPage;

```

File: frontend/src/pages/FuelPurchasesPage.js

```
import React, { useState, useEffect, useCallback } from 'react';
import { purchases, reports } from '../api/services';
import { Plus, X, AlertCircle, CheckCircle, Filter, Search, FileText } from 'lucide-react';
import { useAuth } from '../contexts/AuthContext';

const FUEL_TYPES = ['', 'Jet A1', 'Avgas'];

const AddPurchaseModal = ({ onClose, onSuccess, currentPrices }) => {
  const [form, setForm] = useState({
    supplier_name: '', location: '', fuel_type: 'Jet A1',
    quantity_liters: '', price_per_liter: currentPrices?['Jet A1']?.price_per_liter ||
    invoice_number: '', notes: ''
  });
  const [error, setError] = useState('');
  const [saving, setSaving] = useState(false);

  useEffect(() => {
    if (currentPrices && currentPrices[form.fuel_type]?.price_per_liter) {
      setForm(f => ({ ...f, price_per_liter: currentPrices[form.fuel_type].price_per_liter }));
    }
  }, [form.fuel_type, currentPrices]);

  const set = (k, v) => setForm(f => ({ ...f, [k]: v }));

  const total = ((parseFloat(form.quantity_liters) || 0) * (parseFloat(form.price_per_liter) || 0));

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setSaving(true);
    try {
      await purchases.create({
        ...form,
        quantity_liters: parseFloat(form.quantity_liters),
        price_per_liter: parseFloat(form.price_per_liter),
      });
      onSuccess();
    } catch (err) {
      setError(err.response?.data?.error || 'Failed to record purchase');
    } finally {
      setSaving(false);
    }
  };
};
```

```

return (
  <div className="modal-overlay" onClick={e => e.target === e.currentTarget && onClose}
    <div className="modal-box" style={{ maxWidth: 560 }}>
      <div className="modal-header">
        <div>
          <h2 className="text-h3">Record Fuel Purchase</h2>
          <p className="text-muted" style={{ marginTop: 4, fontSize: '0.85rem' }}>Log
        </div>
          <button className="btn-icon" onClick={onClose} id="close-purchase-modal">X si
        </div>
      <div className="modal-body">
        {error && (
          <div className="alert alert-error" style={{ marginBottom: 16 }}>
            <AlertCircle size={15} /><span style={{ fontSize: '0.85rem' }}>{error}</sp
          </div>
        )}
        <form onSubmit={handleSubmit}>
          <div className="form-grid">
            <div className="form-group">
              <label className="form-label">Supplier Name *</label>
              <input id="pur-supplier" className="form-input" placeholder="AirFuel Ind
            </div>
            <div className="form-group">
              <label className="form-label">Location *</label>
              <input id="pur-location" className="form-input" placeholder="Mumbai Airp
            </div>
            <div className="form-group">
              <label className="form-label">Fuel Type *</label>
              <select id="pur-fuel-type" className="form-select" value={form.fuel_type}
                <option>Jet A1</option>
                <option>Avgas</option>
              </select>
            </div>
            <div className="form-group">
              <label className="form-label">Purchase Date *</label>
              <input id="pur-date" type="date" className="form-input" value={form.purc
            </div>
            <div className="form-group">
              <label className="form-label">Quantity (Litres) *</label>
              <input id="pur-qty" type="number" step="0.1" className="form-input" plac
            </div>
            <div className="form-group">
              <label className="form-label">Price / Litre (₹) *</label>
              <input id="pur-price" type="number" step="0.01" className="form-input" p
            </div>
            <div className="form-group">

```

```

        <label className="form-label">Invoice Number</label>
        <input id="pur-invoice" className="form-input" placeholder="INV-2024-001" />
      </div>
      <div className="form-group">
        <label className="form-label">Total Cost</label>
        <input className="form-input" value={`₹ ${parseFloat(total).toLocaleString()} `} />
      </div>
    </div>
    <div className="form-group">
      <label className="form-label">Notes</label>
      <input id="pur-notes" className="form-input" placeholder="Optional notes" />
    </div>
    <div className="modal-footer">
      <button type="button" className="btn-secondary" onClick={onClose}>Cancel</button>
      <button id="save-purchase" type="submit" className="btn-primary" disabled={saving ? 'Saving...' : <><CheckCircle size={16} /> Record Purchase</>} />
    </div>
  </form>
</div>
</div>
</div>
);
};

```

```

const FuelPurchasesPage = () => {
  const [list, setList] = useState([]);
  const [total, setTotal] = useState(0);
  const [currentPrices, setCurrentPrices] = useState(null);
  const [loading, setLoading] = useState(true);
  const [showModal, setShowModal] = useState(false);
  const [fuelFilter, setFuelFilter] = useState('');
  const [startDate, setStartDate] = useState('');
  const [endDate, setEndDate] = useState('');
  const [page, setPage] = useState(1);
  const perPage = 15;
  const { user } = useAuth();

  const fetchPurchases = useCallback(async () => {
    setLoading(true);
    try {
      const params = { page, per_page: perPage };
      if (fuelFilter) params.fuel_type = fuelFilter;
      if (startDate) params.start_date = startDate;
      if (endDate) params.end_date = endDate;

      const [res, priceRes] = await Promise.all([

```

```
        purchases.list(params),
        purchases.currentPrice()
    ]]);
    const data = res.data;
    setList(data.purchases || data);
    setTotal(data.total || (data.purchases || data).length);
    setCurrentPrices(priceRes.data);
  } catch (err) {
    console.error('Failed to load purchases', err);
  } finally {
    setLoading(false);
  }
}, [page, fuelFilter, startDate, endDate]);

useEffect(() => { fetchPurchases(); }, [fetchPurchases]);

const handleAdded = () => { setShowModal(false); fetchPurchases(); };

const handleExportPDF = async () => {
  try {
    const res = await reports.exportPdf({
      report_type: 'fuel_purchases',
      start_date: startDate || undefined,
      end_date: endDate || undefined
    });
    const url = window.URL.createObjectURL(new Blob([res.data]));
    const link = document.createElement('a');
    link.href = url;
    link.setAttribute('download', `fuel_purchases_${startDate || 'all'}_to_${endDate || 'all'}`);
    document.body.appendChild(link);
    link.click();
    link.remove();
  } catch (err) {
    alert('Failed to generate PDF report');
  }
};

const totalPages = Math.ceil(total / perPage);

return (
  <div>
    <div className="page-header">
      <div className="page-header-left">
        <h1 className="text-h1">Fuel Purchases</h1>
        <p className="text-muted">Record and track all fuel procurement</p>
      </div>
      <div className="page-actions">
```



```

    <button className="btn-secondary" onClick={handleExportPDF} title="Download pr
      <FileText size={16} /> Export PDF
    </button>
    {user?.role === 'admin' && (
      <button id="new-purchase-btn" className="btn-primary" onClick={() => setShow
        <Plus size={16} /> New Purchase
      </button>
    )}
  </div>
</div>

<div style={{ display: 'grid', gridTemplateColumns: '1fr 1fr', gap: 20, marginBott
  <div className="glass-card" style={{ padding: 24, display: 'flex', flexDirection
    <div className="text-label" style={{ marginBottom: 8, color: 'var(--text-mute
    <div style={{ fontFamily: 'Outfit,sans-serif', fontSize: '2rem', fontWeight:
      ₹{currentPrices?.['Avgas']?.price_per_liter?.toFixed(2) || '---'}/L
    </div>
    <div style={{ fontSize: '0.8rem', color: 'var(--text-muted)' }}>
      {currentPrices?.['Avgas']?.last_purchase_date || 'N/A'} · {currentPrices?.[
    </div>
  </div>
  <div className="glass-card" style={{ padding: 24, display: 'flex', flexDirection
    <div className="text-label" style={{ marginBottom: 8, color: 'var(--text-mute
    <div style={{ fontFamily: 'Outfit,sans-serif', fontSize: '2rem', fontWeight:
      ₹{currentPrices?.['Jet A1']?.price_per_liter?.toFixed(2) || '---'}/L
    </div>
    <div style={{ fontSize: '0.8rem', color: 'var(--text-muted)' }}>
      {currentPrices?.['Jet A1']?.last_purchase_date || 'N/A'} · {currentPrices?.
    </div>
  </div>
</div>

<div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'cente
  <div style={{ display: 'flex', gap: 10, flexWrap: 'wrap' }}>
    <select
      id="purchase-filter-fuel"
      className="form-select"
      style={{ width: 140, height: 40, background: 'var(--card-bg)' }}
      value={fuelFilter}
      onChange={e => { setFuelFilter(e.target.value); setPage(1); }}
    >
      <option value="">All Fuel Types</option>
      <option>Jet A1</option>
      <option>Avgas</option>
    </select>
    <input
      type="date"

```

```

        className="form-input"
        style={{ width: 140, height: 40 }}
        value={startDate}
        onChange={e => { setStartDate(e.target.value); setPage(1); }}
        placeholder="Start Date"
      />
      <input
        type="date"
        className="form-input"
        style={{ width: 140, height: 40 }}
        value={endDate}
        onChange={e => { setEndDate(e.target.value); setPage(1); }}
        placeholder="End Date"
      />
      <button
        className="btn-secondary"
        style={{ height: 40 }}
        onClick={() => { setFuelFilter(''); setStartDate(''); setEndDate(''); setP
      >
        Clear
      </button>
    </div>
    <span className="text-muted" style={{ fontSize: '0.85rem' }}>{total} records</
  </div>

  <div className="glass-card" style={{ padding: 0, overflow: 'hidden' }}>
    {loading ? (
      <div className="loading-screen"><div className="spinner" /><span>Loading purch
    ) : (
      <>
        <div className="table-container">
          <table className="data-table">
            <thead>
              <tr>
                <th>Invoice</th>
                <th>Date</th>
                <th>Supplier</th>
                <th>Location</th>
                <th>Type</th>
                <th style={{ textAlign: 'right' }}>Qty (L)</th>
                <th style={{ textAlign: 'right' }}>₹ / L</th>
                <th style={{ textAlign: 'right' }}>Total Cost</th>
              </tr>
            </thead>
            <tbody>
              {list.map(p => (
                <tr key={p.id}>

```

```

        <td>
          <span style={{ fontFamily: 'monospace', fontSize: '0.82rem', col
            {p.invoice_number || `P-${String(p.id).padStart(4, '0')}`}}
          </span>
        </td>
        <td style={{ color: 'var(--text-muted)' }}>{p.purchase_date}</td>
        <td style={{ fontWeight: 600 }}>{p.supplier_name}</td>
        <td>{p.location}</td>
        <td>
          <span className={`badge ${p.fuel_type === 'Jet A1' ? 'badge-blue
            {p.fuel_type}
          </span>
        </td>
        <td style={{ textAlign: 'right', fontWeight: 500 }}>{p.quantity_li
        <td style={{ textAlign: 'right', color: 'var(--primary-accent)', f
          ₹{parseFloat(p.price_per_liter)?.toFixed(2)}
        </td>
        <td style={{ textAlign: 'right', fontWeight: 700 }}>
          ₹{parseFloat(p.total_cost)?.toLocaleString(undefined, { minimumF
        </td>
      </tr>
    )))
    {list.length === 0 && (
      <tr><td colspan="8" style={{ textAlign: 'center', padding: '48px', c
    )}
  </tbody>
</table>
</div>

{totalPages > 1 && (
  <div style={{ display: 'flex', justifyContent: 'space-between', alignItems
    <span className="text-muted" style={{ fontSize: '0.85rem' }}>Page {page}
    <div style={{ display: 'flex', gap: 8 }}>
      <button className="btn-secondary" style={{ padding: '6px 14px', fontSi
      <button className="btn-secondary" style={{ padding: '6px 14px', fontSi
    </div>
  </div>
  )}
</>
)}
</div>

  {showModal && <AddPurchaseModal onClose={() => setShowModal(false)} onSuccess={han
</div>
);
};

```

```
export default FuelPurchasesPage;
```

File: frontend/src/pages/LoginPage.js

```
import React, { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../contexts/AuthContext';
import { Plane, Lock, User, AlertCircle, Eye, EyeOff } from 'lucide-react';

const LoginPage = () => {
  const [username, setUsername] = useState('');
  const [password, setPassword] = useState('');
  const [showPwd, setShowPwd] = useState(false);
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);
  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setLoading(true);
    const result = await login(username, password);
    setLoading(false);
    if (result.success) navigate('/');
    else setError(result.error || 'Invalid credentials. Please try again.');
```

```
  };

  return (
    <div style={{
      minHeight: '100vh',
      display: 'flex',
      alignItems: 'center',
      justifyContent: 'center',
      position: 'relative',
      backgroundImage: 'url("/a380_sky.png")',
      backgroundSize: 'cover',
      backgroundPosition: 'center',
      backgroundRepeat: 'no-repeat',
      fontFamily: 'Plus Jakarta Sans, sans-serif'
    }}>
      { /* Light Glassmorphism Overlay */ }
```

```

<div style={{
  position: 'absolute',
  inset: 0,
  background: 'linear-gradient(rgba(255, 255, 255, 0.1), rgba(226, 232, 240, 0.3))',
  zIndex: 1
}} />

<div style={{
  position: 'relative',
  zIndex: 2,
  width: '100%',
  maxWidth: '460px',
  padding: '24px'
}}>
  { /* Centered Login Card */ }
  <div style={{
    background: 'rgba(255, 255, 255, 0.9)',
    backdropFilter: 'blur(16px)',
    borderRadius: 24,
    padding: '48px 40px',
    boxShadow: '0 25px 50px -12px rgba(0, 0, 0, 0.5)',
    border: '1px solid rgba(255, 255, 255, 0.2)'
  }}>
    { /* Header */ }
    <div style={{ textAlign: 'center', marginBottom: 32 }}>
      <h1 style={{ fontFamily: 'Outfit, sans-serif', fontSize: '2.5rem', fontWeight: 'normal' }}>
        AeroFuel
      </h1>
      <p style={{ color: 'var(--text-muted)', fontSize: '0.95rem', fontWeight: 500 }}>
        Fueling the future of aviation.
      </p>
    </div>

    {error && (
      <div className="alert alert-error" style={{ marginBottom: 20 }}>
        <AlertCircle size={16} />
        <span style={{ fontSize: '0.875rem' }}>{error}</span>
      </div>
    )}

    <form onSubmit={handleSubmit}>
      <div className="form-group">
        <label className="form-label" style={{ fontWeight: 700 }}>Username</label>
        <div style={{ position: 'relative' }}>
          <User size={18} style={{ position: 'absolute', left: 14, top: '50%', transform: 'translateY(-50%)' }}>
            <input
              id="login-username"

```

```

        className="form-input"
        style={{ paddingLeft: 42, height: 48, background: '#f8fafc' }}
        placeholder="admin"
        value={username}
        onChange={e => setUsername(e.target.value)}
        required
      />
    </div>
  </div>

  <div className="form-group">
    <label className="form-label" style={{ fontWeight: 700 }}>Password</label>
    <div style={{ position: 'relative' }}>
      <Lock size={18} style={{ position: 'absolute', left: 14, top: '50%', tra
      <input
        id="login-password"
        type={showPwd ? 'text' : 'password'}
        className="form-input"
        style={{ paddingLeft: 42, paddingRight: 44, height: 48, background: '#
        placeholder="....."
        value={password}
        onChange={e => setPassword(e.target.value)}
        required
      />
      <button type="button" onClick={() => setShowPwd(!showPwd)} style={{
        position: 'absolute', right: 14, top: '50%', transform: 'translateY(-5
        background: 'none', border: 'none', cursor: 'pointer', color: 'var(--t
      }}>
        {showPwd ? <EyeOff size={18} /> : <Eye size={18} />}
      </button>
    </div>
  </div>

  <button
    id="login-submit"
    type="submit"
    className="btn-primary"
    style={{ width: '100%', height: 50, justifyContent: 'center', fontSize: '1
    disabled={loading}
  >
    {loading ? <div className="spinner" style={{ width: 20, height: 20 }} /> :
  </button>
</form>

  { /* Demo Credits */ }
  <div style={{ marginTop: 24, fontSize: '0.8rem', textAlign: 'center', color: '
    admin / admin123 | operator / operator123

```

```

    </div>
  </div>

  { /* Branding & Features Below Panel */ }
  <div style={{ marginTop: 32, textAlign: 'center', color: 'var(--text-main)' }}>
    <h2 style={{ fontSize: '1.2rem', fontWeight: 700, marginBottom: 4 }}>AeroFuel
    <p style={{ fontSize: '0.85rem', opacity: 0.8, marginBottom: 24, color: 'var(

    <div className="marquee-container">
      <div className="marquee-content">
        {[
          { icon: '📊', text: 'Real-time Dashboards' },
          { icon: '📉', text: 'Price Predictions' },
          { icon: '📄', text: 'One-click Reports' },
          { icon: '📊', text: 'Real-time Dashboards' },
          { icon: '📉', text: 'Price Predictions' },
          { icon: '📄', text: 'One-click Reports' },
          { icon: '📊', text: 'Real-time Dashboards' },
          { icon: '📉', text: 'Price Predictions' },
          { icon: '📄', text: 'One-click Reports' },
          { icon: '📊', text: 'Real-time Dashboards' },
          { icon: '📉', text: 'Price Predictions' },
          { icon: '📄', text: 'One-click Reports' }
        ]}.map((f, i) => (
          <div key={i} style={{
            padding: '12px 24px',
            background: 'rgba(255, 255, 255, 0.1)',
            backdropFilter: 'blur(8px)',
            borderRadius: 12,
            border: '1px solid rgba(255, 255, 255, 0.15)',
            whiteSpace: 'nowrap',
            display: 'flex',
            alignItems: 'center',
            gap: '10px'
          }}>
            <span style={{ fontSize: '18px' }}>{f.icon}</span>
            <span style={{ fontSize: '0.75rem', fontWeight: 700, textTransform: '
          </div>
        )))
      </div>
    </div>
  </div>
</div>
</div>
</div>
);
};

```

```
export default LoginPage;
```

File: frontend/src/pages/PriceTrendsPage.js

```
import React, { useState, useEffect } from 'react';
import { purchases } from '../api/services';
import {
  LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip,
  ResponsiveContainer, Legend
} from 'recharts';
import { TrendingUp, Activity } from 'lucide-react';

const RANGES = [
  { label: '14d', days: 14 },
  { label: '30d', days: 30 },
  { label: '60d', days: 60 },
  { label: '90d', days: 90 },
];

const PriceTrendsPage = () => {
  const [data, setData] = useState([]);
  const [activeFuel, setActiveFuel] = useState('Jet A1');
  const [range, setRange] = useState(30);
  const [kpis, setKpis] = useState(null);
  const [predictions, setPredictions] = useState(null);
  const [loading, setLoading] = useState(true);
  const [predicting, setPredicting] = useState(false);

  const fetchData = async () => {
    setLoading(true);
    setPredictions(null);
    try {
      const [trendRes, avgRes, curRes] = await Promise.all([
        purchases.priceTrend({ fuel_type: activeFuel, days: range }),
        purchases.averagePrice(),
        purchases.currentPrice()
      ]);

      const currentPrice = curRes.data[activeFuel]?.price_per_liter || 0;
      const fuelStats = avgRes.data.find(r => r.fuel_type === activeFuel) || {};

      setKpis({
```



```
        current: currentPrice,
        avg: fuelStats.avg_price_per_liter || 0,
        totalPurchased: fuelStats.total_quantity_liters || 0,
        count: fuelStats.purchase_count || 0
    });

    setData(trendRes.data.map(d => ({
        ...d,
        label: new Date(d.date).toLocaleDateString('en-US', { month: 'short', day: 'nume
    })));
} catch (err) {
    console.error('Trend fetch error', err);
} finally {
    setLoading(false);
}
};

useEffect(() => {
    fetchData();
}, [activeFuel, range]);

const handlePredict = async () => {
    setPredicting(true);
    try {
        const res = await purchases.predictPrice({ fuel_type: activeFuel, days_ahead: 7 })
        setPredictions(res.data);
    } catch (err) {
        alert("Failed to predict: " + (err.response?.data?.error || err.message));
    } finally {
        setPredicting(false);
    }
};

const getChartData = () => {
    const arr = [...data];
    if (predictions?.predictions && arr.length > 0) {
        // Connect predicting line to the last real data point by copying the object
        arr[arr.length - 1] = {
            ...arr[arr.length - 1],
            predicted_price: arr[arr.length - 1].price_per_liter
        };
        predictions.predictions.forEach(p => {
            arr.push({
                label: new Date(p.date).toLocaleDateString('en-US', { month: 'short', day: 'nu
                predicted_price: p.predicted_price
            });
        });
    }
};
```

```

    }
    return arr;
  };

const chartData = getChartData();
const formatLiters = (val) => val >= 1000 ? (val / 1000).toFixed(1) + 'K L' : val.toLo

return (
  <div>
    <div className="page-header" style={{ marginBottom: 20 }}>
      <div className="page-header-left">
        <h1 className="text-h1" style={{ display: 'flex', alignItems: 'center', gap: 1
          <div style={{ width: 32, height: 32, background: 'var(--tertiary-accent)', b
            <TrendingUp size={18} style={{ color: 'white' }} />
          </div>
          Price Trends
        </h1>
        <p className="text-muted">Historical fuel price analysis and ML-powered predic
      </div>
    </div>

    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'cente
      <div style={{ display: 'flex', gap: 24 }}>
        { /* Fuel Toggle */ }
        <div style={{ display: 'flex', background: 'white', padding: 4, borderRadius:
          [['Jet A1', 'Avgas']].map(f => (
            <button
              key={f}
              onClick={() => setActiveFuel(f)}
              style={{
                padding: '6px 16px', borderRadius: 7, border: 'none', cursor: 'pointer'
                background: activeFuel === f ? 'var(--primary-accent)' : 'transparent'
                color: activeFuel === f ? 'white' : 'var(--text-muted)',
                transition: 'all 0.2s ease'
              }}
            >{f}</button>
          )))
        </div>

        { /* Range Toggle */ }
        <div style={{ display: 'flex', background: 'white', padding: 4, borderRadius:
          RANGES.map(r => (
            <button
              key={r.days}
              onClick={() => setRange(r.days)}
              style={{
                padding: '6px 14px', borderRadius: 7, border: 'none', cursor: 'pointer'

```

```

        background: range === r.days ? 'var(--primary-accent)' : 'transparent'
        color: range === r.days ? 'white' : 'var(--text-muted)',
        transition: 'all 0.2s ease'
      }}
    >{r.label}</button>
  )))
</div>
</div>

<button
  onClick={handlePredict}
  disabled={predicting || predictions}
  style={{
    background: 'linear-gradient(135deg, #0ea5e9, #2563eb)',
    color: 'white', border: 'none', padding: '8px 18px', borderRadius: 'var(--ra
    fontWeight: 700, cursor: (predicting || predictions) ? 'not-allowed' : 'poin
    boxShadow: '0 4px 10px rgba(37, 99, 235, 0.2)', display: 'flex', alignItems:
    opacity: predictions ? 0.6 : 1
  }}
>
  {predicting ? <div className="spinner" style={{ width: 14, height: 14, borderW
  Predict 7 days
</button>
</div>

{kpis && (
  <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fit, minmax(200
    <div className="glass-card card" style={{ padding: '20px 24px' }}>
      <div className="text-label" style={{ marginBottom: 8, color: 'var(--text-mut
        <div style={{ fontSize: '2rem', fontWeight: 800, color: 'var(--primary-accen
      </div>
    <div className="glass-card card" style={{ padding: '20px 24px' }}>
      <div className="text-label" style={{ marginBottom: 8, color: 'var(--text-mut
        <div style={{ fontSize: '2rem', fontWeight: 800, color: 'var(--secondary-acc
      </div>
    <div className="glass-card card" style={{ padding: '20px 24px' }}>
      <div className="text-label" style={{ marginBottom: 8, color: 'var(--text-mut
        <div style={{ fontSize: '2rem', fontWeight: 800, color: 'var(--secondary-acc
      </div>
    <div className="glass-card card" style={{ padding: '20px 24px' }}>
      <div className="text-label" style={{ marginBottom: 8, color: 'var(--text-mut
        <div style={{ fontSize: '2rem', fontWeight: 800, color: 'var(--warning)' }}>
      </div>
    </div>
  </div>
))

<div className="glass-card card" style={{ padding: 24 }}>

```

```

<div style={{ display: 'flex', alignItems: 'center', gap: 8, marginBottom: 24 }}
  <Activity size={18} style={{ color: 'var(--text-muted)' }} />
  <h3 className="text-h3">{activeFuel} Price Trend – Last {range} Days</h3>
</div>

{loading ? (
  <div className="loading-screen"><div className="spinner" /><span>Loading chart
) : (
  <div style={{ height: 420 }}>
    <ResponsiveContainer width="100%" height="100%">
      <LineChart data={chartData} margin={{ top: 10, right: 10, left: 0, bottom:
        <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#e2e8f0" /
        <XAxis dataKey="label" stroke="#94a3b8" fontSize={11} tickLine={false} a
        <YAxis stroke="#94a3b8" fontSize={11} tickLine={false} axisLine={false}
        <Tooltip
          contentStyle={{ borderRadius: 10, border: '1px solid #e2e8f0', boxShad
          formatter={(v, name) => [`₹${v?.toFixed(2)} || v}`, name === 'price_per
        />
        <Legend iconType="circle" iconSize={8} formatter={(val) => <span style={
        <Line type="monotone" dataKey="price_per_liter" name="price_per_liter" s
        {predictions && (
          <Line type="monotone" dataKey="predicted_price" name="predicted_price"
        )}
      </LineChart>
    </ResponsiveContainer>
  </div>
)}
</div>
</div>
);
};

export default PriceTrendsPage;

```

File: frontend/src/pages/ReportsPage.js

```

import React, { useState, useEffect, useCallback } from 'react';
import { reports, aircraft as aircraftApi } from '../api/services';
import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip,
  ResponsiveContainer, Cell
} from 'recharts';
import { FileText, Download, Loader, BarChart3, Plane, CalendarRange } from 'lucide-react';

```

```
const REPORT_TYPES = [
  { id: 'trip_costs',          label: 'Trip Costs',          icon: <FileText size={18}/>,
  { id: 'monthly_usage',      label: 'Monthly Usage',      icon: <CalendarRange size=
  { id: 'aircraft_consumption',label: 'Aircraft Comparison', icon: <Plane size={18}/>,
];

const COLORS = ['#1d4ed8', '#059669', '#7c3aed', '#d97706', '#dc2626'];

const ReportsPage = () => {
  const [activeType, setActiveType] = useState('trip_costs');
  const [startDate, setStartDate]   = useState(() => { const d = new Date(); d.setDate(1
  const [endDate, setEndDate]       = useState(() => new Date().toISOString().slice(0,10
  const [data, setData]             = useState(null);
  const [loading, setLoading]       = useState(false);
  const [exporting, setExporting]   = useState(false);
  const [fleet, setFleet]           = useState([]);
  const [aircraftFilter, setAircraftFilter] = useState('');

  useEffect(() => { aircraftApi.list().then(r => setFleet(r.data)).catch(()=>{}); }, [])

  const fetchReport = useCallback(async () => {
    setLoading(true);
    try {
      let res;
      if (activeType === 'trip_costs') {
        const params = { start_date: startDate, end_date: endDate };
        if (aircraftFilter) params.aircraft_id = aircraftFilter;
        res = await reports.getTripCosts(params);
      } else if (activeType === 'monthly_usage') {
        res = await reports.getMonthlyUsage();
      } else {
        res = await reports.getAircraftConsumption({ start_date: startDate, end_date: en
      }
      setData(res.data);
    } catch (err) {
      console.error('Report fetch error', err);
    } finally {
      setLoading(false);
    }
  }, [activeType, startDate, endDate, aircraftFilter]);

  useEffect(() => { fetchReport(); }, [fetchReport]);

  const handleExport = async () => {
    setExporting(true);
    try {
```

```

const res = await reports.exportPdf({ report_type: activeType, start_date: startDa
const url = URL.createObjectURL(new Blob([res.data], { type: 'application/pdf' }))
const a = document.createElement('a');
a.href = url;
a.download = `aerofuel_${activeType}_${startDate}.pdf`;
a.click();
URL.revokeObjectURL(url);
} catch (err) {
  alert('PDF export failed. Please try again.');
```

```

} finally {
  setExporting(false);
}
};

/* — Render helpers ————— */
const TripCostsView = () => {
  if (!data) return null;
  const { trips = [], summary = {} } = data;
  return (
    <>
    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(3,1fr)', gap: 16, ma
      {[
        { label: 'Total Trips',    value: summary.total_trips || 0
        { label: 'Total Fuel',     value: `${(summary.total_fuel_used || 0).toLocale
        { label: 'Total Cost',    value: `₹${(summary.total_fuel_cost || 0).toLocal
      ].map(s => (
        <div key={s.label} className="glass-card card" style={{ padding: '18px 20px'
          <div className="text-label" style={{ marginBottom: 6 }}>{s.label}</div>
          <div style={{ fontFamily:'Outfit,sans-serif', fontWeight:700, fontSize:'1.
        </div>
      ))}
    </div>
    <div className="table-container">
      <table className="data-table">
        <thead>
          <tr>
            <th>Flight #</th><th>Date</th><th>Route</th><th>Aircraft</th>
            <th style={{textAlign:'right'}}>Dist (km)</th>
            <th style={{textAlign:'right'}}>Fuel Used</th>
            <th style={{textAlign:'right'}}>Efficiency</th>
            <th style={{textAlign:'right'}}>Trip Cost (₹)</th>
          </tr>
        </thead>
        <tbody>
          {trips.map(t => (
            <tr key={t.flight_number}>
              <td style={{fontWeight:700, fontFamily:'Outfit,sans-serif'}}>{t.flight

```

```

        <td style={{color:'var(--text-muted)', fontSize:'0.85rem'}}>{t.flight_
        <td style={{fontSize:'0.88rem'}}>{t.route}</td>
        <td style={{color:'var(--primary-accent)', fontWeight:600}}>{t.aircraf
        <td style={{textAlign:'right'}}>{t.distance_km?.toLocaleString()}</td>
        <td style={{textAlign:'right'}}>{t.actual_fuel?.toLocaleString()} L</t
        <td style={{textAlign:'right'}}>
            <span className={`badge ${t.efficiency < 1.0 ? 'badge-green' : (t.ef
            {(t.efficiency * 100).toFixed(1)}}%
            </span>
        </td>
        <td style={{textAlign:'right', fontWeight:700}}>₹{t.trip_fuel_cost?.to
        </tr>
    ))}
    {trips.length === 0 && <tr><td colSpan="8" style={{textAlign:'center',padd
    </tbody>
    </table>
    </div>
    </>
    );
    };

const MonthlyUsageView = () => {
    if (!data) return null;
    const months = data.data || [];
    return (
        <>
        <div style={{ height: 280, marginBottom: 24 }}>
            <ResponsiveContainer width="100%" height="100%">
                <BarChart data={months} margin={{ top: 5, right: 10, left: 0, bottom: 0 }} b
                <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#e2e8f0" />
                <XAxis dataKey="month_name" stroke="#94a3b8" fontSize={11} tickLine={false}
                <YAxis stroke="#94a3b8" fontSize={11} tickLine={false} axisLine={false} ti
                <Tooltip contentStyle={{ borderRadius:10, border:'1px solid #e2e8f0', boxS
                <Bar dataKey="total_fuel_used" name="Fuel Used" radius={[6,6,0,0]}>
                    {months.map((_,i) => <Cell key={i} fill={COLORS[i % COLORS.length]} />)}
                </Bar>
            </BarChart>
        </ResponsiveContainer>
        </div>
        <div className="table-container">
            <table className="data-table">
                <thead>
                    <tr>
                        <th>Month</th>
                        <th style={{textAlign:'right'}}>Flights</th>
                        <th style={{textAlign:'right'}}>Jet A1 (L)</th>
                        <th style={{textAlign:'right'}}>Avgas (L)</th>

```

```

        <th style={{textAlign:'right'}}>Total Fuel (L)</th>
        <th style={{textAlign:'right'}}>Total Cost (₹)</th>
      </tr>
    </thead>
    <tbody>
      {months.map(m => (
        <tr key={m.month_name}>
          <td style={{fontWeight:600}}>{m.month_name}</td>
          <td style={{textAlign:'right'}}>{m.total_flights}</td>
          <td style={{textAlign:'right'}}>{m.jet_a1_used?.toLocaleString()}</td>
          <td style={{textAlign:'right'}}>{m.avgas_used?.toLocaleString()}</td>
          <td style={{textAlign:'right', fontWeight:600}}>{m.total_fuel_used?.to
          <td style={{textAlign:'right', fontWeight:700, color:'var(--primary-ac
        </tr>
      )})}
    </tbody>
  </table>
</div>
</>
);
};

const AircraftView = () => {
  if (!data) return null;
  const items = data.data || [];
  return (
    <>
      <div style={{ height: 260, marginBottom: 24 }}>
        <ResponsiveContainer width="100%" height="100%">
          <BarChart data={items} margin={{ top: 5, right: 10, left: 0, bottom: 0 }} ba
            <CartesianGrid strokeDasharray="3 3" vertical={false} stroke="#e2e8f0" />
            <XAxis dataKey="aircraft_id" stroke="#94a3b8" fontSize={11} tickLine={fals
            <YAxis stroke="#94a3b8" fontSize={11} tickLine={false} axisLine={false} ti
            <Tooltip contentStyle={{ borderRadius:10, border:'1px solid #e2e8f0', boxS
            <Bar dataKey="total_fuel_used_liters" name="Fuel Used" radius=[[6,6,0,0]]>
              {items.map((_,i) => <Cell key={i} fill={COLORS[i % COLORS.length]} />)}
            </Bar>
          </BarChart>
        </ResponsiveContainer>
      </div>
      <div className="table-container">
        <table className="data-table">
          <thead>
            <tr>
              <th>Aircraft</th><th>Model</th>
              <th style={{textAlign:'right'}}>Flights</th>
              <th style={{textAlign:'right'}}>Fuel Used (L)</th>

```



```

        <th style={{textAlign:'right'}}>Distance (km)</th>
        <th style={{textAlign:'right'}}>Avg Efficiency</th>
        <th style={{textAlign:'right'}}>Total Cost (₹)</th>
      </tr>
    </thead>
    <tbody>
      {items.map(a => (
        <tr key={a.aircraft_id}>
          <td style={{fontWeight:700, color:'var(--primary-accent)', fontFamily:
            <td>{a.model}</td>
          <td style={{textAlign:'right'}}>{a.total_flights}</td>
          <td style={{textAlign:'right', fontWeight:600}}>{a.total_fuel_used_lit
            <td style={{textAlign:'right'}}>{a.total_distance_km?.toLocaleString()
            <td style={{textAlign:'right'}}>
              <span className={`badge ${a.avg_efficiency < 1.0 ? 'badge-green' : '
                {(a.avg_efficiency||1)*100).toFixed(1)}%
              </span>
            </td>
          <td style={{textAlign:'right', fontWeight:700}}>₹{a.total_fuel_cost?.t
        </tr>
      )}}
      {items.length === 0 && <tr><td colspan="7" style={{textAlign:'center',padd
    </tbody>
  </table>
</div>
</>
);
};

return (
  <div>
    <div className="page-header">
      <div className="page-header-left">
        <h1 className="text-h1">Reports & Analytics</h1>
        <p className="text-muted">Comprehensive fuel cost and usage reports with PDF e
      </div>
      <button id="export-pdf-btn" className="btn-primary" onClick={handleExport} disab
        {exporting ? <><Loader size={16} style={{ animation:'spin 0.7s linear infinite
      </button>
    </div>

    { /* Report type tabs */}
    <div style={{ display: 'flex', gap: 12, marginBottom: 20 }}>
      {REPORT_TYPES.map(rt => (
        <button
          key={rt.id}
          id={`report-tab-${rt.id}`}

```

```

onClick={() => setActiveType(rt.id)}
style={{
  flex: 1, padding: '14px 16px', borderRadius: 'var(--radius-md)',
  border: `2px solid ${activeType === rt.id ? 'var(--primary-accent)' : 'var(
  background: activeType === rt.id ? 'var(--primary-light)' : 'white',
  cursor: 'pointer', textAlign: 'left', transition: 'all 0.2s ease',
  fontFamily: 'Plus Jakarta Sans, sans-serif'
}}
>
<div style={{ display: 'flex', alignItems: 'center', gap: 8, color: activeTy
  {rt.icon}
  <span style={{ fontWeight: 700, fontSize: '0.9rem' }}>{rt.label}</span>
</div>
  <div style={{ fontSize: '0.78rem', color: 'var(--text-muted)' }}>{rt.desc}</
</button>
))}
</div>

{/* Date filters */}
<div className="glass-card" style={{ padding: '16px 20px', marginBottom: 20 }}>
  <div style={{ display: 'flex', gap: 16, flexWrap: 'wrap', alignItems: 'flex-end'
    {activeType !== 'monthly_usage' && (
      <>
        <div className="filter-group">
          <label className="form-label" style={{ fontSize: '0.78rem' }}>Start Date
          <input type="date" id="report-start" className="form-input" style={{ hei
        </div>
        <div className="filter-group">
          <label className="form-label" style={{ fontSize: '0.78rem' }}>End Date</
          <input type="date" id="report-end" className="form-input" style={{ heigh
        </div>
      </>
    )}
    {activeType === 'trip_costs' && (
      <div className="filter-group">
        <label className="form-label" style={{ fontSize: '0.78rem' }}>Aircraft</la
        <select id="report-aircraft" className="form-select" style={{ height: 38,
          <option value="">All Aircraft</option>
          {fleet.map(a => <option key={a.id} value={a.id}>{a.aircraft_id}</option>
        </select>
      </div>
    )}
  </div>
</div>

{/* Report content */}
<div className="glass-card" style={{ padding: 24 }}>

```

```

    {loading ? (
      <div className="loading-screen"><div className="spinner" /><span>Generating re
    ) : (
      <>
        {activeType === 'trip_costs'      && <TripCostsView />}
        {activeType === 'monthly_usage'   && <MonthlyUsageView />}
        {activeType === 'aircraft_consumption' && <AircraftView />}
      </>
    )}
  </div>
</div>
);
};

export default ReportsPage;

```

File: frontend/src/pages/TransactionsPage.js

```

import React, { useState, useEffect, useCallback } from 'react';
import { inventory } from '../api/services';
import { ArrowLeftRight, Filter } from 'lucide-react';

const TX_BADGE = {
  allocation: { label: 'Allocation', cls: 'badge-amber' },
  consumption: { label: 'Consumed', cls: 'badge-red' },
  refund: { label: 'Refund', cls: 'badge-green' },
  purchase: { label: 'Purchase', cls: 'badge-blue' },
};

const TransactionsPage = () => {
  const [txs, setTxs] = useState([]);
  const [total, setTotal] = useState(0);
  const [loading, setLoading] = useState(true);
  const [typeFilter, setTypeFilter] = useState('');
  const [fuelFilter, setFuelFilter] = useState('');
  const [page, setPage] = useState(1);
  const perPage = 20;

  const fetchTxs = useCallback(async () => {
    setLoading(true);
    try {
      const params = { page, per_page: perPage };
      if (typeFilter) params.transaction_type = typeFilter;
    }
  }, [typeFilter, perPage]);

```

```

    if (fuelFilter) params.fuel_type = fuelFilter;
    const res = await inventory.getTransactions(params);
    const data = res.data;
    setTxs(data.transactions || data);
    setTotal(data.total || (data.transactions || data).length);
  } catch (err) {
    console.error('Failed to load transactions', err);
  } finally {
    setLoading(false);
  }
}, [page, typeFilter, fuelFilter]);

useEffect(() => { fetchTxs(); }, [fetchTxs]);

const totalPages = Math.ceil(total / perPage);

const summary = {
  totalIn: txs.filter(t => ['refund', 'purchase'].includes(t.transaction_type)).reduce(
    totalOut: txs.filter(t => ['allocation', 'consumption'].includes(t.transaction_type))
  };

return (
  <div>
    <div className="page-header">
      <div className="page-header-left">
        <h1 className="text-h1" style={{ display: 'flex', alignItems: 'center', gap: 1
          <div style={{ width: 32, height: 32, background: 'var(--primary-accent)', bo
            <ArrowLeftRight size={18} style={{ color: 'white' }} />
          </div>
          Fuel Transactions
        </h1>
        <p className="text-muted">Complete audit trail of all fuel movements</p>
      </div>
    </div>

    { /* Summary strip */ }
    <div style={{ display: 'grid', gridTemplateColumns: 'repeat(3, 1fr)', gap: 20, mar
      [
        { label: 'RECORDS SHOWN', val: total, color: 'var(--primar
        { label: 'TOTAL LITERS', val: `${summary.totalIn.toLocaleString()} L`, color
        { label: 'TOTAL VALUE', val: `₹${(summary.totalOut * 95).toLocaleString()}`
      ].map(s => (
        <div key={s.label} className="glass-card" style={{ padding: '20px 24px', displ
          <span className="text-label" style={{ fontSize: '0.75rem', color: 'var(--tex
          <span style={{ fontFamily: 'Outfit,sans-serif', fontWeight: 800, fontSize: '
        </div>
      ))}

```

```

</div>

<div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center' }}>
  <div style={{ display: 'flex', gap: 10, flexWrap: 'wrap' }}>
    <select id="tx-filter-type" className="form-select" style={{ width: 140, height: 40 }}>
      <option value="">All Types</option>
      <option value="allocation">Allocation</option>
      <option value="consumption">Consumption</option>
      <option value="refund">Refund</option>
      <option value="purchase">Purchase</option>
    </select>
    <select id="tx-filter-fuel" className="form-select" style={{ width: 140, height: 40 }}>
      <option value="">All Fuel Types</option>
      <option>Jet A1</option>
      <option>Avgas</option>
    </select>
    <input type="date" className="form-input" style={{ width: 140, height: 40 }}>
    <input type="date" className="form-input" style={{ width: 140, height: 40 }}>
    <button className="btn-secondary" style={{ height: 40 }}>Clear</button>
  </div>
  <span className="text-muted" style={{ fontSize: '0.85rem' }}>{total} total</span>
</div>

<div className="glass-card" style={{ padding: 0, overflow: 'hidden' }}>
  {loading ? (
    <div className="loading-screen"><div className="spinner" /><span>Loading...</span></div>
  ) : txs.length === 0 ? (
    <div className="empty-state">
      <div style={{ margin: '0 auto', width: 64, height: 64, background: 'var(--primary)', border: '1px solid #ccc', display: 'flex', align-items: center, justify-content: center; }}>
        <ArrowLeftRight size={28} style={{ color: 'white' }} />
      </div>
      <div style={{ font-size: '1.1rem', font-weight: 500, color: 'var(--text-sub)' }}>No transactions found</div>
    </div>
  ) : (
    <>
      <div className="table-container">
        <table className="data-table">
          <thead>
            <tr>
              <th>ID</th>
              <th>Timestamp</th>
              <th>Type</th>
              <th>Fuel</th>
              <th>Flight Ref</th>
              <th style={{ textAlign: 'right' }}>Qty (L)</th>
              <th style={{ textAlign: 'right' }}>₹ / L</th>
              <th style={{ textAlign: 'right' }}>Total (₹)</th>
            </tr>
          </thead>
          <tbody>
            {txs.map(tx => (
              <tr>
                <td>{tx.id}</td>
                <td>{tx.timestamp}</td>
                <td>{tx.type}</td>
                <td>{tx.fuel}</td>
                <td>{tx.flightRef}</td>
                <td style={{ text-align: 'right' }}>{tx.qty}</td>
                <td style={{ text-align: 'right' }}>{tx.rate}</td>
                <td style={{ text-align: 'right' }}>{tx.total}</td>
              </tr>
            ))}
          </tbody>
        </table>
      </div>
    </>
  )}

```

```

        <th>Notes</th>
      </tr>
    </thead>
    <tbody>
      {txs.map(tx => {
        const badge = TX_BADGE[tx.transaction_type] || { label: tx.transacti
        const isIn = ['refund', 'purchase'].includes(tx.transaction_type);
        return (
          <tr key={tx.id}>
            <td style={{ fontFamily: 'monospace', fontSize: '0.8rem', color:
            <td style={{ fontSize: '0.82rem', color: 'var(--text-muted)' }}>
              {new Date(tx.transaction_date).toLocaleString('en-IN', { day:
            </td>
            <td><span className={`badge ${badge.cls}`}>{badge.label}</span><
            <td style={{ fontWeight: 600 }}>{tx.fuel_type}</td>
            <td style={{ fontFamily: 'monospace', fontSize: '0.82rem', color:
            <td style={{ textAlign: 'right', fontWeight: 700, color: isIn ?
              {isIn ? '+' : '-'}>{tx.quantity_liters?.toLocaleString()}
            </td>
            <td style={{ textAlign: 'right', color: 'var(--text-muted)' }}>₹
            <td style={{ textAlign: 'right', fontWeight: 600 }}>₹{parseFloat
            <td style={{ maxWidth: 200, overflow: 'hidden', textOverflow: 'e
          </tr>
        );
      })}
      {txs.length === 0 && (
        <tr><td colspan="9" style={{ textAlign: 'center', padding: '48px', c
        )}
      </tbody>
    </table>
  </div>

  {totalPages > 1 && (
    <div style={{ display: 'flex', justifyContent: 'space-between', alignItems
      <span className="text-muted" style={{ fontSize: '0.85rem' }}>Page {page}
      <div style={{ display: 'flex', gap: 8 }}>
        <button className="btn-secondary" style={{ padding: '6px 14px', fontSi
        <button className="btn-secondary" style={{ padding: '6px 14px', fontSi
      </div>
    </div>
  )}
</>
)}
</div>
</div>
);
};

```

```
export default TransactionsPage;
```

File: frontend/src/pages/UsersPage.js

```
import React, { useState, useEffect } from 'react';
import { auth } from '../api/services';
import { useAuth } from '../contexts/AuthContext';
import { Users, Plus, X, AlertCircle, CheckCircle, ShieldCheck, UserCheck } from 'lucide-react';

const AddUserModal = ({ onClose, onSuccess }) => {
  const [form, setForm] = useState({ username: '', password: '', role: 'operator' });
  const [error, setError] = useState('');
  const [saving, setSaving] = useState(false);

  const set = (k, v) => setForm(f => ({ ...f, [k]: v }));

  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    setSaving(true);
    try {
      await auth.createUser(form);
      onSuccess();
    } catch (err) {
      setError(err.response?.data?.error || err.response?.data?.message || 'Failed to create user');
    } finally {
      setSaving(false);
    }
  };

  return (
    <div className="modal-overlay" onClick={e => e.target === e.currentTarget && onClose}>
      <div className="modal-box" style={{ maxWidth: 440 }}>
        <div className="modal-header">
          <div>
            <h2 className="text-h3">Create User</h2>
            <p className="text-muted" style={{ marginTop: 4, fontSize: '0.85rem' }}>Add User</p>
          </div>
          <button className="btn-icon" onClick={onClose} id="close-user-modal"><X size={24}></button>
        </div>
        <div className="modal-body">
          {error && (
            <div>
              <div><AlertCircle size={24}></div>
              <p>{error}</p>
            </div>
          )}
          <div>
            <div>
              <div><input type="text" value={form.username} /></div>
              <div><input type="password" value={form.password} /></div>
            </div>
            <div>
              <div><input type="text" value={form.role} /></div>
              <div><input type="text" value={form.role} /></div>
            </div>
            <div>
              <div><input type="button" value="Add User" /></div>
              <div><input type="button" value="Cancel" /></div>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
};
```

```

    <div className="alert alert-error" style={{ marginBottom: 16 }}>
      <AlertCircle size={15} /><span style={{ fontSize: '0.85rem' }}>{error}</span>
    </div>
  )}
  <form onSubmit={handleSubmit}>
    <div className="form-group">
      <label className="form-label">Username *</label>
      <input id="new-username" className="form-input" placeholder="e.g. john.doe" />
    </div>
    <div className="form-group">
      <label className="form-label">Password *</label>
      <input id="new-password" type="password" className="form-input" placeholder="e.g. 123456" />
    </div>
    <div className="form-group">
      <label className="form-label">Role *</label>
      <select id="new-role" className="form-select" value={form.role} onChange={handleChange}>
        <option value="operator">Operator</option>
        <option value="admin">Admin</option>
      </select>
      <div style={{ marginTop: 6, fontSize: '0.78rem', color: 'var(--text-muted)' }}>
        {form.role === 'admin' ? '⚠ Admins can manage users, aircraft, and purchases' : ''}
      </div>
    </div>
    <div className="modal-footer">
      <button type="button" className="btn-secondary" onClick={onClose}>Cancel</button>
      <button id="save-user" type="submit" className="btn-primary" disabled={saving}>
        {saving ? 'Creating...' : '<><CheckCircle size={16} /> Create User</>}
      </button>
    </div>
  </form>
</div>
</div>
);
};

const UsersPage = () => {
  const { user } = useAuth();
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(true);
  const [showModal, setShowModal] = useState(false);
  const [error, setError] = useState('');

  const fetchUsers = async () => {
    setLoading(true);
    setError('');
    try {

```



```

    const res = await auth.listUsers();
    setUsers(Array.isArray(res.data) ? res.data : (res.data.users || []));
  } catch (err) {
    setError('Failed to load users. This endpoint may require admin privileges.');
```

} finally {

```
    setLoading(false);
  }
};

useEffect(() => { fetchUsers(); }, []);

const handleCreated = () => { setShowModal(false); fetchUsers(); };

return (
  <div>
    <div className="page-header" style={{ marginBottom: 24 }}>
      <div className="page-header-left">
        <h1 className="text-h1" style={{ display: 'flex', alignItems: 'center', gap: 1
          <Users size={24} style={{ color: 'var(--tertiary-accent)' }} />
          User Management
        </h1>
        <p className="text-muted">Manage system users and roles</p>
      </div>
      <button id="add-user-btn" className="btn-primary" onClick={() => setShowModal(tr
        <Plus size={16} /> Create User
      </button>
    </div>

    {error && (
      <div className="alert alert-error" style={{ marginBottom: 20 }}>
        <AlertCircle size={16} />
        <span>{error}</span>
      </div>
    )}

    <div className="glass-card" style={{ padding: 0, overflow: 'hidden' }}>
      {loading ? (
        <div className="loading-screen"><div className="spinner" /><span>Loading users
      ) : (
        <table className="data-table">
          <thead>
            <tr>
              <th>USER</th>
              <th>EMAIL</th>
              <th>ROLE</th>
              <th>STATUS</th>
              <th>CREATED</th>

```

```

    </tr>
  </thead>
  <tbody>
    {users.map(u => {
      const isYou = u.username === user?.username; // Assuming `user` from Aut
      return (
        <tr key={u.id}>
          <td>
            <div style={{ display: 'flex', alignItems: 'center', gap: 14 }}>
              <div style={{
                width: 36, height: 36, borderRadius: '50%',
                background: u.role === 'admin' ? '#0ea5e9' : '#38bdf8', /* Jus
                display: 'flex', alignItems: 'center', justifyContent: 'center'
                color: 'white', fontWeight: 800, fontSize: '15px', flexShrink:
              }}>
                {u.username?.charAt(0).toUpperCase()}
              </div>
              <div style={{ display: 'flex', flexDirection: 'column' }}>
                <span style={{ fontWeight: 700, color: 'var(--text-main)' }}>{
                  {isYou && <span style={{ fontSize: '0.75rem', color: '#0ea5e9'
                </div>
              </div>
            </td>
            <td style={{ color: 'var(--text-muted)' }}>{u.email || `${u.username
            <td>
              <span style={{
                padding: '4px 12px', borderRadius: 999, fontSize: '0.7rem', font
                border: `1.5px solid ${u.role === 'admin' ? 'var(--primary-accen
                color: u.role === 'admin' ? 'var(--primary-accent)' : 'var(--sec
                background: 'transparent'
              }}>
                {u.role.toUpperCase()}
              </span>
            </td>
            <td>
              <span style={{
                padding: '4px 12px', borderRadius: 999, fontSize: '0.7rem', font
                border: `1.5px solid var(--secondary-accent)` ,
                color: 'var(--secondary-accent)',
                background: 'transparent'
              }}>
                {u.is_active !== false ? 'ACTIVE' : 'INACTIVE'}
              </span>
            </td>
            <td style={{ color: 'var(--text-muted)', fontSize: '0.85rem' }}>
              {u.created_at ? u.created_at.split('T')[0] : '2026-04-07'}
            </td>

```

```
        </tr>
      );
    }}}
    {users.length === 0 && (
      <tr><td colSpan="5" style={{ textAlign: 'center', padding: '48px', color
    })
  </tbody>
</table>
  )}
</div>

  {showModal && <AddUserModal onClose={() => setShowModal(false)} onSuccess={handleC
</div>
  );
};

export default UsersPage;
```